MySQL Connector/J Developer Guide

Abstract

This manual describes how to install, configure, and develop database applications using MySQL Connector/J 8.3, a JDBC and X DevAPI driver for communicating with MySQL servers.

MySQL Connector/J 8.3 supersedes the 8.2 series and is recommended for use on production systems. It is highly recommended for use with MySQL Server 8.0 and up. Please upgrade to MySQL Connector/J 8.3.

For notes detailing the changes in each release of Connector/J, see MySQL Connector/J Release Notes.

For legal information, including licensing information, see the Preface and Legal Notices.

For help with using MySQL, please visit the MySQL Forums, where you can discuss your issues with other MySQL users.

Document generated on: 2024-03-18 (revision: 78098)

Table of Contents

Preface and Legal Notices v
1 Overview of MySQL Connector/J 1
2 Compatibility with MySQL and Java Versions 3
3 What's New in Connector/J 8.3? 5
4 Connector/J Installation 7
4.1 Installing Connector/J from a Binary Distribution 7
4.2 Installing Connector/J Using Maven 9
4.3 Installing from Source 9
4.4 Upgrading from an Older Version 11
4.4.1 Upgrading to MySQL Connector/J 8.3 from Connector/J 5.1 12
4.5 Testing Connector/J 16
5 Connector/J Examples 19
6 Connector/J Reference 21
6.1 Driver/Datasource Class Name 22
6.2 Connection URL Syntax 22
6.3 Configuration Properties 25
6.3.1 Authentication 32
6.3.2 Connection 34
6.3.3 Session 36
6.3.4 Networking 38
6.3.5 Security 40
6.3.6 Statements 44
6.3.7 Prepared Statements 45
6.3.8 Result Sets 46
6.3.9 Metadata 48
6.3.10 BLOB/CLOB processing 49
6.3.11 Datetime types processing 50
6.3.12 High Availability and Clustering 53
6.3.13 Performance Extensions 58
6.3.14 Debugging/Profiling 62
6.3.15 Exceptions/Warnings 64
6.3.16 Tunes for integration with other products 65
6.3.17 JDBC compliance 66
6.3.18 X Protocol and X DevAPI 66
6.4 JDBC API Implementation Notes 70
6.5 Java, JDBC, and MySQL Types 73
6.6 Handling of Date-Time Values 75
6.6.1 Preserving Time Instants 75
6.6.2 Fractional Seconds 80
6.6.3 Handling of YEAR Values 80
6.7 Using Character Sets and Unicode 80
6.8 Using Query Attributes 83
6.9 Connecting Securely Using SSL 85
6.9.1 Setting up Server Authentication 86
6.9.2 Setting up Client Authentication 88
6.9.3 Setting up 2-Way Authentication 89
6.9.4 JSSE in FIPS Mode 89
6.9.5 Debugging an SSL Connection 90
6.10 Connecting Using Unix Domain Sockets 90
6.11 Connecting Using Named Pipes 90
6.12 Connecting Using Various Authentication Methods 91
6.12.1 Connecting Using PAM Authentication 91
6.12.2 Connecting Using Kerberos 92
6.12.3 Connecting Using Multifactor Authentication 93
6.12.4 Connecting Using Fast Identity Online (FIDO) Authentication 93

6.12.5 Connecting Using Web Authentication (WebAuthn) Authentication 93
6.13 Using Source/Replica Replication with ReplicationConnection 98
6.14 Support for DNS SRV Records 98
6.15 Client Session State Tracker 99
6.16 Mapping MySQL Error Numbers to JDBC SQLState Codes 101
7 JDBC Concepts 107
7.1 Connecting to MySQL Using the JDBC `DriverManager` Interface 107
7.2 Using JDBC `Statement` Objects to Execute SQL 108
7.3 Using JDBC `CallableStatements` to Execute Stored Procedures 109
7.4 Retrieving `AUTO_INCREMENT` Column Values through JDBC 112
8 Connection Pooling with Connector/J 117
9 Multi-Host Connections 121
9.1 Configuring Server Failover for Connections Using JDBC 121
9.2 Configuring Server Failover for Connections Using X DevAPI 124
9.3 Configuring Load Balancing with Connector/J 124
9.4 Configuring Source/Replica Replication with Connector/J 127
9.5 Advanced Load-balancing and Failover Configuration 130
10 Using the X DevAPI with Connector/J: Special Topics 133
10.1 Connection Compression Using X DevAPI 133
10.2 Schema Validation 134
11 Using the Connector/J Interceptor Classes 137
12 Using Logging Frameworks with SLF4J 139
13 Using Connector/J with Tomcat 141
14 Using Connector/J with Spring 143
14.1 Using `JdbcTemplate` 144
14.2 Transactional JDBC Access 145
14.3 Connection Pooling with Spring 147
15 Troubleshooting Connector/J Applications 149
16 Known Issues and Limitations 157
17 Connector/J Support 159
17.1 Connector/J Community Support 159
17.2 How to Report Connector/J Bugs or Problems 159
Index 161

Preface and Legal Notices

This manual describes how to install, configure, and develop database applications using MySQL Connector/J, the JDBC driver for communicating with MySQL servers.

Licensing information. This product may include third-party software, used under license. If you are using a *Commercial* release of MySQL Connector/J 8.3, see the MySQL Connector/J 8.3 Commercial License Information User Manual for licensing information, including licensing information relating to third-party software that may be included in this Commercial release. If you are using a *Community* release of MySQL Connector/J 8.3, see the MySQL Connector/J 8.3 Community License Information User Manual for licensing information, including licensing information relating to third-party software that may be included in this Community release.

Legal Notices

Copyright © 1998, 2024, Oracle and/or its affiliates.

License Restrictions

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

Warranty Disclaimer

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

Restricted Rights Notice

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

Hazardous Applications Notice

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Trademark Notice

Oracle, Java, MySQL, and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

Third-Party Content, Products, and Services Disclaimer

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

Use of This Documentation

This documentation is NOT distributed under a GPL license. Use of this documentation is subject to the following terms:

You may create a printed copy of this documentation solely for your own personal use. Conversion to other formats is allowed as long as the actual content is not altered or edited in any way. You shall not publish or distribute this documentation in any form or on any media, except if you distribute the documentation in a manner similar to how Oracle disseminates it (that is, electronically for download on a Web site with the software) or on a CD-ROM or similar medium, provided however that the documentation is disseminated together with the software on the same medium. Any other use, such as any dissemination of printed copies or use of this documentation, in whole or in part, in another publication, requires the prior written consent from an authorized representative of Oracle. Oracle and/or its affiliates reserve any and all rights to this documentation not expressly granted above.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at
http://www.oracle.com/pls/topic/lookup?ctx=acc&id=docacc.

Access to Oracle Support for Accessibility

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit
http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info or visit `http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs` if you are hearing impaired.

Chapter 1 Overview of MySQL Connector/J

MySQL provides connectivity for client applications developed in the Java programming language with MySQL Connector/J. Connector/J implements the Java Database Connectivity (JDBC) API, as well as a number of value-adding extensions of it. It also supports the new X DevAPI.

MySQL Connector/J is a JDBC Type 4 driver, implementing the JDBC 4.2 specification. The Type 4 designation means that the driver is a pure Java implementation of the MySQL protocol and does not rely on the MySQL client libraries. See Chapter 2, *Compatibility with MySQL and Java Versions* for compatibility information.

Connector/J 8.3 provides ease of development features including auto-registration with the Driver Manager, standardized validity checks, categorized SQLExceptions, support for large update counts, support for local and offset date-time variants from the `java.time` package, support for JDBC-4.x XML processing, support for per connection client information, and support for the `NCHAR`, `NVARCHAR` and `NCLOB` data types. See Chapter 2, *Compatibility with MySQL and Java Versions* for compatibility information.

For large-scale programs that use common design patterns of data access, consider using one of the popular persistence frameworks such as Hibernate, Spring's JDBC templates or MyBatis SQL Maps to reduce the amount of JDBC code for you to debug, tune, secure, and maintain.

Key Topics

- For installation instructions for Connector/J, see Chapter 4, *Connector/J Installation*.
- For help with connection strings, connection options, and setting up your connection through JDBC, see Chapter 6, *Connector/J Reference*.
- For information on connection pooling, see Chapter 8, *Connection Pooling with Connector/J*.
- For information on multi-host connections, see Chapter 9, *Multi-Host Connections*.
- For information on using the X DevAPI with Connector/J, see Chapter 10, *Using the X DevAPI with Connector/J: Special Topics*.

Chapter 2 Compatibility with MySQL and Java Versions

Here is some compatibility information for Connector/J 8.3:

- **JDBC versions**: Connector/J 8.3 implements JDBC 4.2. While Connector/J 8.3 works with libraries of higher JDBC versions, it returns a `SQLFeatureNotSupportedException` for any calls of methods supported only by JDBC 4.3 and higher.
- **MySQL Server versions**: Connector/J 8.3 supports MySQL 8.0 and up.
- **JRE versions**: Connector/J 8.3 supports JRE 8 or higher.
- **JDK Required for Compilation**: JDK 8.0 or higher is required for compiling Connector/J 8.3. Also, a customized JSSE provider might be required to use some later TLS versions and cipher suites when connecting to MySQL servers. For example, because Oracle's Java 8 releases before 8u261 were shipped with JSSE implementations that support TLS up to version 1.2 only, you need a customized JSSE implementation to use TLSv1.3 on those Java 8 platforms. Oracle Java 8u261 and above do support TLSv1.3, so no customized JSSE implementation is needed.

Chapter 3 What's New in Connector/J 8.3?

Version 8.3.0 is a new GA release version of the MySQL Connector/J. MySQL Connector/J 8.3.0 supersedes the 8.2 series and is recommended for use on production systems. This release can be used against MySQL Server version 8.0 and up. It supports the Java Database Connectivity (JDBC) 4.2 API, and implements the X DevAPI.

For notes detailling the changes in Connector/J 8.3, see MySQL Connector/J Release Notes

Chapter 4 Connector/J Installation

Table of Contents

4.1 Installing Connector/J from a Binary Distribution 7
4.2 Installing Connector/J Using Maven 9
4.3 Installing from Source 9
4.4 Upgrading from an Older Version 11
4.4.1 Upgrading to MySQL Connector/J 8.3 from Connector/J 5.1 12
4.5 Testing Connector/J 16

You can install the Connector/J package using either a binary or source distribution. While the binary distribution provides the easiest method for installation, the source distribution lets you customize your installation. Both types of distributions are available from the Connector/J Download page. The source code for Connector/J is also available on GitHub at https://github.com/mysql/mysql-connector-j.

Connector/J is also available as a Maven artifact in the Central Repository. See Section 4.2, “Installing Connector/J Using Maven” for details.

If you are upgrading from a previous version, read the upgrade information in Section 4.4, “Upgrading from an Older Version” before continuing.

> **Important**
>
> *Third-party Libraries:* According to how you use Connector/J 8.3, you may also need to install the following third-party libraries on your system for it to work:
>
> - Protocol Buffers (`protobuf-java`) 3.25.1 is required for using X DevAPI
> - Oracle Cloud Infrastructure SDK for Java (`oci-java-sdk`) 3.29.0 is required to support OCI AIM authentication
> - Simple Logging Facade API (`slf4j-api`) 2.0.9 is required for using the logging capabilities provided by the default implementation of `org.slf4j.Logger.Slf4JLogger` by Connector/J
>
> These and other third-party libraries are required for building Connector/J from source—see the section for more information.

4.1 Installing Connector/J from a Binary Distribution

Obtaining and Using the Binary Distribution Packages

Different types of binary distribution packages for Connector/J are available from the Connector/J Download page. The following explains how to use each type of the packages to install Connector/J.

Using Platform-independent Archives: `.tar.gz` or `.zip` archives are available for installing Connector/J on any platform. Using the appropriate graphical or command-line utility (for example, `tar` for the `.tar.gz` archive and `WinZip` for the .zip archive), extract the JAR archive from the `.tar.gz` or `.zip` archive to a suitable location.

> **Note**
>
> Because there are potentially long file names in the distribution, the Connector/J archives use the GNU Tar archive format. Use GNU Tar or a compatible application to unpack the `.tar.gz` variant of the distribution.

Using Packages for Software Package Management Systems on Linux Platforms: RPM and Debian packages are available for installing Connector/J on a number of Linux distributions like Oracle Linux,

Debian, Ubuntu, SUSE, and so on. Install these packages using your system's software package management system.

On Windows Platforms: You cannot install Connector/J on Windows platforms using the MySQL Installer for Windows. Notice that there are also no stand-alone Windows installer files (.msi) for installing Connector/J. Use the platform-independent archives instead for installations on Windows platforms.

Configuring the CLASSPATH

Once `mysql-connector-j-version.jar` has been extracted from the binary distribution package to the right place, finish installing the driver by placing the JAR archive in your Java classpath, either by adding its full file path to your `CLASSPATH` environment variable, or by directly specifying the file path with the command line switch `-cp` when starting the JVM.

For example, on Linux platforms, add the Connector/J driver to your `CLASSPATH` using one of the following forms, depending on your command shell:

```
# Bourne-compatible shell (sh, ksh, bash, zsh):
$> export CLASSPATH=/path/mysql-connector-j-ver.jar:$CLASSPATH

# C shell (csh, tcsh):
$> setenv CLASSPATH /path/mysql-connector-j-ver.jar:$CLASSPATH
```

You can also set the `CLASSPATH` environment variable in a profile file, either locally for a user within the user's `.profile`, `.login`, or other login file, or globally by editing the global `/etc/profile` file.

For Windows platforms, you set the environment variable through the System Control Panel.

> **Important**
>
> Remember to also add the locations of the third-party libraries required for using Connector/J to CLASSPATH.

Configuring Connector/J for Application Servers

To use MySQL Connector/J with an application server such as GlassFish or Tomcat, read your vendor's documentation for information on how to configure third-party class libraries, as most application servers ignore the `CLASSPATH` environment variable. For configuration examples for some J2EE application servers, see Chapter 8, *Connection Pooling with Connector/J*, Section 9.3, "Configuring Load Balancing with Connector/J", and Section 9.5, "Advanced Load-balancing and Failover Configuration". However, the authoritative source for JDBC connection pool configuration information is the documentation for your own application server.

If you are developing servlets or JSPs and your application server is J2EE-compliant, you can put the driver's `.jar` file in the `WEB-INF/lib` subdirectory of your web application, as this is a standard location for third-party class libraries in J2EE web applications. You can also use the `MysqlDataSource` or `MysqlConnectionPoolDataSource` classes in the `com.mysql.cj.jdbc` package, if your J2EE application server supports or requires them. The `javax.sql.XADataSource` interface is implemented using the `com.mysql.cj.jdbc.MysqlXADataSource` class, which supports XA distributed transactions. The various `MysqlDataSource` classes support the following parameters (through standard set mutators):

- `user`
- `password`
- `serverName`
- `databaseName`
- `port`

4.2 Installing Connector/J Using Maven

You can also use Maven dependencies manager to install and configure the Connector/J library in your project. Connector/J is published in The Maven Central Repository with the following groupId and artifactId:

- groupId: `com.mysql`
- artifactId: `mysql-connector-j`

You can link the Connector/J library to your project by adding the following dependency in your `pom.xml` file:

```
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>x.y.z</version>
</dependency>
```

Notice that if you use Maven to manage your project dependencies, you do not need to explicitly refer to the library `protobuf-java` as it is resolved by dependency transitivity. However, if you do *not* want to use the X DevAPI features, you may also want to add a dependency exclusion to avoid linking the unneeded sub-library. For example:

```
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>x.y.z</version>
    <exclusions>
        <exclusion>
            <groupId>com.google.protobuf</groupId>
            <artifactId>protobuf-java</artifactId>
        </exclusion>
    </exclusions>
</dependency>
```

4.3 Installing from Source

> **Caution**
>
> You need to install Connector/J from source only if you want to build a customized version of Connector/J or if you are interested in helping us test our new code. To just get MySQL Connector/J up and running on your system, install Connector/J using a standard binary release distribution; see Section 4.1, “Installing Connector/J from a Binary Distribution” for instructions.

To install MySQL Connector/J from source, make sure that you have the following software on your system:

> **Tip**
>
> It is suggested that the latest versions available for the following software be used for compiling Connector/J; otherwise, some features might not be available.

- A Git client, if you want to check out the sources from our GitHub repository (available from http://git-scm.com/downloads).
- Apache Ant version 1.10.6 or newer (available from http://ant.apache.org/).
- JDK 1.8.x (available from https://www.oracle.com/technetwork/java/javase/downloads/jdk8-downloads-2133151.html).
- The following third-party libraries:

- JUnit 5.10 (see installation and download information in the JUnit 5 User Guide). The following JAR files are required:
  - `junit-jupiter-api-5.10.1.jar` (available from, for example, https://search.maven.org/artifact/org.junit.jupiter/junit-jupiter-api/5.10.1/jar).
  - `junit-jupiter-engine-5.10.1.jar` (available from, for example, https://search.maven.org/artifact/org.junit.jupiter/junit-jupiter-engine/5.10.1/jar).
  - `junit-platform-commons-1.10.1.jar` (available from, for example, https://search.maven.org/artifact/org.junit.platform/junit-platform-commons/1.10.1/jar).
  - `junit-platform-engine-1.10.1.jar` (available from, for example, https://search.maven.org/artifact/org.junit.platform/junit-platform-engine/1.10.1/jar).
  - `junit-platform-launcher-1.10.1.jar` (available from, for example, https://search.maven.org/artifact/org.junit.platform/junit-platform-launcher/1.10.1/jar).
  - These additional JAR files, which JUnit 5 depends on:
    - `apiguardian-api-1.1.2.jar` (available from, for example, https://search.maven.org/artifact/org.apiguardian/apiguardian-api/1.1.2/jar).
    - `opentest4j-1.3.0.jar` (available from, for example, https://search.maven.org/artifact/org.opentest4j/opentest4j/1.3.0/jar).
- Javassist 3.29.2 (`javassist 3.29.2-GA.jar`, available from, for example, https://search.maven.org/artifact/org.javassist/javassist/3.29.2-GA/bundle).
- Protocol Buffers Java API 3.25.1 (`protobuf-java-3.25.1.jar`, available from, for example, https://search.maven.org/artifact/com.google.protobuf/protobuf-java/3.25.1/bundle).
- Simple Logging Facade API 2.0.9 or newer (`slf4j-api-2.0.9.jar`, available from, for example, https://search.maven.org/artifact/org.slf4j/slf4j-api/2.0.9/jar).
- Java Hamcrest 2.2 or newer (`hamcrest-2.2.jar`, available from, for example, https://search.maven.org/artifact/org.hamcrest/hamcrest/2.2/jar).
- Oracle Cloud Infrastructure SDK for Java (`oci-java-sdk-common-3.29.0.jar`, available from, for example, https://search.maven.org/artifact/com.oracle.oci.sdk/oci-java-sdk-common/3.29.0/jar).

To build MySQL Connector/J from source, follow these steps:

1. Make sure that you have JDK 1.8.x installed.
2. Obtain the sources for Connector/J by one of the following means:
   - Download the platform independent distribution archive (in `.tar.gz` or `.zip` format) for Connector/J, which contains the sources, from the Connector/J Download page. Extract contents of the archive into a folder named, for example, `mysql-connector-j`.
   - Download a source RPM package for Connector/J from Connector/J Download page and install it.
   - Check out the code from the source code repository for MySQL Connector/J located on GitHub at https://github.com/mysql/mysql-connector-j. The latest release of the Connector/J 8.3 series is on the `release/8.3` branch; use the following command to check it out:

     ```
     $> git clone --branch release/8.3 https://github.com/mysql/mysql-connector-j.git
     ```

Under the current directory, the command creates a `mysql-connector-j` subdirectory , which contains the code you want.

3. Place all the required third-party libraries in a the directory called `lib` at the root of the source tree (that is, in `mysql-connector-j/lib`, if you have followed the steps above), or put them elsewhere and supply the location to Ant later (see Step 5 below).

4. Change your current working directory to the `mysql-connector-j` directory created in step 2 above.

5. In the directory, create a file named `build.properties` to indicate to Ant the location of the root directory for your JDK 1.8.x installation with the property `com.mysql.cj.build.jdk`, as well as the location for the extra libraries, if they are not in `mysql-connector-j/lib`, with the property `com.mysql.cj.extra.libs`. Here is a sample file with those properties set (replace the "*`path_to_*`*" parts with the appropriate file paths):

   ```
   com.mysql.cj.build.jdk=path_to_jdk_1.8
   com.mysql.cj.extra.libs=path_to_folder_for_extra_libraries
   ```

   Alternatively, you can set the values of those properties through the Ant `-D` options.

   > **Note**
   >
   > Going from Connector/J 5.1 to 8.0 and beyond, a number of Ant properties for building Connector/J have been renamed or removed; see Section 4.4.1.4, "Changes for Build Properties" for details.

6. Issue the following command to compile the driver and create a `.jar` file for Connector/J:

   ```
   $> ant build
   ```

   This creates a `build` directory in the current directory, where all the build output goes. A directory is created under the `build` directory, whose name includes the version number of the release you are building. That directory contains the sources, the compiled `.class` files, and a `.jar` file for deployment.

   For information on all the build targets, including those that create a fully packaged distribution, issue the following command:

   ```
   $> ant -projecthelp
   ```

7. Install the newly created `.jar` file for the JDBC driver as you would install a binary `.jar` file you download from MySQL by following the instructions given in Configuring the CLASSPATH or Configuring Connector/J for Application Servers.

4.4 Upgrading from an Older Version

This section has information for users who are upgrading from one version of Connector/J to another, or to a new version of the MySQL server that supports a more recent level of JDBC. A newer version of Connector/J might include changes to support new features, improve existing functionality, or comply with new standards.

Depending on the platform and the way you used to install Connector/J, upgrading can be performed by one of the following methods:

- Downloading a new platform-independent archive (`.tar`, `.tar.gz`, `.zip`, etc.) and overwriting with it your original installation created by an older archive.
- Updating the version of the Connector/J dependency in your Maven `.pom` file.
- Using the upgrade command of your Linux distro's package management system.

- Using the MySQL Installer for Windows, which can also perform automatic updates for Connector/J

See Chapter 4, *Connector/J Installation* for details on the installation and upgrade methods. You should also pay attention to any important changes in the new version like changes in 3rd-party dependencies, incompatibilities, etc.

4.4.1 Upgrading to MySQL Connector/J 8.3 from Connector/J 5.1

Upgrading an application developed for Connector/J 5.1 to use Connector/J 8.0 and beyond might require certain changes to your code or the environment in which it runs. Here are some changes for Connector/J going from 5.1 to 8.0 and beyond, for which adjustments might be required:

4.4.1.1 Running on the Java 8 Platform

Connector/J 8.0 and beyond is created specifically to run on the Java 8 platform. While Java 8 is known to be strongly compatible with earlier Java versions, incompatibilities do exist, and code designed to work on Java 7 might need to be adjusted before being run on Java 8. Developers should refer to the incompatibility information provided by Oracle.

4.4.1.2 Changes in Connection Properties

A complete list of Connector/J 8.3 connection properties are available in Section 6.3, “Configuration Properties”. The following are connection properties that have been changed (removed, added, have their names changed, or have their default values changed) going from Connector/J 5.1 to 8.0 and beyond.

Properties that have been removed (do not use them during connection):

- `useDynamicCharsetInfo`
- `useBlobToStoreUTF8OutsideBMP` , `utf8OutsideBmpExcludedColumnNamePattern`, and `utf8OutsideBmpIncludedColumnNamePattern`: MySQL 5.6 and later supports the utf8mb4 character set, which is the character set that should be used by Connector/J applications for supporting characters beyond the Basic Multilingual Plane (BMP) of Unicode Version 3.
- `useJvmCharsetConverters`: JVM character set conversion is now used in all cases
- The following date and time properties:
  - `dynamicCalendars`
  - `noTzConversionForTimeType`
  - `noTzConversionForDateType`
  - `cacheDefaultTimezone`
  - `useFastIntParsing`
  - `useFastDateParsing`
  - `useJDBCCompliantTimezoneShift`
  - `useLegacyDatetimeCode`
  - `useSSPSCompatibleTimezoneShift`
  - `useTimezone`
  - `useGmtMillisForDatetimes`
- `dumpMetadataOnColumnNotFound`

- `relaxAutoCommit`
- `strictFloatingPoint`
- `runningCTS13`
- `retainStatementAfterResultSetClose`
- `nullNamePatternMatchesAll`

Properties that have been added:

- `mysqlx.useAsyncProtocol` (deprecated)

Property that has its name changed:

- `com.mysql.jdbc.faultInjection.serverCharsetIndex` changed to `com.mysql.cj.testsuite.faultInjection.serverCharsetIndex`
- `loadBalanceEnableJMX` to `ha.enableJMX`
- `replicationEnableJMX` to `ha.enableJMX`

Properties that have their default values changed:

- `nullCatalogMeansCurrent` is now `false` by default

4.4.1.3 Changes in the Connector/J API

This section describes some of the more important changes to the Connector/J API going from version 5.1 to 8.0 and beyond. You might need to adjust your API calls accordingly:

- The name of the class that implements `java.sql.Driver` in MySQL Connector/J has changed from `com.mysql.jdbc.Driver` to `com.mysql.cj.jdbc.Driver`. The old class name has been deprecated.
- The names of these commonly-used classes and interfaces have also been changed:
  - ExceptionInterceptor: from `com.mysql.jdbc.ExceptionInterceptor` to `com.mysql.cj.exceptions.ExceptionInterceptor`
  - StatementInterceptor: from `com.mysql.jdbc.StatementInterceptorV2` to `com.mysql.cj.interceptors.QueryInterceptor`
  - ConnectionLifecycleInterceptor: from `com.mysql.jdbc.ConnectionLifecycleInterceptor` to `com.mysql.cj.jdbc.interceptors.ConnectionLifecycleInterceptor`
  - AuthenticationPlugin: from `com.mysql.jdbc.AuthenticationPlugin` to `com.mysql.cj.protocol.AuthenticationPlugin`
  - BalanceStrategy: from `com.mysql.jdbc.BalanceStrategy` to `com.mysql.cj.jdbc.ha.BalanceStrategy`
  - MysqlDataSource: from `com.mysql.jdbc.jdbc2.optional.MysqlDataSource` to `com.mysql.cj.jdbc.MysqlDataSource`
  - MysqlDataSourceFactory: from `com.mysql.jdbc.jdbc2.optional.MysqlDataSourceFactory` to `com.mysql.cj.jdbc.MysqlDataSourceFactory`
  - MysqlConnectionPoolDataSource: from `com.mysql.jdbc.jdbc2.optional.MysqlConnectionPoolDataSource` to `com.mysql.cj.jdbc.MysqlConnectionPoolDataSource`

- MysqlXADataSource: from `com.mysql.jdbc.jdbc2.optional.MysqlXADataSource` to `com.mysql.cj.jdbc.MysqlXADataSource`
- MysqlXid: from `com.mysql.jdbc.jdbc2.optional.MysqlXid` to `com.mysql.cj.jdbc.MysqlXid`

4.4.1.4 Changes for Build Properties

A number of Ant properties for building Connector/J from source have been renamed; see Table 4.1, "Changes with the Build Properties from Connector/J 5.1 to 8.0 and Beyond"

Table 4.1 Changes with the Build Properties from Connector/J 5.1 to 8.0 and Beyond

Old name	New name
com.mysql.jdbc.extra.libs	com.mysql.cj.extra.libs
com.mysql.jdbc.jdk	com.mysql.cj.build.jdk
debug.enable	com.mysql.cj.build.addDebugInfo
com.mysql.jdbc.noCleanBetweenCompiles	com.mysql.cj.build.noCleanBetweenCompiles
com.mysql.jdbc.commercialBuild	com.mysql.cj.build.commercial
com.mysql.jdbc.filterLicense	com.mysql.cj.build.filterLicense
com.mysql.jdbc.noCryptoBuild	com.mysql.cj.build.noCrypto
com.mysql.jdbc.noSources	com.mysql.cj.build.noSources
com.mysql.jdbc.noMavenSources	com.mysql.cj.build.noMavenSources
major_version	com.mysql.cj.build.driver.version.major
minor_version	com.mysql.cj.build.driver.version.minor
subminor_version	com.mysql.cj.build.driver.version.subminor
version_status	com.mysql.cj.build.driver.version.status
extra.version	com.mysql.cj.build.driver.version.extra
snapshot.version	com.mysql.cj.build.driver.version.snapshot
version	com.mysql.cj.build.driver.version
full.version	com.mysql.cj.build.driver.version.full
prodDisplayName	com.mysql.cj.build.driver.displayName
prodName	com.mysql.cj.build.driver.name
fullProdName	com.mysql.cj.build.driver.fullName
buildDir	com.mysql.cj.build.dir
buildDriverDir	com.mysql.cj.build.dir.driver
mavenUploadDir	com.mysql.cj.build.dir.maven
distDir	com.mysql.cj.dist.dir
toPackage	com.mysql.cj.dist.dir.prepare
packageDest	com.mysql.cj.dist.dir.package
com.mysql.jdbc.docs.sourceDir	com.mysql.cj.dist.dir.prebuilt.docs

4.4.1.5 Change for Test Properties

A number of Ant properties for testing Connector/J have been renamed or removed; see Table 4.2, "Changes with the Test Properties from Connector/J 5.1 to 8.0 and Beyond"

Table 4.2 Changes with the Test Properties from Connector/J 5.1 to 8.0 and Beyond

Old name	New name
buildTestDir	com.mysql.cj.testsuite.build.dir
junit.results	com.mysql.cj.testsuite.junit.results
com.mysql.jdbc.testsuite.jvm	com.mysql.cj.testsuite.jvm
test	com.mysql.cj.testsuite.test.class
methods	com.mysql.cj.testsuite.test.methods
com.mysql.jdbc.testsuite.url	com.mysql.cj.testsuite.url
com.mysql.jdbc.testsuite.admin-url	com.mysql.cj.testsuite.url.admin
com.mysql.jdbc.testsuite.ClusterUrl	com.mysql.cj.testsuite.url.cluster
com.mysql.jdbc.testsuite.url.sha256default	com.mysql.cj.testsuite.url.openssl
com.mysql.jdbc.testsuite.cantGrant	com.mysql.cj.testsuite.cantGrant
com.mysql.jdbc.testsuite.no-multi-hosts-tests	com.mysql.cj.testsuite.disable.multihost.tes
com.mysql.jdbc.test.ds.host	com.mysql.cj.testsuite.ds.host
com.mysql.jdbc.test.ds.port	com.mysql.cj.testsuite.ds.port
com.mysql.jdbc.test.ds.db	com.mysql.cj.testsuite.ds.db
com.mysql.jdbc.test.ds.user	com.mysql.cj.testsuite.ds.user
com.mysql.jdbc.test.ds.password	com.mysql.cj.testsuite.ds.password
com.mysql.jdbc.test.tabletype	com.mysql.cj.testsuite.loadstoreperf.tablety
com.mysql.jdbc.testsuite.loadstoreperf.useBigResults	com.mysql.cj.testsuite.loadstoreperf.useBigR
com.mysql.jdbc.testsuite.MiniAdminTest.runShutdown	com.mysql.cj.testsuite.miniAdminTest.runShut
com.mysql.jdbc.testsuite.noDebugOutput	com.mysql.cj.testsuite.noDebugOutput
com.mysql.jdbc.testsuite.retainArtifacts	com.mysql.cj.testsuite.retainArtifacts
com.mysql.jdbc.testsuite.runLongTests	com.mysql.cj.testsuite.runLongTests
com.mysql.jdbc.test.ServerController.basedir	com.mysql.cj.testsuite.serverController.base
com.mysql.jdbc.ReplicationConnection.isSlave	com.mysql.cj.testsuite.replicationConnection
com.mysql.jdbc.test.isLocalHostnameReplacement	Removed
com.mysql.jdbc.testsuite.driver	Removed
com.mysql.jdbc.testsuite.url.default	Removed. No longer needed, as multi-JVM tests have been removed from the test suite.

4.4.1.6 Changes for Exceptions

Some exceptions have been removed from Connector/J going from version 5.1 to 8.0 and beyond. Applications that used to catch the removed exceptions should now catch the corresponding exceptions listed in Table 4.3 below.

> **Note**
>
> Some of these Connector/J 5.1 exceptions are duplicated in the com.mysql.jdbc.exception.jdbc4 package; that is indicated by “[jdbc4.]” in their names in Table 4.3.

Table 4.3 Changes for Exceptions from Connector/J 5.1 to 8.0 and Beyond

Removed Exception in Connector/J 5.1
com.mysql.jdbc.exceptions.jdbc4.CommunicationsException

Removed Exception in Connector/J 5.1
com.mysql.jdbc.exceptions.[jdbc4.]MySQLDataException
com.mysql.jdbc.exceptions.[jdbc4.]MySQLIntegrityConstraintViolationException
com.mysql.jdbc.exceptions.[jdbc4.]MySQLInvalidAuthorizationSpecException
com.mysql.jdbc.exceptions.[jdbc4.]MySQLNonTransientConnectionException
com.mysql.jdbc.exceptions.[jdbc4.]MySQLNonTransientException
com.mysql.jdbc.exceptions.[jdbc4.]MySQLQueryInterruptedException
com.mysql.jdbc.exceptions.MySQLStatementCancelledException
com.mysql.jdbc.exceptions.[jdbc4.]MySQLSyntaxErrorException
com.mysql.jdbc.exceptions.[jdbc4.]MySQLTimeoutException
com.mysql.jdbc.exceptions.[jdbc4.]MySQLTransactionRollbackException
com.mysql.jdbc.exceptions.[jdbc4.]MySQLTransientConnectionException
com.mysql.jdbc.exceptions.[jdbc4.]MySQLTransientException
com.mysql.jdbc.exceptions.[jdbc4.]MySQLIntegrityConstraintViolationException

4.4.1.7 Other Changes

Here are other changes with Connector/J 8.0 and beyond:

- Removed `ReplicationDriver`. Instead of using a separate driver, you can now obtain a connection for a replication setup just by using the `jdbc:mysql:replication://` scheme.
- See Chapter 4, *Connector/J Installation* for third-party libraries required for Connector/J 8.3 to work.
- *For Connector/J 8.0.22 and earlier:* Connector/J always performs time offset adjustments on date-time values, and the adjustments require one of the following to be true:
  - The MySQL server is configured with a canonical time zone that is recognizable by Java (for example, Europe/Paris, Etc/GMT-5, UTC, etc.)
  - The server's time zone is overridden by setting the Connector/J connection property `serverTimezone` (for example, `serverTimezone=Europe/Paris`).

    > **Note**
    >
    > The Connector/J's behavior in this respect has changed since release 8.0.23. See Section 6.6.1, “Preserving Time Instants” for details. `serverTimezone` is now an alias for the connection property `connectionTimeZone`, which has replaced `serverTimezone`.

4.5 Testing Connector/J

The Connector/J source code repository or packages that are shipped with source code include an extensive test suite, containing test cases that can be executed independently. The test cases are divided into the following categories:

- *Unit tests:* They are methods located in packages aligning with the classes that they test.
- *Functional tests:* Classes from the package `testsuite.simple`. Include test code for the main features of Connector/J.
- *Performance tests:* Classes from the package `testsuite.perf`. Include test code to make measurements for the performance of Connector/J.
- *Regression tests:* Classes from the package `testsuite.regression`. Includes code for testing bug and regression fixes.

- *X DevAPI and X Protocol tests:* Classes from the package `testsuite.x` for testing X DevAPI and X Protocol functionality.

The bundled Ant build file contains targets like `test`, which can facilitate the process of running the Connector/J tests; see the target descriptions in the build file for details. To run the tests, in addition to fulfilling the requirements described in Section 4.3, “Installing from Source”, you must also set the following properties in the `build.properties` file or through the Ant `-D` options:

- `com.mysql.cj.testsuite.jvm`: the JVM to be used for the tests. If the property is not set, the JVM supplied with `com.mysql.cj.build.jdk` will be used.
- `com.mysql.cj.testsuite.url`: it specifies the JDBC URL for connection to a MySQL test server; see Section 6.2, “Connection URL Syntax”.
- `com.mysql.cj.testsuite.url.openssl`: (*for release 8.0.26 and earlier only*) it specifies the JDBC URL for connection to a MySQL test server compiled with OpenSSL; see Section 6.2, “Connection URL Syntax”.
- `com.mysql.cj.testsuite.mysqlx.url`: it specifies the X DevAPI URL for connection to a MySQL test server; see Section 6.2, “Connection URL Syntax”.
- `com.mysql.cj.testsuite.mysqlx.url.openssl`: (*for release 8.0.26 and earlier only*) it specifies the X DevAPI URL for connection to a MySQL test server compiled with OpenSSL; see Section 6.2, “Connection URL Syntax”.

After setting these parameters, run the tests with Ant in the following ways:

- Building the `test` target with `ant test` runs all test cases by default on a single server instance. If you want to run a particular test case, put the test's fully qualified class names in the `com.mysql.cj.testsuite.test.class` variable; for example:

```
shell > ant -Dcom.mysql.cj.testsuite.test.class=testsuite.simple.StringUtilsTest test
```

  You can also run individual tests in a test case by specifying the names of the corresponding methods in the `com.mysql.cj.testsuite.test.methods` variable, separating multiple methods by commas; for example:

```
shell > ant -Dcom.mysql.cj.testsuite.test.class=testsuite.simple.StringUtilsTest \
-Dcom.mysql.cj.testsuite.test.methods=testIndexOfIgnoreCase,testGetBytes test
```

While the test results are partially reported by the console, complete reports in HTML and XML formats are provided. View the HTML report by opening `buildtest/junit/report/index.html`. XML version of the reports are located in the folder `buildtest/junit`.

> **Note**
>
> Going from Connector/J 5.1 to 8.0 and beyond, a number of Ant properties for testing Connector/J have been renamed or removed; see Section 4.4.1.5, “Change for Test Properties” for details.

Chapter 5 Connector/J Examples

Examples of using Connector/J are located throughout this document. This section provides a summary and links to these examples.

- Example 7.1, “Connector/J: Obtaining a connection from the `DriverManager`”
- Example 7.2, “Connector/J: Using java.sql.Statement to execute a `SELECT` query”
- Example 7.3, “Connector/J: Calling Stored Procedures”
- Example 7.4, “Connector/J: Using `Connection.prepareCall()`”
- Example 7.5, “Connector/J: Registering output parameters”
- Example 7.6, “Connector/J: Setting `CallableStatement` input parameters”
- Example 7.7, “Connector/J: Retrieving results and output parameter values”
- Example 7.8, “Connector/J: Retrieving `AUTO_INCREMENT` column values using `Statement.getGeneratedKeys()`”
- Example 7.9, “Connector/J: Retrieving `AUTO_INCREMENT` column values using `SELECT LAST_INSERT_ID()`”
- Example 7.10, “Connector/J: Retrieving `AUTO_INCREMENT` column values in `Updatable ResultSets`”
- Example 8.1, “Connector/J: Using a connection pool with a J2EE application server”
- Example 15.1, “Connector/J: Example of transaction with retry logic”

Chapter 6 Connector/J Reference

Table of Contents

6.1 Driver/Datasource Class Name 22
6.2 Connection URL Syntax 22
6.3 Configuration Properties 25
6.3.1 Authentication 32
6.3.2 Connection 34
6.3.3 Session 36
6.3.4 Networking 38
6.3.5 Security 40
6.3.6 Statements 44
6.3.7 Prepared Statements 45
6.3.8 Result Sets 46
6.3.9 Metadata 48
6.3.10 BLOB/CLOB processing 49
6.3.11 Datetime types processing 50
6.3.12 High Availability and Clustering 53
6.3.13 Performance Extensions 58
6.3.14 Debugging/Profiling 62
6.3.15 Exceptions/Warnings 64
6.3.16 Tunes for integration with other products 65
6.3.17 JDBC compliance 66
6.3.18 X Protocol and X DevAPI 66
6.4 JDBC API Implementation Notes 70
6.5 Java, JDBC, and MySQL Types 73
6.6 Handling of Date-Time Values 75
6.6.1 Preserving Time Instants 75
6.6.2 Fractional Seconds 80
6.6.3 Handling of YEAR Values 80
6.7 Using Character Sets and Unicode 80
6.8 Using Query Attributes 83
6.9 Connecting Securely Using SSL 85
6.9.1 Setting up Server Authentication 86
6.9.2 Setting up Client Authentication 88
6.9.3 Setting up 2-Way Authentication 89
6.9.4 JSSE in FIPS Mode 89
6.9.5 Debugging an SSL Connection 90
6.10 Connecting Using Unix Domain Sockets 90
6.11 Connecting Using Named Pipes 90
6.12 Connecting Using Various Authentication Methods 91
6.12.1 Connecting Using PAM Authentication 91
6.12.2 Connecting Using Kerberos 92
6.12.3 Connecting Using Multifactor Authentication 93
6.12.4 Connecting Using Fast Identity Online (FIDO) Authentication 93
6.12.5 Connecting Using Web Authentication (WebAuthn) Authentication 93
6.13 Using Source/Replica Replication with ReplicationConnection 98
6.14 Support for DNS SRV Records 98
6.15 Client Session State Tracker 99
6.16 Mapping MySQL Error Numbers to JDBC SQLState Codes 101

This section of the manual contains reference material for MySQL Connector/J.

6.1 Driver/Datasource Class Name

The name of the class that implements `java.sql.Driver` in MySQL Connector/J is `com.mysql.cj.jdbc.Driver`.

6.2 Connection URL Syntax

This section explains the syntax of the URLs for connecting to MySQL.

This is the generic format of the connection URL:

```
protocol//[hosts][/database][?properties]
```

The URL consists of the following parts:

> **Important**
>
> Any reserved characters for URLs (for example, `/`, `:`, `@`, `(`, `)`, `[`, `]`, `&`, `#`, `=`, `?`, and space) that appear in any part of the connection URL must be percent encoded.

protocol

There are the possible protocols for a connection:

- `jdbc:mysql:` is for ordinary and basic JDBC failover connections.
- `jdbc:mysql:loadbalance:` is for load-balancing JDBC connections. See Section 9.3, “Configuring Load Balancing with Connector/J” for details.
- `jdbc:mysql:replication:` is for JDBC replication connections. See Section 9.4, “Configuring Source/Replica Replication with Connector/J” for details.
- `mysqlx:` is for X DevAPI connections.
- `jdbc:mysql+srv:` is for ordinary and basic failover JDBC connections that make use of DNS SRV records.
- `jdbc:mysql+srv:loadbalance:` is for load-balancing JDBC connections that make use of DNS SRV records.
- `jdbc:mysql+srv:replication:` is for replication JDBC connections that make use of DNS SRV records.
- `mysqlx+srv:` is for X DevAPI connections that make use of DNS SRV records.

hosts

Depending on the situation, the *host*s part may consist simply of a host name, or it can be a complex structure consisting of various elements like multiple host names, port numbers, host-specific properties, and user credentials.

- Single host:
  - Single-host connections without adding host-specific properties:
    - The *hosts* part is written in the format of *host*:*port*. This is an example of a simple single-host connection URL:

      ```
      jdbc:mysql://host1:33060/sakila
      ```

    - *host* can be an IPv4 or an IPv6 host name string, and in the latter case it must be put inside square brackets, for example “[1000:2000::abcd].” When *host* is not specified, the default value of `localhost` is used.

- *port* is a standard port number, i.e., an integer between 1 and 65535. The default port number for an ordinary MySQL connection is 3306, and it is 33060 for a connection using the X Protocol. If *port* is not specified, the corresponding default is used.

- Single-host connections adding host-specific properties:

  - In this case, the host is defined as a succession of *key=value* pairs. Keys are used to identify the host, the port, as well as any host-specific properties. There are two alternate formats for specifying keys:

    - The “address-equals” form:

      ```
      address=(host=host_or_ip)(port=port)(key1=value1)(key2=value2)...(keyN=valueN)
      ```

      Here is a sample URL using the“address-equals” form :

      ```
      jdbc:mysql://address=(host=myhost)(port=1111)(key1=value1)/db
      ```

    - The “key-value” form:

      ```
      (host=host,port=port,key1=value1,key2=value2,...,keyN=valueN)
      ```

      Here is a sample URL using the “key-value” form :

      ```
      jdbc:mysql://(host=myhost,port=1111,key1=value1)/db
      ```

  - The host and the port are identified by the keys *host* and *port*. The descriptions of the format and default values of *host* and *port* in Single host without host-specific properties [22] above also apply here.

  - Other keys that can be added include `user`, `password`, `protocol`, and so on. They override the global values set in the *properties* part of the URL. Limit the overrides to user, password, network timeouts, and statement and metadata cache sizes; the effects of other per-host overrides are not defined.

  - Different protocols may require different keys. For example, the `mysqlx:` scheme uses two special keys, *address* and *priority*. *address* is a *host*:*port* pair and *priority* an integer. For example:

    ```
    mysqlx://(address=host:1111,priority=1,key1=value1)/db
    ```

  - *key* is case-sensitive. Two keys differing in case only are considered conflicting, and there are no guarantees on which one will be used.

- Multiple hosts

  There are two formats for specifying multiple hosts:

  - List hosts in a comma-separated list:

    ```
    host1,host2,...,hostN
    ```

    Each host can be specified in any of the three ways described in Single host [22] above. Here are some examples:

    ```
    jdbc:mysql://myhost1:1111,myhost2:2222/db
    jdbc:mysql://address=(host=myhost1)(port=1111)(key1=value1),address=(host=myhost2)(port=2222)(key2=
    jdbc:mysql://(host=myhost1,port=1111,key1=value1),(host=myhost2,port=2222,key2=value2)/db
    jdbc:mysql://myhost1:1111,(host=myhost2,port=2222,key2=value2)/db
    ```

```
mysqlx://(address=host1:1111,priority=1,key1=value1),(address=host2:2222,priority=2,key2=value2)/db
```

- List hosts in a comma-separated list, and then encloses the list by square brackets:

  ```
  [host1,host2,...,hostN]
  ```

  This is called the host sublist form, which allows sharing of the user credentials by all hosts in the list as if they are a single host. Each host in the list can be specified in any of the three ways described in Single host [22] above. Here are some examples:

  ```
  jdbc:mysql://sandy:secret@[myhost1:1111,myhost2:2222]/db
  jdbc:mysql://sandy:secret@[address=(host=myhost1)(port=1111)(key1=value1),address=(host=myhost2)(port=2
  jdbc:mysql://sandy:secret@[myhost1:1111,address=(host=myhost2)(port=2222)(key2=value2)]/db
  ```

  While it is not possible to write host sublists recursively, a host list may contain host sublists as its member hosts.

- User credentials

  User credentials can be set outside of the connection URL—for example, as arguments when getting a connection from the `java.sql.DriverManager` (see Section 6.3, “Configuration Properties” for details). When set with the connection URL, there are several ways to specify them:

  - Prefix the a single host, a host sublist (see Multiple hosts [23]), or any host in a list of hosts with the user credentials with an `@`:

    ```
    user:password@host_or_host_sublist
    ```

    For example:

    ```
    mysqlx://sandy:secret@[(address=host1:1111,priority=1,key1=value1),(address=host2:2222,priority=2,key2=
    ```

  - Use the keys `user` and `password` to specify credentials for each host:

    ```
    (user=sandy)(password=mypass)
    ```

    For example:

    ```
    jdbc:mysql://[(host=myhost1,port=1111,user=sandy,password=secret),(host=myhost2,port=2222,user=finn,pas
    jdbc:mysql://address=(host=myhost1)(port=1111)(user=sandy)(password=secret),address=(host=myhost2)(port
    ```

  In both forms, when multiple user credentials are specified, the one to the left takes precedence—that is, going from left to right in the connection string, the first one found that is applicable to a host is the one that is used.

  Inside a host sublist, no host can have user credentials in the @ format, but individual host can have user credentials specified in the key format.

database

The default database or catalog to open. If the database is not specified, the connection is made with no default database. In this case, either call the `setCatalog()` method on the `Connection` instance, or specify table names using the database name (that is, `SELECT dbname.tablename.colname FROM dbname.tablename...`) in your SQL statements. Opening a connection without specifying the database to use is, in general, only useful when building tools that work with multiple databases, such as GUI database managers.

> **Note**
>
> Always use the `Connection.setCatalog()` method to specify the desired database in JDBC applications, rather than the `USE database` statement.

properties

A succession of global properties applying to all hosts, preceded by ? and written as key=value pairs separated by the symbol "&." Here are some examples:

```
jdbc:mysql://(host=myhost1,port=1111),(host=myhost2,port=2222)/db?key1=value1&key2=value2&key3=value3
```

The following are true for the key-value pairs:

- key and value are just strings. Proper type conversion and validation are performed internally in Connector/J.
- key is case-sensitive. Two keys differing in case only are considered conflicting, and it is uncertain which one will be used.
- Any host-specific values specified with key-value pairs as explained in Single host with host-specific properties [23] and Multiple hosts [23] above override the global values set here.

See Section 6.3, "Configuration Properties" for details about the configuration properties.

6.3 Configuration Properties

Configuration properties define how Connector/J will make a connection to a MySQL server. Unless otherwise noted, properties can be set for a DataSource object or for a Connection object.

Configuration properties can be set in one of the following ways:

- Using the set*() methods on MySQL implementations of java.sql.DataSource (which is the preferred method when using implementations of java.sql.DataSource):
  - com.mysql.cj.jdbc.MysqlDataSource
  - com.mysql.cj.jdbc.MysqlConnectionPoolDataSource
- As a key-value pair in the java.util.Properties instance passed to DriverManager.getConnection() or Driver.connect()
- As a JDBC URL parameter in the URL given to java.sql.DriverManager.getConnection(), java.sql.Driver.connect() or the MySQL implementations of the javax.sql.DataSource setURL() method. If you specify a configuration property in the URL without providing a value for it, nothing will be set; for example, adding useServerPrepStmts alone to the URL does not make Connector/J use server-side prepared statements; you need to add useServerPrepStmts=true.

> **Note**
>
> If the mechanism you use to configure a JDBC URL is XML-based, use the XML character literal & to separate configuration parameters, as the ampersand is a reserved character for XML.

The properties are listed by categories in the following tables and then in the subsections that follow. Click on a property name in the tables to see its full description in the subsections.

Table 6.1 Authentication Properties

Name	Default Value	Since Version
user	-	all versions
password	-	all versions
password1	-	8.0.28

Name	Default Value	Since Version
password2	-	8.0.28
password3	-	8.0.28
authenticationPlugins	-	5.1.19
disabledAuthenticationPlugins		5.1.19
defaultAuthenticationPlugin	mysql_native_password	5.1.19
ldapServerHostname	-	8.0.23
ociConfigFile	-	8.0.27
ociConfigProfile	DEFAULT	8.0.33
authenticationFidoCallbackHandler	-	8.0.29
authenticationWebAuthnCallbackHandler	-	8.2.0

Table 6.2 Connection Properties

Name	Default Value	Since Version
connectionAttributes	-	5.1.25
connectionLifecycleInterceptors	-	5.1.4
useConfigs	-	3.1.5
clientInfoProvider	com.mysql.cj.jdbc.CommentClientInfoProvider	5.1.0
createDatabaseIfNotExist	false	3.1.9
databaseTerm	CATALOG	8.0.17
detectCustomCollations	false	5.1.29
disconnectOnExpiredPasswords	true	5.1.23
interactiveClient	false	3.1.0
passwordCharacterEncoding	-	5.1.7
propertiesTransform	-	3.1.4
rollbackOnPooledClose	true	3.0.15
useAffectedRows	false	5.1.7

Table 6.3 Session Properties

Name	Default Value	Since Version
sessionVariables	-	3.1.8
characterEncoding	-	1.1g
characterSetResults	-	3.0.13
connectionCollation	-	3.0.13
customCharsetMapping	-	8.0.26
trackSessionState	false	8.0.26

Table 6.4 Networking Properties

Name	Default Value	Since Version
socksProxyHost	-	5.1.34
socksProxyPort	1080	5.1.34
socketFactory	com.mysql.cj.protocol.StandardSocketFactory	3.0.3

Name	Default Value	Since Version
connectTimeout	0	3.0.1
socketTimeout	0	3.0.1
dnsSrv	false	8.0.19
localSocketAddress	-	5.0.5
maxAllowedPacket	65535	5.1.8
socksProxyRemoteDns	false	8.0.29
tcpKeepAlive	true	5.0.7
tcpNoDelay	true	5.0.7
tcpRcvBuf	0	5.0.7
tcpSndBuf	0	5.0.7
tcpTrafficClass	0	5.0.7
useCompression	false	3.0.17
useUnbufferedInput	true	3.0.11

Table 6.5 Security Properties

Name	Default Value	Since Version
paranoid	false	3.0.1
serverRSAPublicKeyFile	-	5.1.31
allowPublicKeyRetrieval	false	5.1.31
sslMode	PREFERRED	8.0.13
trustCertificateKeyStoreUrl	-	5.1.0
trustCertificateKeyStoreType	JKS	5.1.0
trustCertificateKeyStorePassword		5.1.0
fallbackToSystemTrustStore	true	8.0.22
clientCertificateKeyStoreUrl	-	5.1.0
clientCertificateKeyStoreType	JKS	5.1.0
clientCertificateKeyStorePassword		5.1.0
fallbackToSystemKeyStore	true	8.0.22
tlsCiphersuites	-	5.1.35
tlsVersions	-	8.0.8
fipsCompliantJsse	false	8.1.0
KeyManagerFactoryProvider	-	8.1.0
trustManagerFactoryProvider	-	8.1.0
keyStoreProvider	-	8.1.0
sslContextProvider	-	8.1.0
allowLoadLocalInfile	false	3.0.3
allowLoadLocalInfileInPath	-	8.0.22
allowMultiQueries	false	3.1.1
allowUrlInLocalInfile	false	3.1.4
requireSSL	false	3.1.0
useSSL	true	3.0.2

Name	Default Value	Since Version
verifyServerCertificate	false	5.1.6

Table 6.6 Statements Properties

Name	Default Value	Since Version
cacheDefaultTimeZone	true	8.0.20
continueBatchOnError	true	3.0.3
dontTrackOpenResources	false	3.1.7
queryInterceptors	-	8.0.7
queryTimeoutKillsConnection	false	5.1.9

Table 6.7 Prepared Statements Properties

Name	Default Value	Since Version
allowNanAndInf	false	3.1.5
autoClosePStmtStreams	false	3.1.12
compensateOnDuplicateKeyUpdateCounts	false	5.1.7
emulateUnsupportedPstmts	true	3.1.7
generateSimpleParameterMetadata	false	5.0.5
processEscapeCodesForPrepStmts	true	3.1.12
useServerPrepStmts	false	3.1.0
useStreamLengthsInPrepStmts	true	3.0.2

Table 6.8 Result Sets Properties

Name	Default Value	Since Version
clobberStreamingResults	false	3.0.9
emptyStringsConvertToZero	true	3.1.8
holdResultsOpenOverStatementClose	false	3.1.7
jdbcCompliantTruncation	true	3.1.2
maxRows	-1	all versions
netTimeoutForStreamingResults	600	5.1.0
padCharsWithSpace	false	5.0.6
populateInsertRowWithDefaultValues	false	5.0.5
scrollTolerantForwardOnly	false	8.0.24
strictUpdates	true	3.0.4
tinyInt1isBit	true	3.0.16
transformedBitIsBoolean	false	3.1.9

Table 6.9 Metadata Properties

Name	Default Value	Since Version
getProceduresReturnsFunctions	true	5.1.26
noAccessToProcedureBodies	false	5.0.3
nullDatabaseMeansCurrent	false	3.1.8

Name	Default Value	Since Version
useHostsInPrivileges	true	3.0.2
useInformationSchema	false	5.0.0

Table 6.10 BLOB/CLOB processing Properties

Name	Default Value	Since Version
blobSendChunkSize	1048576	3.1.9
blobsAreStrings	false	5.0.8
clobCharacterEncoding	-	5.0.0
emulateLocators	false	3.1.0
functionsNeverReturnBlobs	false	5.0.8
locatorFetchBufferSize	1048576	3.2.1

Table 6.11 Datetime types processing Properties

Name	Default Value	Since Version
connectionTimeZone	-	3.0.2
forceConnectionTimeZoneToSession	false	8.0.23
noDatetimeStringSync	false	3.1.7
preserveInstants	true	8.0.23
sendFractionalSeconds	true	5.1.37
sendFractionalSecondsForTime	true	8.0.23
treatMysqlDatetimeAsTimestamp	false	8.2.0
treatUtilDateAsTimestamp	true	5.0.5
yearIsDateType	true	3.1.9
zeroDateTimeBehavior	EXCEPTION	3.1.4

Table 6.12 High Availability and Clustering Properties

Name	Default Value	Since Version
autoReconnect	false	1.1
autoReconnectForPools	false	3.1.3
failOverReadOnly	true	3.0.12
maxReconnects	3	1.1
reconnectAtTxEnd	false	3.0.10
retriesAllDown	120	5.1.6
initialTimeout	2	1.1
queriesBeforeRetrySource	50	3.0.2
secondsBeforeRetrySource	30	3.0.2
allowReplicaDownConnections	false	6.0.2
allowSourceDownConnections	false	5.1.27
ha.enableJMX	false	5.1.27
loadBalanceHostRemovalGracePeriod	15000	6.0.3
readFromSourceWhenNoReplicas	false	6.0.2

Name	Default Value	Since Version
selfDestructOnPingMaxOperations	0	5.1.6
selfDestructOnPingSecondsLifetime	0	5.1.6
ha.loadBalanceStrategy	random	5.0.6
loadBalanceAutoCommitStatementRegex	-	5.1.15
loadBalanceAutoCommitStatementThreshold	0	5.1.15
loadBalanceBlocklistTimeout	0	5.1.0
loadBalanceConnectionGroup	-	5.1.13
loadBalanceExceptionChecker	com.mysql.cj.jdbc.ha.StandardLoadBalanceExceptionChecker	5.1.13
loadBalancePingTimeout	0	5.1.13
loadBalanceSQLExceptionSubclassFailover		5.1.13
loadBalanceSQLStateFailover	-	5.1.13
loadBalanceValidateConnectionOnSwapServer	false	5.1.13
pinGlobalTxToPhysicalConnection	false	5.0.1
replicationConnectionGroup	-	8.0.7
resourceId	-	5.0.1
serverAffinityOrder	-	8.0.8

Table 6.13 Performance Extensions Properties

Name	Default Value	Since Version
callableStmtCacheSize	100	3.1.2
metadataCacheSize	50	3.1.1
useLocalSessionState	false	3.1.7
useLocalTransactionState	false	5.1.7
prepStmtCacheSize	25	3.0.10
prepStmtCacheSqlLimit	256	3.0.10
queryInfoCacheFactory	com.mysql.cj.PerConnectionLRUFactory	5.1.1
serverConfigCacheFactory	com.mysql.cj.util.PerVmServerConfigCacheFactory	5.1.1
alwaysSendSetIsolation	true	3.1.7
maintainTimeStats	true	3.1.9
useCursorFetch	false	5.0.0
cacheCallableStmts	false	3.1.2
cachePrepStmts	false	3.0.10
cacheResultSetMetadata	false	3.1.1
cacheServerConfiguration	false	3.1.5
defaultFetchSize	0	3.1.9
dontCheckOnDuplicateKeyUpdateInSQL	false	5.1.32
elideSetAutoCommits	false	3.1.3
enableEscapeProcessing	true	6.0.1
enableQueryTimeouts	true	5.0.6
largeRowSizeThreshold	2048	5.1.1
readOnlyPropagatesToServer	true	5.1.35
rewriteBatchedStatements	false	3.1.13

Name	Default Value	Since Version
useReadAheadInput	true	3.1.5

Table 6.14 Debugging/Profiling Properties

Name	Default Value	Since Version
logger	com.mysql.cj.log.StandardLogger	3.1.1
profilerEventHandler	com.mysql.cj.log.LoggingProfilerEventHandler	5.1.6
useNanosForElapsedTime	false	5.0.7
maxQuerySizeToLog	2048	3.1.3
maxByteArrayAsHex	1024	8.0.31
profileSQL	false	3.1.0
logSlowQueries	false	3.1.2
slowQueryThresholdMillis	2000	3.1.2
slowQueryThresholdNanos	0	5.0.7
autoSlowLog	true	5.1.4
explainSlowQueries	false	3.1.2
gatherPerfMetrics	false	3.1.2
reportMetricsIntervalMillis	30000	3.1.2
logXaCommands	false	5.0.5
traceProtocol	false	3.1.2
enablePacketDebug	false	3.1.3
packetDebugBufferSize	20	3.1.3
useUsageAdvisor	false	3.1.1
resultSetSizeThreshold	100	5.0.5
autoGenerateTestcaseScript	false	3.1.9

Table 6.15 Exceptions/Warnings Properties

Name	Default Value	Since Version
dumpQueriesOnException	false	3.1.3
exceptionInterceptors	-	5.1.8
ignoreNonTxTables	false	3.0.9
includeInnodbStatusInDeadlockExceptions	false	5.0.7
includeThreadDumpInDeadlockExceptions	false	5.1.15
includeThreadNamesAsStatementComment	false	5.1.15
useOnlyServerErrorMessages	true	3.0.15

Table 6.16 Tunes for integration with other products Properties

Name	Default Value	Since Version
overrideSupportsIntegrityEnhancementFacility	false	3.1.12
ultraDevHack	false	2.0.3

Table 6.17 JDBC compliance Properties

Name	Default Value	Since Version
useColumnNamesInFindColumn	false	5.1.7

Name	Default Value	Since Version
pedantic	false	3.0.0
useOldAliasMetadataBehavior	false	5.0.4

Table 6.18 X Protocol and X DevAPI Properties

Name	Default Value	Since Version
xdevapi.auth	PLAIN	8.0.8
xdevapi.compression	PREFERRED	8.0.20
xdevapi.compression-algorithms	zstd_stream,lz4_message,deflate_stream	8.0.22
xdevapi.compression-extensions	-	8.0.22
xdevapi.connect-timeout	10000	8.0.13
xdevapi.connection-attributes	-	8.0.16
xdevapi.dns-srv	false	8.0.19
xdevapi.fallback-to-system-keystore	true	8.0.22
xdevapi.fallback-to-system-truststore	true	8.0.22
xdevapi.ssl-keystore	-	8.0.22
xdevapi.ssl-keystore-password	-	8.0.22
xdevapi.ssl-keystore-type	JKS	8.0.22
xdevapi.ssl-mode	REQUIRED	8.0.7
xdevapi.ssl-truststore	-	6.0.6
xdevapi.ssl-truststore-password	-	6.0.6
xdevapi.ssl-truststore-type	JKS	6.0.6
xdevapi.tls-ciphersuites	-	8.0.19
xdevapi.tls-versions	-	8.0.19

6.3.1 Authentication

- user

  The user to connect as. If none is specified, it is authentication plugin dependent what user name is used. Built-in authentication plugins default to the session login user name.

Since Version	all versions

- password

  The password to use when authenticating the user.

Since Version	all versions

- password1

The password to use in the first phase of a Multi-Factor Authentication workflow. It is a synonym of the connection property 'password' and can also be set with user credentials in the connection string.

Since Version	8.0.28

- `password2`

  The password to use in the second phase of a Multi-Factor Authentication workflow.

Since Version	8.0.28

- `password3`

  The password to use in the third phase of a Multi-Factor Authentication workflow.

Since Version	8.0.28

- `authenticationPlugins`

  Comma-delimited list of classes that implement the interface 'com.mysql.cj.protocol.AuthenticationPlugin'. These plugins will be loaded at connection initialization and can be used together with their sever-side counterparts for authenticating users, unless they are disabled in the connection property 'disabledAuthenticationPlugins'.

Since Version	5.1.19

- `disabledAuthenticationPlugins`

  Comma-delimited list of authentication plugins client-side protocol names or classes implementing the interface 'com.mysql.cj.protocol.AuthenticationPlugin'. The authentication plugins listed will not be used for authenticating users and, if anyone of them is required during the authentication exchange, the connection fails. The default authentication plugin specified in the property 'defaultAuthenticationPlugin' cannot be disabled.

Since Version	5.1.19

- `defaultAuthenticationPlugin`

  The default authentication plugin client-side protocol name or a fully qualified name of a class that implements the interface 'com.mysql.cj.protocol.AuthenticationPlugin'. The specified authentication plugin must be either one of the built-in authentication plugins or one of the plugins listed in the property 'authenticationPlugins'. Additionally, the default authentication plugin cannot be disabled with the property 'disabledAuthenticationPlugins'. Neither an empty nor unknown plugin name or class can be set for this property.

  By default, Connector/J honors the server-side default authentication plugin, which is known after receiving the initial handshake packet, and falls back to this property's default value if that plugin cannot be used. However, when a value is explicitly provided to this property, Connector/J then overrides the server-side default authentication plugin and always tries first the plugin specified with this property.

Default Value	mysql_native_password
Since Version	5.1.19

- `ldapServerHostname`

  When using MySQL's LDAP pluggable authentication with GSSAPI/Kerberos authentication method, allows setting the LDAP service principal hostname as configured in the Kerberos KDC. If this

property is not set, Connector/J takes the system property 'java.security.krb5.kdc' and extracts the hostname (short name) from its value and uses it. If neither is set, the connection fails with an exception.

Since Version	8.0.23

- `ociConfigFile`

  The location of the OCI configuration file as required by the OCI SDK for Java. Default value is "~/.oci/config" for Unix-like systems and "%HOMEDRIVE%%HOMEPATH%.oci\config" for Windows.

Since Version	8.0.27

- `ociConfigProfile`

  The profile in the OCI configuration file specified in 'ociConfigFile', from where the configuration to use in the 'authentication_oci_client' authentication plugin is to be read.

Default Value	DEFAULT
Since Version	8.0.33

- `authenticationFidoCallbackHandler`

  Fully-qualified class name of a class implementing the interface 'com.mysql.cj.callback.MysqlCallbackHandler'. This class will be used by the FIDO authentication plugin to obtain the authenticator data and signature required for the FIDO authentication process. See the documentation of 'com.mysql.cj.callback.FidoAuthenticationCallback' for more details.

Since Version	8.0.29

- `authenticationWebAuthnCallbackHandler`

  Fully-qualified class name of a class implementing the interface 'com.mysql.cj.callback.MysqlCallbackHandler'. This class will be used by the WebAuthn authentication plugin to obtain the authenticator data and signature required for the FIDO authentication process. See the documentation of com.mysql.cj.callback.WebAuthnAuthenticationCallback for more details.

Since Version	8.2.0

6.3.2 Connection

- `connectionAttributes`

  A comma-delimited list of user-defined "key:value" pairs, in addition to standard MySQL-defined "key:value" pairs, to be passed to MySQL Server for display as connection attributes in the 'PERFORMANCE_SCHEMA' tables 'session_account_connect_attrs' and 'session_connect_attrs'. Example usage: "connectionAttributes=key1:value1,key2:value2" This functionality is available for use with MySQL Server version 5.6 or later only. Earlier versions of MySQL Server do not support connection attributes, causing this configuration option to be ignored. Setting "connectionAttributes=none" will cause connection attribute processing to be bypassed for situations where Connection creation/initialization speed is critical.

Since Version	5.1.25

- `connectionLifecycleInterceptors`

  A comma-delimited list of classes that implement 'com.mysql.cj.jdbc.interceptors.ConnectionLifecycleInterceptor' that should be notified of

connection lifecycle events (creation, destruction, commit, rollback, setting the current database and changing the autocommit mode) and potentially alter the execution of these commands. 'ConnectionLifecycleInterceptors' are stackable, more than one interceptor may be specified via the configuration property as a comma-delimited list, with the interceptors executed in order from left to right.

Since Version	5.1.4

- `useConfigs`

  Load the comma-delimited list of configuration properties for specifying combinations of options for particular scenarios. These properties are loaded before parsing the URL or applying user-specified properties. Allowed values are "3-0-Compat", "clusterBase", "coldFusion", "fullDebug", "maxPerformance", "maxPerformance-8-0" and "solarisMaxPerformance", and they correspond to properties files shipped within the Connector/J jar file, under "com/mysql/cj/configurations".

Since Version	3.1.5

- `clientInfoProvider`

  The name of a class that implements the 'com.mysql.cj.jdbc.ClientInfoProvider' interface in order to support JDBC-4.0's 'Connection.get/setClientInfo()' methods.

Default Value	com.mysql.cj.jdbc.CommentClientInfoProvider
Since Version	5.1.0

- `createDatabaseIfNotExist`

  Creates the database given in the URL if it doesn't yet exist. Assumes the configured user has permissions to create databases.

Default Value	false
Since Version	3.1.9

- `databaseTerm`

  MySQL uses the term "schema" as a synonym of the term "database," while Connector/J historically takes the JDBC term "catalog" as synonymous to "database". This property sets for Connector/J which of the JDBC terms "catalog" and "schema" is used in an application to refer to a database. The property takes one of the two values "CATALOG" or "SCHEMA" and uses it to determine (1) which Connection methods can be used to set/get the current database (e.g. 'setCatalog()' or 'setSchema()'?), (2) which arguments can be used within the various 'DatabaseMetaData' methods to filter results (e.g. the catalog or 'schemaPattern' argument of 'getColumns()'?), and (3) which fields in the result sets returned by 'DatabaseMetaData' methods contain the database identification information (i.e., the 'TABLE_CAT' or 'TABLE_SCHEM' field in the result set returned by 'getTables()'?).

  If "databaseTerm=CATALOG", 'schemaPattern' for searches are ignored and calls of schema methods (like 'setSchema()' or get 'Schema()') become no-ops, and vice versa.

Default Value	CATALOG
Since Version	8.0.17

- `detectCustomCollations`

  Should the driver detect custom charsets/collations installed on server? If this option set to "true" the driver gets actual charsets/collations from the server each time a connection establishes. This could slow down connection initialization significantly.

Default Value	false
Since Version	5.1.29

- `disconnectOnExpiredPasswords`

  If 'disconnectOnExpiredPasswords' is set to "false" and password is expired then server enters sandbox mode and sends 'ERR(08001, ER_MUST_CHANGE_PASSWORD)' for all commands that are not needed to set a new password until a new password is set.

Default Value	true
Since Version	5.1.23

- `interactiveClient`

  Set the 'CLIENT_INTERACTIVE' flag, which tells MySQL to timeout connections based on 'interactive_timeout' instead of 'wait_timeout'.

Default Value	false
Since Version	3.1.0

- `passwordCharacterEncoding`

  Instructs the server to use the default character set for the specified Java encoding during the authentication phase. If this property is not set, Connector/J falls back to the collation name specified in the property 'connectionCollation' or to the Java encoding specified in the property 'characterEncoding', in that order of priority. The default collation of the character set utf8mb4 is used if none of the properties is set.

Since Version	5.1.7

- `propertiesTransform`

  An implementation of 'com.mysql.cj.conf.ConnectionPropertiesTransform' that the driver will use to modify connection string properties passed to the driver before attempting a connection.

Since Version	3.1.4

- `rollbackOnPooledClose`

  Should the driver issue a 'rollback()' when the logical connection in a pool is closed?

Default Value	true
Since Version	3.0.15

- `useAffectedRows`

  Don't set the 'CLIENT_FOUND_ROWS' flag when connecting to the server. Note that this is not JDBC-compliant and it will break most applications that rely on "found" rows vs. "affected rows" for DML statements, but does cause correct update counts from "INSERT ... ON DUPLICATE KEY UPDATE" statements to be returned by the server.

Default Value	false
Since Version	5.1.7

6.3.3 Session

- `sessionVariables`

A comma or semicolon separated list of "name=value" pairs to be sent as "SET [SESSION] ..." to the server when the driver connects.

Since Version	3.1.8

- `characterEncoding`

  Instructs the server to set session system variables 'character_set_client' and 'character_set_connection' to the default character set supported by MySQL for the specified Java character encoding and set 'collation_connection' to the default collation for this character set. If neither this property nor the property 'connectionCollation' is set:

  For Connector/J 8.0.25 and earlier, the driver will try to use the server's default character set;

  For Connector/J 8.0.26 and later, the driver will use "utf8mb4".

Since Version	1.1g

- `characterSetResults`

  Instructs the server to return the data encoded with the default character set for the specified Java encoding. If not set or set to "null", the server will send data in its original character set and the driver will decode it according to the result metadata.

Since Version	3.0.13

- `connectionCollation`

  Instructs the server to set session system variable 'collation_connection' to the specified collation name and set 'character_set_client' and 'character_set_connection' to a corresponding character set. This property overrides the value of 'characterEncoding' with the default character set this collation belongs to, if and only if 'characterEncoding' is not configured or is configured with a character set that is incompatible with the collation. That means 'connectionCollation' may not always correct a mismatch of character sets. For example, if 'connectionCollation' is set to "latin1_swedish_ci", the corresponding character set is "latin1" for MySQL, which maps it to the Java character set "windows-1252"; so if 'characterEncoding' is not set,"windows-1252" is the character set that will be used; but if 'characterEncoding' has been set to, e.g. "ISO-8859-1", that is compatible with "latin1_swedish_ci", so the character encoding setting is left unchanged; and if client is actually using "windows-1252" (which is similar but different from "ISO-8859-1"), errors would occur for some characters. If neither this property nor the property 'characterEncoding' is set:

  For Connector/J 8.0.25 and earlier, the driver will try to use the server's default character set;

  For Connector/J 8.0.26 and later, the driver will use utf8mb4's default collation.

Since Version	3.0.13

- `customCharsetMapping`

  A comma-delimited list of custom "charset:java encoding" pairs.

  In case the MySQL server is configured with custom character sets and "detectCustomCollations=true", Connector/J needs to know which Java character encoding to use for the data represented by these character sets. Example usage: "customCharsetMapping=charset1:UTF-8,charset2:Cp1252".

Since Version	8.0.26

- `trackSessionState`

Receive server session state changes on query results. These changes are accessible via 'MysqlConnection.getServerSessionStateController()'.

Default Value	false
Since Version	8.0.26

6.3.4 Networking

- `socksProxyHost`

  Name or IP address of a SOCKS host to connect through.

Since Version	5.1.34

- `socksProxyPort`

  Port of the SOCKS server.

Default Value	1080
Since Version	5.1.34

- `socketFactory`

  The name of the class that the driver should use for creating socket connections to the server. This class must implement the interface 'com.mysql.cj.protocol.SocketFactory' and have a public no-args constructor.

Default Value	com.mysql.cj.protocol.StandardSocketFactory
Since Version	3.0.3

- `connectTimeout`

  Timeout for socket connect (in milliseconds), with 0 being no timeout.

Default Value	0
Since Version	3.0.1

- `socketTimeout`

  Timeout, specified in milliseconds, on network socket operations. Value "0" means no timeout.

Default Value	0
Since Version	3.0.1

- `dnsSrv`

  Should the driver use the given host name to lookup for DNS SRV records and use the resulting list of hosts in a multi-host failover connection? Note that a single host name and no port must be provided when this option is enabled.

Default Value	false
Since Version	8.0.19

- `localSocketAddress`

  Hostname or IP address given to explicitly configure the interface that the driver will bind the client side of the TCP/IP connection to when connecting.

Since Version	5.0.5

- `maxAllowedPacket`

  Maximum allowed packet size to send to server. If not set, the value of system variable 'max_allowed_packet' will be used to initialize this upon connecting. This value will not take effect if set larger than the value of 'max_allowed_packet'. Also, due to an internal dependency with the property 'blobSendChunkSize', this setting has a minimum value of "8203" if 'useServerPrepStmts' is set to "true".

Default Value	65535
Since Version	5.1.8

- `socksProxyRemoteDns`

  When using a SOCKS proxy, whether the DNS lookup for the database host should be performed locally or through the SOCKS proxy.

Default Value	false
Since Version	8.0.29

- `tcpKeepAlive`

  If connecting using TCP/IP, should the driver set 'SO_KEEPALIVE'?

Default Value	true
Since Version	5.0.7

- `tcpNoDelay`

  If connecting using TCP/IP, should the driver set 'SO_TCP_NODELAY', disabling the Nagle Algorithm?

Default Value	true
Since Version	5.0.7

- `tcpRcvBuf`

  If connecting using TCP/IP, should the driver set 'SO_RCV_BUF' to the given value? The default value of "0", means use the platform default value for this property.

Default Value	0
Since Version	5.0.7

- `tcpSndBuf`

  If connecting using TCP/IP, should the driver set 'SO_SND_BUF' to the given value? The default value of "0", means use the platform default value for this property.

Default Value	0
Since Version	5.0.7

- `tcpTrafficClass`

  If connecting using TCP/IP, should the driver set traffic class or type-of-service fields? See the documentation for 'java.net.Socket.setTrafficClass()' for more information.

Default Value	0
Since Version	5.0.7

- `useCompression`

  Use zlib compression when communicating with the server?

Default Value	false
Since Version	3.0.17

- `useUnbufferedInput`

  Don't use 'BufferedInputStream' for reading data from the server.

Default Value	true
Since Version	3.0.11

6.3.5 Security

- `paranoid`

  Take measures to prevent exposure sensitive information in error messages and clear data structures holding sensitive data when possible?

Default Value	false
Since Version	3.0.1

- `serverRSAPublicKeyFile`

  File path to the server RSA public key file for 'sha256_password' authentication. If not specified, the public key will be retrieved from the server.

Since Version	5.1.31

- `allowPublicKeyRetrieval`

  Allows special handshake round-trip to get an RSA public key directly from server.

Default Value	false
Since Version	5.1.31

- `sslMode`

  By default, network connections are SSL encrypted; this property permits secure connections to be turned off, or a different levels of security to be chosen. The following values are allowed: "DISABLED" - Establish unencrypted connections; "PREFERRED" - Establish encrypted connections if the server enabled them, otherwise fall back to unencrypted connections; "REQUIRED" - Establish secure connections if the server enabled them, fail otherwise; "VERIFY_CA" - Like "REQUIRED" but additionally verify the server TLS certificate against the configured Certificate Authority (CA) certificates; "VERIFY_IDENTITY" - Like "VERIFY_CA", but additionally verify that the server certificate matches the host to which the connection is attempted.

  This property replaced the deprecated legacy properties 'useSSL', 'requireSSL', and 'verifyServerCertificate', which are still accepted but translated into a value for 'sslMode' if 'sslMode' is not explicitly set: "useSSL=false" is translated to "sslMode=DISABLED"; {"useSSL=true", "requireSSL=false", "verifyServerCertificate=false"} is translated to "sslMode=PREFERRED"; {"useSSL=true", "requireSSL=true", "verifyServerCertificate=false"}

is translated to "sslMode=REQUIRED"; {"useSSL=true", "verifyServerCertificate=true"} is translated to "sslMode=VERIFY_CA". There is no equivalent legacy settings for "sslMode=VERIFY_IDENTITY". Note that, for all server versions, the default setting of 'sslMode' is "PREFERRED", and it is equivalent to the legacy settings of "useSSL=true", "requireSSL=false", and "verifyServerCertificate=false", which are different from their default settings for Connector/J 8.0.12 and earlier in some situations. Applications that continue to use the legacy properties and rely on their old default settings should be reviewed.

The legacy properties are ignored if 'sslMode' is set explicitly. If none of 'sslMode' or 'useSSL' is set explicitly, the default setting of "sslMode=PREFERRED" applies.

Default Value	PREFERRED
Since Version	8.0.13

- `trustCertificateKeyStoreUrl`

  URL for the trusted root certificates key store.

  If not specified, the property 'fallbackToSystemTrustStore' determines if system-wide trust store is used.

Since Version	5.1.0

- `trustCertificateKeyStoreType`

  Key store type for trusted root certificates.

  Null or empty means use the default, which is "JKS". Standard key store types supported by the JVM are "JKS" and "PKCS12", your environment may have more available depending on what security providers are installed and available to the JVM.

Default Value	JKS
Since Version	5.1.0

- `trustCertificateKeyStorePassword`

  Password for the trusted root certificates key store.

Since Version	5.1.0

- `fallbackToSystemTrustStore`

  Whether the absence of setting a value for 'trustCertificateKeyStoreUrl' falls back to using the system-wide default trust store or one defined through the system properties 'javax.net.ssl.trustStore*'.

Default Value	true
Since Version	8.0.22

- `clientCertificateKeyStoreUrl`

  URL for the client certificate KeyStore.

  If not specified, the property 'fallbackToSystemKeyStore' determines if system-wide key store is used.

Since Version	5.1.0

- `clientCertificateKeyStoreType`

Key store type for client certificates.

Null or empty means use the default, which is "JKS". Standard key store types supported by the JVM are "JKS" and "PKCS12", your environment may have more available depending on what security providers are installed and available to the JVM.

Default Value	JKS
Since Version	5.1.0

- `clientCertificateKeyStorePassword`

Password for the client certificates key store.

Since Version	5.1.0

- `fallbackToSystemKeyStore`

Whether the absence of setting a value for 'clientCertificateKeyStoreUrl' falls back to using the system-wide key store defined through the system properties 'javax.net.ssl.keyStore*'.

Default Value	true
Since Version	8.0.22

- `tlsCiphersuites`

When establishing secure connections, overrides the cipher suites enabled for use on the underlying SSL sockets. This may be required when using external JSSE providers or to specify cipher suites compatible with both MySQL server and used JVM. Prior to version 8.0.28, this property was named 'enabledSSLCipherSuites', which remains as an alias.

Since Version	5.1.35

- `tlsVersions`

List of TLS protocols to allow when establishing secure connections. Overrides the TLS protocols enabled in the underlying SSL sockets. This can be used to restrict connections to specific TLS versions and, by doing that, avoid TLS negotiation fallback. Allowed and default values are "TLSv1.2" and "TLSv1.3". Prior to version 8.0.28, this property was named 'enabledTLSProtocols', which remains as an alias.

Since Version	8.0.8

- `fipsCompliantJsse`

Enables Connector/J to be compatible to JSSE operating in FIPS mode. Should be set to "true" if the JSSE is configured to operate in FIPS mode and Connector/J receives the error "FIPS mode: only SunJSSE TrustManagers may be used" when creating secure connections. If set to "true" then, when establishing secure connections, the driver operates as if the 'sslMode' was set to "VERIFY_CA" or "VERIFY_IDENTITY", i.e., all secure connections require at least server certificate validation, for which a trust store must be configured or fall back to the system-wide trust store must be enabled.

Default Value	false
Since Version	8.1.0

- `KeyManagerFactoryProvider`

  The name of the a Java Security Provider that provides a 'javax.net.ssl.KeyManagerFactory' implementation. If none is specified then the default one is used.

Since Version	8.1.0

- `trustManagerFactoryProvider`

  The name of the a Java Security Provider that provides a 'javax.net.ssl.TrustManagerFactory' implementation. If none is specified then the default one is used.

Since Version	8.1.0

- `keyStoreProvider`

  The name of the a Java Security Provider that provides a 'java.security.KeyStore' implementation that supports the key stores types specified with 'clientCertificateKeyStoreType' and 'trustCertificateKeyStoreType'. If none is specified then the default one is used.

Since Version	8.1.0

- `sslContextProvider`

  The name of the a Java Security Provider that provides a 'javax.net.ssl.SSLContext' implementation. If none is specified then the default one is used.

Since Version	8.1.0

- `allowLoadLocalInfile`

  Should the driver allow use of "LOAD DATA LOCAL INFILE ..."?

  Setting to "true" overrides whatever path is set in 'allowLoadLocalInfileInPath', allowing uploading files from any location.

Default Value	false
Since Version	3.0.3

- `allowLoadLocalInfileInPath`

  Enables "LOAD DATA LOCAL INFILE ..." statements, but only allows loading files from the specified path. Files within sub-directories are also allowed, but relative paths or symlinks that fall outside this path are forbidden.

Since Version	8.0.22

- `allowMultiQueries`

  Allow the use of ";" to delimit multiple queries during one statement. This option does not affect the 'addBatch()' and 'executeBatch()' methods, which rely on 'rewriteBatchStatements' instead.

Default Value	false
Since Version	3.1.1

- `allowUrlInLocalInfile`

  Should the driver allow URLs in "LOAD DATA LOCAL INFILE ..." statements?

Default Value	false
Since Version	3.1.4

- `requireSSL`

  DEPRECATED: See 'sslMode' property description for details.

  For 8.0.12 and earlier: Require server support of SSL connection if "useSSL=true".

Default Value	false
Since Version	3.1.0

- `useSSL`

  DEPRECATED: See 'sslMode' property description for details.

  For 8.0.12 and earlier: Use SSL when communicating with the server, default is "true" when connecting to MySQL 5.5.45+, 5.6.26+ or 5.7.6+, otherwise default is "false".

  For 8.0.13 and later: Default is "true".

Default Value	true
Since Version	3.0.2

- `verifyServerCertificate`

  DEPRECATED: See 'sslMode' property description for details.

  For 8.0.12 and earlier: If 'useSSL' is set to "true", should the driver verify the server's certificate? When using this feature, the key store parameters should be specified by the 'clientCertificateKeyStore*' properties, rather than system properties. Default is "false" when connecting to MySQL 5.5.45+, 5.6.26+ or 5.7.6+ and 'useSSL' was not explicitly set to "true". Otherwise default is "true".

  For 8.0.13 and later: Default is "false".

Default Value	false
Since Version	5.1.6

6.3.6 Statements

- `cacheDefaultTimeZone`

  Caches client's default time zone. This results in better performance when dealing with time zone conversions in Date and Time data types, however it won't be aware of time zone changes if they happen at runtime.

Default Value	true
Since Version	8.0.20

- `continueBatchOnError`

  Should the driver continue processing batch commands if one statement fails. The JDBC spec allows either way.

Default Value	true
Since Version	3.0.3

- `dontTrackOpenResources`

  The JDBC specification requires the driver to automatically track and close resources, however if your application doesn't do a good job of explicitly calling 'close()' on statements or result sets this can cause memory leakage. Setting this property to "true" relaxes this constraint, and can be more memory efficient for some applications. Also the automatic closing of the statement and current result set in 'Statement.closeOnCompletion()' and 'Statement.getMoreResults([Statement.CLOSE_CURRENT_RESULT | Statement.CLOSE_ALL_RESULTS])', respectively, ceases to happen. This property automatically sets "holdResultsOpenOverStatementClose=true".

Default Value	false
Since Version	3.1.7

- `queryInterceptors`

  A comma-delimited list of classes that implement 'com.mysql.cj.interceptors.QueryInterceptor' that intercept query executions and are able influence the results. Query iterceptors are chainable: the results returned by the current interceptor will be passed on to the next in the chain, from left-to-right in the order specified in this property.

Since Version	8.0.7

- `queryTimeoutKillsConnection`

  If the timeout given in 'Statement.setQueryTimeout()' expires, should the driver forcibly abort the connection instead of attempting to abort the query?

Default Value	false
Since Version	5.1.9

6.3.7 Prepared Statements

- `allowNanAndInf`

  Should the driver allow NaN or +/- INF values in 'PreparedStatement.setDouble()'?

Default Value	false
Since Version	3.1.5

- `autoClosePStmtStreams`

  Should the driver automatically call the method 'close()' on streams/readers passed as arguments via 'set*()' methods?

Default Value	false
Since Version	3.1.12

- `compensateOnDuplicateKeyUpdateCounts`

  Should the driver compensate for the update counts of "INSERT ... ON DUPLICATE KEY UPDATE" statements (2 = 1, 0 = 1) when using prepared statements?

Default Value	false
Since Version	5.1.7

- `emulateUnsupportedPstmts`

  Should the driver detect prepared statements that are not supported by the server, and replace them with client-side emulated versions?

Default Value	true
Since Version	3.1.7

- `generateSimpleParameterMetadata`

  Should the driver generate simplified parameter metadata for prepared statements when no metadata is available either because the server couldn't support preparing the statement, or server-side prepared statements are disabled?

Default Value	false
Since Version	5.0.5

- `processEscapeCodesForPrepStmts`

  Should the driver process escape codes in queries that are prepared? Default escape processing behavior in non-prepared statements must be defined with the property 'enableEscapeProcessing'.

Default Value	true
Since Version	3.1.12

- `useServerPrepStmts`

  Use server-side prepared statements if the server supports them? The server may limit the number of prepared statements with 'max_prepared_stmt_count' or disable them altogether. In case of not being possible to prepare new server-side prepared statements, it depends on the value of 'emulateUnsupportedPstmts' to whether return an error or fall back to client-side emulated prepared statements.

Default Value	false
Since Version	3.1.0

- `useStreamLengthsInPrepStmts`

  Honor stream length parameter in 'PreparedStatement/ResultSet.set*Stream()' method calls?

Default Value	true
Since Version	3.0.2

6.3.8 Result Sets

- `clobberStreamingResults`

  This will cause a streaming result set to be automatically closed, and any outstanding data still streaming from the server to be discarded if another query is executed before all the data has been read from the server.

Default Value	false
Since Version	3.0.9

- `emptyStringsConvertToZero`

  Should the driver allow conversions from empty string fields to numeric values of "0"?

Default Value	true
Since Version	3.1.8

- `holdResultsOpenOverStatementClose`

  Should the driver close result sets on 'Statement.close()' as required by the JDBC specification?

Default Value	false
Since Version	3.1.7

- `jdbcCompliantTruncation`

  Should the driver throw 'java.sql.DataTruncation' exceptions when data is truncated as is required by the JDBC specification? This property has no effect if the server sql-mode includes 'STRICT_TRANS_TABLES'.

Default Value	true
Since Version	3.1.2

- `maxRows`

  The maximum number of rows to return. The default "0" means return all rows.

Default Value	-1
Since Version	all versions

- `netTimeoutForStreamingResults`

  What value should the driver automatically set the server setting 'net_write_timeout' to when the streaming result sets feature is in use? Value has unit of seconds, the value "0" means the driver will not try and adjust this value.

Default Value	600
Since Version	5.1.0

- `padCharsWithSpace`

  If a result set column has the CHAR type and the value does not fill the amount of characters specified in the DDL for the column, should the driver pad the remaining characters with space (for ANSI compliance)?

Default Value	false
Since Version	5.0.6

- `populateInsertRowWithDefaultValues`

  When using result sets that are 'CONCUR_UPDATABLE', should the driver pre-populate the insert row with default values from the DDL for the table used in the query so those values are immediately available for 'ResultSet' accessors? This functionality requires a call to the database for metadata each time a result set of this type is created. If disabled, the default values will be populated by the an internal call to 'refreshRow()' which pulls back default values and/or values changed by triggers.

Default Value	false

Since Version	5.0.5

- `scrollTolerantForwardOnly`

  Should the driver contradict the JDBC API and tolerate and support backward and absolute cursor movement on result sets of type 'ResultSet.TYPE_FORWARD_ONLY'?

  Regardless of this setting, cursor-based and row streaming result sets cannot be navigated in the prohibited directions.

Default Value	false
Since Version	8.0.24

- `strictUpdates`

  Should the driver do strict checking, i.e. all primary keys selected, of updatable result sets?

Default Value	true
Since Version	3.0.4

- `tinyInt1isBit`

  Since the MySQL server silently converts BIT to TINYINT(1) when creating tables, should the driver treat the datatype TINYINT(1) as the BIT type?

Default Value	true
Since Version	3.0.16

- `transformedBitIsBoolean`

  If the driver converts TINYINT(1) to a different type, should it use BOOLEAN instead of BIT?

Default Value	false
Since Version	3.1.9

6.3.9 Metadata

- `getProceduresReturnsFunctions`

  Pre-JDBC4 'DatabaseMetaData' API has only the 'getProcedures()' and 'getProcedureColumns()' methods, so they return metadata info for both stored procedures and functions. JDBC4 was extended with the 'getFunctions()' and 'getFunctionColumns()' methods and the expected behaviours of previous methods are not well defined. For JDBC4 and higher, default "true" value of the option means that calls of 'DatabaseMetaData.getProcedures()' and 'DatabaseMetaData.getProcedureColumns()' return metadata for both procedures and functions as before, keeping backward compatibility. Setting this property to "false" decouples Connector/J from its pre-JDBC4 behaviours for 'DatabaseMetaData.getProcedures()' and 'DatabaseMetaData.getProcedureColumns()', forcing them to return metadata for procedures only.

Default Value	true
Since Version	5.1.26

- `noAccessToProcedureBodies`

  When determining procedure parameter types for 'CallableStatement', and the connected user can't access procedure bodies through "SHOW CREATE PROCEDURE" or SELECT on mysql.proc

should the driver instead create basic metadata, with all parameters reported as INOUT VARCHARs, instead of throwing an exception?

Default Value	false
Since Version	5.0.3

- `nullDatabaseMeansCurrent`

  In 'DatabaseMetaData' methods that take a 'catalog' or 'schema' parameter, does the value "null" mean to use the current database? See also the property 'databaseTerm'.

Default Value	false
Since Version	3.1.8

- `useHostsInPrivileges`

  Add '@hostname' to users in 'DatabaseMetaData.getColumn/TablePrivileges()'.

Default Value	true
Since Version	3.0.2

- `useInformationSchema`

  Should the driver use the INFORMATION_SCHEMA to derive information used by 'DatabaseMetaData'? Default is "true" when connecting to MySQL 8.0.3+, otherwise default is "false".

Default Value	false
Since Version	5.0.0

6.3.10 BLOB/CLOB processing

- `blobSendChunkSize`

  Chunk size to use when sending BLOB/CLOBs via server-prepared statements. Note that this value cannot exceed the value of 'maxAllowedPacket' and, if that is the case, then this value will be corrected automatically.

Default Value	1048576
Since Version	3.1.9

- `blobsAreStrings`

  Should the driver always treat BLOBs as Strings - specifically to work around dubious metadata returned by the server for GROUP BY clauses?

Default Value	false
Since Version	5.0.8

- `clobCharacterEncoding`

  The character encoding to use for sending and retrieving TEXT, MEDIUMTEXT and LONGTEXT values instead of the configured connection 'characterEncoding'.

Since Version	5.0.0

- `emulateLocators`

Should the driver emulate 'java.sql.Blob' with locators? With this feature enabled, the driver will delay loading the actual Blob data until the one of the retrieval methods ('getInputStream()', 'getBytes()', and so forth) on the blob data stream has been accessed. For this to work, you must use a column alias with the value of the column to the actual name of the Blob. The feature also has the following restrictions: The SELECT that created the result set must reference only one table, the table must have a primary key; the SELECT must alias the original blob column name, specified as a string, to an alternate name; the SELECT must cover all columns that make up the primary key.

Default Value	false
Since Version	3.1.0

- `functionsNeverReturnBlobs`

Should the driver always treat data from functions returning BLOBs as Strings - specifically to work around dubious metadata returned by the server for "GROUP BY" clauses?

Default Value	false
Since Version	5.0.8

- `locatorFetchBufferSize`

If 'emulateLocators' is configured to "true", what size buffer should be used when fetching BLOB data for 'getBinaryInputStream()'?

Default Value	1048576
Since Version	3.2.1

6.3.11 Datetime types processing

- `connectionTimeZone`

Configures the connection time zone which is used by Connector/J if conversion between the JVM default and a target time zone is needed when preserving instant temporal values.

Accepts a geographic time zone name or a time zone offset from Greenwich/UTC, using a syntax 'java.time.ZoneId' is able to parse, or one of the two logical values "LOCAL" and "SERVER". Default is "LOCAL". If set to an explicit time zone then it must be one that either the JVM or both the JVM and MySQL support. If set to "LOCAL" then the driver assumes that the connection time zone is the same as the JVM default time zone. If set to "SERVER" then the driver attempts to detect the session time zone from the values configured on the MySQL server session variables 'time_zone' or 'system_time_zone'. The time zone detection and subsequent mapping to a Java time zone may fail due to several reasons, mostly because of time zone abbreviations being used, in which case an explicit time zone must be set or a different time zone must be configured on the server.

This option itself does not set MySQL server session variable 'time_zone' to the given value. To do that the 'forceConnectionTimeZoneToSession' connection option must be set to "true".

Please note that setting a value to 'connectionTimeZone' in conjunction with "forceConnectionTimeZoneToSession=false" and "preserveInstants=false" has no effect since, in this case, neither this option is used to change the session time zone nor used for time zone conversions of time-based data.

Former connection option 'serverTimezone' is still valid as an alias of this one but may be deprecated in the future.

See also 'forceConnectionTimeZoneToSession' and 'preserveInstants' for more details.

Since Version	3.0.2

- `forceConnectionTimeZoneToSession`

  If enabled, sets the time zone value determined by 'connectionTimeZone' connection property to the current server session 'time_zone' variable. If the time zone value is given as a geographical time zone, then Connector/J sets this value as-is in the server session, in which case the time zone system tables must be populated beforehand (consult the MySQL Server documentation for further details); but, if the value is given as an offset from Greenwich/UTC in any of the supported syntaxes, then the server session time zone is set as a numeric offset from UTC.

  With that no intermediate conversion between JVM default time zone and connection time zone is needed to store correct milliseconds value of instant Java objects such as 'java.sql.Timestamp' or 'java.time.OffsetDateTime' when stored in TIMESTAMP columns.

  Note that it also affects the result of MySQL functions such as 'NOW()', 'CURTIME()' or 'CURDATE()'.

  This option has no effect if used in conjunction with "connectionTimeZone=SERVER" since, in this case, the session is already set with the required time zone.

  See also 'connectionTimeZone' and 'preserveInstants' for more details.

Default Value	false
Since Version	8.0.23

- `noDatetimeStringSync`

  Don't ensure that 'ResultSet.getTimestamp().toString().equals(ResultSet.getString())'.

Default Value	false
Since Version	3.1.7

- `preserveInstants`

  If enabled, Connector/J does its best to preserve the instant point on the time-line for Java instant-based objects such as 'java.sql.Timestamp' or 'java.time.OffsetDateTime' instead of their original visual form. Otherwise, the driver always uses the JVM default time zone for rendering the values it sends to the server and for constructing the Java objects from the fetched data.

  MySQL uses implied time zone conversion for TIMESTAMP values: they are converted from the session time zone to UTC for storage, and back from UTC to the session time zone for retrieval. So, to store the correct correct UTC value internally, the driver converts the value from the original time zone to the session time zone before sending to the server. On retrieval, Connector/J converts the received value from the session time zone to the JVM default one.

  When storing, the conversion is performed only if the target 'SQLType', either the explicit one or the default one, is TIMESTAMP. When retrieving, the conversion is performed only if the source column has the TIMESTAMP, DATETIME or character type and the target class is an instant-based one, like 'java.sql.Timestamp' or 'java.time.OffsetDateTime'.

  Note that this option has no effect if used in conjunction with "connectionTimeZone=LOCAL" since, in this case, the source and target time zones are the same. Though, in this case, it's still possible to store a correct instant value if set together with "forceConnectionTimeZoneToSession=true".

  See also 'connectionTimeZone' and 'forceConnectionTimeZoneToSession' for more details.

Default Value	true

Since Version	8.0.23

- `sendFractionalSeconds`

  If set to "false", the fractional seconds will always be truncated before sending any data to the server. This option applies only to prepared statements, callable statements or updatable result sets.

Default Value	true
Since Version	5.1.37

- `sendFractionalSecondsForTime`

  If set to "false", the fractional seconds of 'java.sql.Time' will be ignored as required by JDBC specification. If set to "true", its value is rendered with fractional seconds allowing to store milliseconds into MySQL TIME column. This option applies only to prepared statements, callable statements or updatable result sets. It has no effect if "sendFractionalSeconds=false".

Default Value	true
Since Version	8.0.23

- `treatMysqlDatetimeAsTimestamp`

  Should the driver treat the MySQL DATETIME type as TIMESTAMP in 'ResultSet.getObject()'? Enabling this option changes the default MySQL data type to Java type mapping for DATETIME from 'java.time.LocalDateTime' to 'java.sql.Timestamp'. Given the nature of the DATETIME type and its inability to represent instant values, it is not advisable to enable this option unless the driver is used with a framework or API that expects exclusively objects following the default MySQL data types to Java types mapping, which is the case of, for example, 'javax.sql.rowset.CachedRowSet'.

Default Value	false
Since Version	8.2.0

- `treatUtilDateAsTimestamp`

  Should the driver treat 'java.util.Date' as a TIMESTAMP in 'PreparedStatement.setObject()'?

Default Value	true
Since Version	5.0.5

- `yearIsDateType`

  Should the JDBC driver treat the MySQL type YEAR as a 'java.sql.Date', or as a SHORT?

Default Value	true
Since Version	3.1.9

- `zeroDateTimeBehavior`

  What should happen when the driver encounters DATETIME values that are composed entirely of zeros - used by MySQL to represent invalid dates? Valid values are "EXCEPTION", "ROUND" and "CONVERT_TO_NULL".

Default Value	EXCEPTION
Since Version	3.1.4

6.3.12 High Availability and Clustering

- `autoReconnect`

  Should the driver try to re-establish stale and/or dead connections? If enabled the driver will throw an exception for queries issued on a stale or dead connection, which belong to the current transaction, but will attempt reconnect before the next query issued on the connection in a new transaction. The use of this feature is not recommended, because it has side effects related to session state and data consistency when applications don't handle SQLExceptions properly, and is only designed to be used when you are unable to configure your application to handle SQLExceptions resulting from dead and stale connections properly. Alternatively, as a last option, investigate setting the MySQL server variable 'wait_timeout' to a high value, rather than the default of 8 hours.

Default Value	false
Since Version	1.1

- `autoReconnectForPools`

  Use a reconnection strategy appropriate for connection pools?

Default Value	false
Since Version	3.1.3

- `failOverReadOnly`

  When failing over in 'autoReconnect' mode, should the connection be set to 'read-only'?

Default Value	true
Since Version	3.0.12

- `maxReconnects`

  Maximum number of reconnects to attempt if 'autoReconnect' is "true".

Default Value	3
Since Version	1.1

- `reconnectAtTxEnd`

  If 'autoReconnect' is set to "true", should the driver attempt reconnections at the end of every transaction?

Default Value	false
Since Version	3.0.10

- `retriesAllDown`

  When using load balancing or failover, the number of times the driver should cycle through available hosts, attempting to connect. Between cycles, the driver will pause for 250 ms if no servers are available.

Default Value	120
Since Version	5.1.6

- `initialTimeout`

  If 'autoReconnect' is enabled, the initial time to wait between re-connect attempts (in seconds, defaults to "2").

Default Value	2
Since Version	1.1

- `queriesBeforeRetrySource`

  When using multi-host failover, the number of queries to issue before falling back to the primary host when failed over. Whichever condition is met first, 'queriesBeforeRetrySource' or 'secondsBeforeRetrySource' will cause an attempt to be made to reconnect to the primary host. Setting both properties to "0" disables the automatic fall back to the primary host at transaction boundaries.

Default Value	50
Since Version	3.0.2

- `secondsBeforeRetrySource`

  How long, in seconds, should the driver wait when failed over, before attempting to reconnect to the primary host? Whichever condition is met first, 'queriesBeforeRetrySource' or 'secondsBeforeRetrySource' will cause an attempt to be made to reconnect to the source host. Setting both properties to "0" disables the automatic fall back to the primary host at transaction boundaries.

Default Value	30
Since Version	3.0.2

- `allowReplicaDownConnections`

  By default, a replication-aware connection will fail to connect when configured replica hosts are all unavailable at initial connection. Setting this property to "true" allows to establish the initial connection. It won't prevent failures when switching to replicas i.e. by setting the replication connection to read-only state. The property 'readFromSourceWhenNoReplicas' should be used for this purpose.

Default Value	false
Since Version	6.0.2

- `allowSourceDownConnections`

  By default, a replication-aware connection will fail to connect when configured source hosts are all unavailable at initial connection. Setting this property to "true" allows to establish the initial connection, by failing over to the replica servers, in read-only state. It won't prevent subsequent failures when switching back to the source hosts i.e. by setting the replication connection to read/write state.

Default Value	false
Since Version	5.1.27

- `ha.enableJMX`

  Enables JMX-based management of load-balanced connection groups, including live addition/removal of hosts from load-balancing pool. Enables JMX-based management of replication

connection groups, including live replica promotion, addition of new replicas and removal of source or replica hosts from load-balanced source and replica connection pools.

Default Value	false
Since Version	5.1.27

- `loadBalanceHostRemovalGracePeriod`

  Sets the grace period to wait for a host being removed from a load-balanced connection, to be released when it is currently the active host.

Default Value	15000
Since Version	6.0.3

- `readFromSourceWhenNoReplicas`

  Replication-aware connections distribute load by using the source hosts when in read/write state and by using the replica hosts when in read-only state. If, when setting the connection to read-only state, none of the replica hosts are available, an 'SQLException' is thrown back. Setting this property to "true" allows to fail over to the source hosts, while setting the connection state to read-only, when no replica hosts are available at switch instant.

Default Value	false
Since Version	6.0.2

- `selfDestructOnPingMaxOperations`

  If set to a non-zero value, the driver will report close the connection and report failure when 'com.mysql.cj.jdbc.JdbcConnection.ping()' or 'java.sql.Connection.isValid(int)' is called if the connection's count of commands sent to the server exceeds this value.

Default Value	0
Since Version	5.1.6

- `selfDestructOnPingSecondsLifetime`

  If set to a non-zero value, the driver will close the connection and report failure when 'com.mysql.cj.jdbc.JdbcConnection.ping()' or 'java.sql.Connection.isValid(int)' is called if the connection's lifetime exceeds this value, specified in milliseconds.

Default Value	0
Since Version	5.1.6

- `ha.loadBalanceStrategy`

  If using a load-balanced connection to connect to SQL servers in a MySQL Cluster configuration (by using the URL prefix "jdbc:mysql:loadbalance://"), which load balancing algorithm should the driver use: (1) "random" - the driver will pick a random host for each request. This tends to work better than round-robin, as the randomness will somewhat account for spreading loads where requests vary in response time, while round-robin can sometimes lead to overloaded nodes if there are variations in response times across the workload. (2) "bestResponseTime" - the driver will route the request to the host that had the best response time for the previous transaction. (3) "serverAffinity" - the driver initially attempts to enforce server affinity while still respecting and benefiting from the fault tolerance aspects of the load-balancing implementation. The server affinity ordered list is provided using the

property 'serverAffinityOrder'. If none of the servers listed in the affinity list is responsive, the driver then refers to the "random" strategy to proceed with choosing the next server.

Default Value	random
Since Version	5.0.6

- `loadBalanceAutoCommitStatementRegex`

  When load-balancing is enabled for auto-commit statements (via 'loadBalanceAutoCommitStatementThreshold'), the statement counter will only increment when the SQL matches the regular expression. By default, every statement issued matches.

Since Version	5.1.15

- `loadBalanceAutoCommitStatementThreshold`

  When auto-commit is enabled, the number of statements which should be executed before triggering load-balancing to rebalance. Default value of "0" causes load-balanced connections to only rebalance when exceptions are encountered, or auto-commit is disabled and transactions are explicitly committed or rolled back.

Default Value	0
Since Version	5.1.15

- `loadBalanceBlocklistTimeout`

  Time in milliseconds between checks of servers which are unavailable, by controlling how long a server lives in the global blocklist.

Default Value	0
Since Version	5.1.0

- `loadBalanceConnectionGroup`

  Logical group of load-balanced connections within a class loader, used to manage different groups independently. If not specified, live management of load-balanced connections is disabled.

Since Version	5.1.13

- `loadBalanceExceptionChecker`

  Fully-qualified class name of custom exception checker. The class must implement 'com.mysql.cj.jdbc.ha.LoadBalanceExceptionChecker' interface, and is used to inspect 'SQLException' exceptions and determine whether they should trigger fail-over to another host in a load-balanced deployment.

Default Value	com.mysql.cj.jdbc.ha.StandardLoadBalanceExceptionChecker
Since Version	5.1.13

- `loadBalancePingTimeout`

  Time in milliseconds to wait for ping responses from each of load-balanced physical connections when using a load-balanced connection.

Default Value	0
Since Version	5.1.13

- `loadBalanceSQLExceptionSubclassFailover`

Comma-delimited list of classes/interfaces used by default load-balanced exception checker to determine whether a given 'SQLException' should trigger a failover. The comparison is done using 'Class.isInstance(SQLException)' using the 'SQLException' thrown.

Since Version	5.1.13

- `loadBalanceSQLStateFailover`

  Comma-delimited list of 'SQLState' codes used by the default load-balanced exception checker to determine whether a given 'SQLException' should trigger a failover. The 'SQLState' of a given 'SQLException' is evaluated to determine whether it begins with any of the values specified in the comma-delimited list.

Since Version	5.1.13

- `loadBalanceValidateConnectionOnSwapServer`

  Should the load-balanced connection explicitly check whether the connection is live when swapping to a new physical connection at commit/rollback?

Default Value	false
Since Version	5.1.13

- `pinGlobalTxToPhysicalConnection`

  When using XA connections, should the driver ensure that operations on a given XID are always routed to the same physical connection? This allows the 'XAConnection' to support "XA START ... JOIN" after "XA END" has been called.

Default Value	false
Since Version	5.0.1

- `replicationConnectionGroup`

  Logical group of replication connections within a class loader, used to manage different groups independently. If not specified, live management of replication connections is disabled.

Since Version	8.0.7

- `resourceId`

  A globally unique name that identifies the resource that this data source or connection is connected to, used for 'XAResource.isSameRM()' when the driver can't determine this value based on hostnames used in the URL.

Since Version	5.0.1

- `serverAffinityOrder`

  A comma separated list containing the host/port pairs that are to be used in load-balancing "serverAffinity" strategy. Only the sub-set of the hosts enumerated in the main hosts section in this URL will be used and they must be identical in case and type, i.e., can't use an IP address in one place and the corresponding host name in the other.

Since Version	8.0.8

6.3.13 Performance Extensions

- `callableStmtCacheSize`

  If 'cacheCallableStmts' is enabled, how many callable statements should be cached?

Default Value	100
Since Version	3.1.2

- `metadataCacheSize`

  The number of queries to cache 'ResultSetMetadata' for if 'cacheResultSetMetaData' is set to "true".

Default Value	50
Since Version	3.1.1

- `useLocalSessionState`

  Should the driver refer to the internal values of auto-commit and transaction isolation that are set by 'Connection.setAutoCommit()' and 'Connection.setTransactionIsolation()' and transaction state as maintained by the protocol, rather than querying the database or blindly sending commands to the database for 'commit()' or 'rollback()' method calls?

Default Value	false
Since Version	3.1.7

- `useLocalTransactionState`

  Should the driver use the in-transaction state provided by the MySQL protocol to determine if a 'commit()' or 'rollback()' should actually be sent to the database?

Default Value	false
Since Version	5.1.7

- `prepStmtCacheSize`

  If prepared statement caching is enabled, how many prepared statements should be cached?

Default Value	25
Since Version	3.0.10

- `prepStmtCacheSqlLimit`

  If prepared statement caching is enabled, what's the largest SQL the driver will cache the parsing for?

Default Value	256
Since Version	3.0.10

- `queryInfoCacheFactory`

  Name of a class implementing 'com.mysql.cj.CacheAdapterFactory' which will be used to create caches for the parsed representation of prepared statements. Prior to version 8.0.29, this property was named 'parseInfoCacheFactory', which remains as an alias.

Default Value	com.mysql.cj.PerConnectionLRUFactory
Since Version	5.1.1

- `serverConfigCacheFactory`

  Name of a class implementing 'com.mysql.cj.CacheAdapterFactory', which will be used to create caches for MySQL server configuration values.

Default Value	com.mysql.cj.util.PerVmServerConfigCacheFactory
Since Version	5.1.1

- `alwaysSendSetIsolation`

  Should the driver always communicate with the database when 'Connection.setTransactionIsolation()' is called? If set to "false", the driver will only communicate with the database when the requested transaction isolation is different than the whichever is newer, the last value that was set via 'Connection.setTransactionIsolation()', or the value that was read from the server when the connection was established. Note that "useLocalSessionState=true" will force the same behavior as "alwaysSendSetIsolation=false", regardless of how 'alwaysSendSetIsolation' is set.

Default Value	true
Since Version	3.1.7

- `maintainTimeStats`

  Should the driver maintain various internal timers to enable idle time calculations as well as more verbose error messages when the connection to the server fails? Setting this property to false removes at least two calls to 'System.getCurrentTimeMillis()' per query.

Default Value	true
Since Version	3.1.9

- `useCursorFetch`

  Should the driver use cursor-based fetching to retrieve rows? If set to "true" and 'defaultFetchSize' is set to a value higher than zero or 'setFetchSize()' with a value higher than zero is called on a statement, then the cursor-based result set will be used. Please note that 'useServerPrepStmts' is automatically set to "true" in this case because cursor functionality is available only for server-side prepared statements.

Default Value	false
Since Version	5.0.0

- `cacheCallableStmts`

  Should the driver cache the parsing stage of CallableStatements?

Default Value	false
Since Version	3.1.2

- `cachePrepStmts`

  Should the driver cache the parsing stage of PreparedStatements of client-side prepared statements, the "check" for suitability of server-side prepared and server-side prepared statements themselves?

Default Value	false
Since Version	3.0.10

- `cacheResultSetMetadata`

Should the driver cache 'ResultSetMetaData' for statements and prepared statements?

Default Value	false
Since Version	3.1.1

- `cacheServerConfiguration`

  Should the driver cache the results of "SHOW VARIABLES" and "SHOW COLLATION" on a per-URL basis?

Default Value	false
Since Version	3.1.5

- `defaultFetchSize`

  The driver will call 'setFetchSize(n)' with this value on all newly-created statements.

Default Value	0
Since Version	3.1.9

- `dontCheckOnDuplicateKeyUpdateInSQL`

  Stops checking if every INSERT statement contains the "ON DUPLICATE KEY UPDATE" clause. As a side effect, obtaining the statement's generated keys information will return a list where normally it would not. Also be aware that, in this case, the list of generated keys returned may not be accurate. The effect of this property is canceled if set simultaneously with "rewriteBatchedStatements=true".

Default Value	false
Since Version	5.1.32

- `elideSetAutoCommits`

  Should the driver only issue 'set autocommit=n' queries when the server's state doesn't match the requested state by 'Connection.setAutoCommit(boolean)'?

Default Value	false
Since Version	3.1.3

- `enableEscapeProcessing`

  Sets the default escape processing behavior for Statement objects. The method 'Statement.setEscapeProcessing()' can be used to specify the escape processing behavior for an individual statement object. Default escape processing behavior in prepared statements must be defined with the property 'processEscapeCodesForPrepStmts'.

Default Value	true
Since Version	6.0.1

- `enableQueryTimeouts`

  When enabled, query timeouts set via 'Statement.setQueryTimeout()' use a shared 'java.util.Timer' instance for scheduling. Even if the timeout doesn't expire before the query is processed, there will be memory used by the 'TimerTask' for the given timeout which won't be reclaimed until the time the

timeout would have expired if it hadn't been cancelled by the driver. High-load environments might want to consider disabling this functionality.

Default Value	true
Since Version	5.0.6

- `largeRowSizeThreshold`

  What size result set row should the JDBC driver consider large, and thus use a more memory-efficient way of representing the row internally?

Default Value	2048
Since Version	5.1.1

- `readOnlyPropagatesToServer`

  Should the driver issue appropriate statements to implicitly set the transaction access mode on server side when 'Connection.setReadOnly()' is called? Setting this property to "true" enables InnoDB read-only potential optimizations but also requires an extra roundtrip to set the right transaction state. Even if this property is set to "false", the driver will do its best effort to prevent the execution of database-state-changing queries.

Default Value	true
Since Version	5.1.35

- `rewriteBatchedStatements`

  Should the driver use multi-queries, regardless of the setting of 'allowMultiQueries', as well as rewriting of prepared statements for INSERT and REPLACE queries into multi-values clause statements when 'executeBatch()' is called?

  Notice that this might allow SQL injection when using plain statements and the provided input is not properly sanitized. Also notice that for prepared statements, if the stream length is not specified when using 'PreparedStatement.set*Stream()', the driver would not be able to determine the optimum number of parameters per batch and might return an error saying that the resultant packet is too large.

  'Statement.getGeneratedKeys()', for statements that are rewritten only works when the entire batch consists of INSERT or REPLACE statements.

  Be aware that when using "rewriteBatchedStatements=true" with "INSERT ... ON DUPLICATE KEY UPDATE" for rewritten statements, the server returns only one value for all affected (or found) rows in the batch, and it is not possible to map it correctly to the initial statements; in this case the driver returns "0" as the result for each batch statement if total count was zero, and 'Statement.SUCCESS_NO_INFO' if total count was above zero.

Default Value	false
Since Version	3.1.13

- `useReadAheadInput`

  Use optimized non-blocking buffered input stream when reading from the server?

Default Value	true
Since Version	3.1.5

6.3.14 Debugging/Profiling

- `logger`

  The name of a class that implements 'com.mysql.cj.log.Log' that will be used to log messages to. (default is 'com.mysql.cj.log.StandardLogger', which logs to STDERR).

Default Value	com.mysql.cj.log.StandardLogger
Since Version	3.1.1

- `profilerEventHandler`

  Name of a class that implements the interface 'com.mysql.cj.log.ProfilerEventHandler' that will be used to handle profiling/tracing events.

Default Value	com.mysql.cj.log.LoggingProfilerEventHandler
Since Version	5.1.6

- `useNanosForElapsedTime`

  For profiling/debugging functionality that measures elapsed time, should the driver try to use nanoseconds resolution?

Default Value	false
Since Version	5.0.7

- `maxQuerySizeToLog`

  Controls the maximum length of the part of a query that will get logged when profiling or tracing.

Default Value	2048
Since Version	3.1.3

- `maxByteArrayAsHex`

  Maximum size for a byte array parameter in a prepared statement that is converted to a hexadecimal literal when interpolated by 'JdbcPreparedStatement.toString()'. Any byte arrays larger than this value are interpolated generically as "** BYTE ARRAY DATA **".

Default Value	1024
Since Version	8.0.31

- `profileSQL`

  Trace queries and their execution/fetch times to the configured 'profilerEventHandler'.

Default Value	false
Since Version	3.1.0

- `logSlowQueries`

  Should queries that take longer than 'slowQueryThresholdMillis' or detected by the 'autoSlowLog' monitoring be reported to the registered 'profilerEventHandler'?

Default Value	false

- `slowQueryThresholdMillis`

  If 'logSlowQueries' is enabled, how long, in milliseconds, should a query take before it is logged as slow?

Default Value	2000
Since Version	3.1.2

- `slowQueryThresholdNanos`

  If 'logSlowQueries' is enabled, 'useNanosForElapsedTime' is set to "true", and this property is set to a non-zero value, the driver will use this threshold, in nanosecond units, to determine if a query was slow.

Default Value	0
Since Version	5.0.7

- `autoSlowLog`

  Instead of using 'slowQueryThreshold*' to determine if a query is slow enough to be logged, maintain statistics that allow the driver to determine queries that are outside the 99th percentile?

Default Value	true
Since Version	5.1.4

- `explainSlowQueries`

  If 'logSlowQueries' is enabled, should the driver automatically issue an 'EXPLAIN' on the server and send the results to the configured logger at a WARN level?

Default Value	false
Since Version	3.1.2

- `gatherPerfMetrics`

  Should the driver gather performance metrics, and report them via the configured logger every 'reportMetricsIntervalMillis' milliseconds?

Default Value	false
Since Version	3.1.2

- `reportMetricsIntervalMillis`

  If 'gatherPerfMetrics' is enabled, how often should they be logged (in milliseconds)?

Default Value	30000
Since Version	3.1.2

- `logXaCommands`

  Should the driver log XA commands sent by 'MysqlXaConnection' to the server, at the DEBUG level of logging?

Default Value	false
Since Version	5.0.5

- `traceProtocol`

Should the network protocol be logged at the TRACE level?

Default Value	false
Since Version	3.1.2

- `enablePacketDebug`

  When enabled, a ring-buffer of 'packetDebugBufferSize' packets will be kept, and dumped when exceptions are thrown in key areas in the driver's code.

Default Value	false
Since Version	3.1.3

- `packetDebugBufferSize`

  The maximum number of packets to retain when 'enablePacketDebug' is "true".

Default Value	20
Since Version	3.1.3

- `useUsageAdvisor`

  Should the driver issue usage warnings advising proper and efficient usage of JDBC and MySQL Connector/J to the 'profilerEventHandler'?

Default Value	false
Since Version	3.1.1

- `resultSetSizeThreshold`

  If 'useUsageAdvisor' is "true", how many rows should a result set contain before the driver warns that it is suspiciously large?

Default Value	100
Since Version	5.0.5

- `autoGenerateTestcaseScript`

  Should the driver dump the SQL it is executing, including server-side prepared statements to STDERR?

Default Value	false
Since Version	3.1.9

6.3.15 Exceptions/Warnings

- `dumpQueriesOnException`

  Should the driver dump the contents of the query sent to the server in the message for SQLExceptions?

Default Value	false
Since Version	3.1.3

- `exceptionInterceptors`

Comma-delimited list of classes that implement the interface 'com.mysql.cj.exceptions.ExceptionInterceptor'. These classes will be instantiated one per 'Connection' instance, and all 'SQLException' exceptions thrown by the driver will be allowed to be intercepted by these interceptors, in a chained fashion, with the first class listed as the head of the chain.

Since Version	5.1.8

- `ignoreNonTxTables`

  Ignore non-transactional table warning for rollback?

Default Value	false
Since Version	3.0.9

- `includeInnodbStatusInDeadlockExceptions`

  Include the output of "SHOW ENGINE INNODB STATUS" in exception messages when deadlock exceptions are detected?

Default Value	false
Since Version	5.0.7

- `includeThreadDumpInDeadlockExceptions`

  Include current Java thread dump in exception messages when deadlock exceptions are detected?

Default Value	false
Since Version	5.1.15

- `includeThreadNamesAsStatementComment`

  Include the name of the current thread as a comment visible in "SHOW PROCESSLIST", or in Innodb deadlock dumps, useful in correlation with "includeInnodbStatusInDeadlockExceptions=true" and "includeThreadDumpInDeadlockExceptions=true".

Default Value	false
Since Version	5.1.15

- `useOnlyServerErrorMessages`

  Don't prepend standard 'SQLState' error messages to error messages returned by the server.

Default Value	true
Since Version	3.0.15

6.3.16 Tunes for integration with other products

- `overrideSupportsIntegrityEnhancementFacility`

  Should the driver return "true" for 'DatabaseMetaData.supportsIntegrityEnhancementFacility()' even if the database doesn't support it to workaround applications that require this method to return "true" to signal support of foreign keys, even though the SQL specification states that this facility contains much more than just foreign key support (one such application being OpenOffice)?

Default Value	false

Since Version	3.1.12

- `ultraDevHack`

  Create prepared statements for 'prepareCall()' when required, because UltraDev is broken and issues a 'prepareCall()' for all statements?

Default Value	false
Since Version	2.0.3

6.3.17 JDBC compliance

- `useColumnNamesInFindColumn`

  Prior to JDBC-4.0, the JDBC specification had a bug related to what could be given as a column name to result set methods like 'findColumn()', or getters that took a String property. JDBC-4.0 clarified "column name" to mean the label, as given in an "AS" clause and returned by 'ResultSetMetaData.getColumnLabel()', and if no "AS" clause is specified, the column name. Setting this property to "true" will result in a behavior that is congruent to JDBC-3.0 and earlier versions of the JDBC specification, but which could have unexpected results. This property is preferred over 'useOldAliasMetadataBehavior' unless in need of the specific behavior that it provides with respect to 'ResultSetMetadata'.

Default Value	false
Since Version	5.1.7

- `pedantic`

  Follow the JDBC specification to the letter.

Default Value	false
Since Version	3.0.0

- `useOldAliasMetadataBehavior`

  Should the driver use the legacy behavior for "AS" clauses on columns and tables, and only return aliases ,if any, for 'ResultSetMetaData.getColumnName()' or 'ResultSetMetaData.getTableName()' rather than the original column/table name?

Default Value	false
Since Version	5.0.4

6.3.18 X Protocol and X DevAPI

- `xdevapi.auth`

  Authentication mechanism to use with the X Protocol. Allowed values are "SHA256_MEMORY", "MYSQL41", "PLAIN", and "EXTERNAL". Value is case insensitive. If the property is not set, the mechanism is chosen depending on the connection type: "PLAIN" is used for TLS connections and "SHA256_MEMORY" or "MYSQL41" is used for unencrypted connections.

Default Value	PLAIN
Since Version	8.0.8

- `xdevapi.compression`

X DevAPI-specific network traffic compression. This option accepts one of the three values: "PREFERRED", "REQUIRED", and "DISABLED". Setting this option to "PREFERRED" or "REQUIRED" enables compression algorithm negotiation between Connector and Server, and turns on compression of large X Protocol packets, as long as a consensus is reached between client and server regarding the compression algorithm to use. If a consensus cannot be reached, connection fails if the option is set to "REQUIRED" and continues without compression if the option is set to "PREFERRED". Setting this option as "DISABLED" skips the compression negotiation phase and forbids the interchange of compressed messages between client and server.

Default Value	PREFERRED
Since Version	8.0.20

- `xdevapi.compression-algorithms`

  A comma-delimited list of compression algorithms, each one identified by its name and operating mode, (e.g. "lz4_message"; consult the description for the MySQL global variable 'mysqlx_compression_algorithms' for a list of supported and enabled algorithms), that defines the order and which algorithms will be attempted when negotiating connection compression with the server.

  The compression algorithm 'deflate_stream' is supported natively. Additional compression algorithms require using third-party libraries and enabling them with the connection property 'xdevapi.compression-extensions'.

  This option is meaningful only when network traffic compression is enabled using the connection property 'xdevapi.compression'.

  As an alternative to the default algorithm names, that contain a reference to the compression operation mode, the aliases "zstd", "lz4", and "deflate" can be used instead of "zstd_stream", "lz4_message", and "deflate_stream".

Default Value	zstd_stream,lz4_message,deflate_stream
Since Version	8.0.22

- `xdevapi.compression-extensions`

  A comma-delimited list of triplets, with their elements delimited by colon, that enables the support for additional compression algorithms. Each triplet must contain: first, an algorithm name and operating mode (e.g. "lz4_message"; consult the description for the MySQL global variable 'mysqlx_compression_algorithms' for a list of supported and enabled algorithms); second, a fully-qualified class name of a class implementing the interface 'java.io.InputStream' that will be used to inflate data compressed with the named algorithm; third, a fully-qualified class name of a class implementing the interface 'java.io.OutputStream' that will be used to deflate data using the named

algorithm. Along with this setting, the library containing implementations of the designated classes must be available in the application's class path.

Any number of triplets defining compression algorithms and their inflater and deflater implementations can be provided but only the ones supported and enabled on the MySQL Server can be used.

The compression algorithm 'deflate_stream' is supported natively. Additional compression algorithms require using third-party libraries.

This option is meaningful only when network traffic compression is enabled using the connection property 'xdevapi.compression'.

As an alternative to the default algorithm names, that contain a reference to the compression operation mode, the aliases "zstd", "lz4", and "deflate" can be used instead of "zstd_stream", "lz4_message", and "deflate_stream".

Since Version	8.0.22

- `xdevapi.connect-timeout`

  X DevAPI-specific timeout, in milliseconds, for socket connect, with "0" being no timeout. If 'xdevapi.connect-timeout' is not set explicitly and 'connectTimeout' is, 'xdevapi.connect-timeout' takes up the value of 'connectTimeout'.

Default Value	10000
Since Version	8.0.13

- `xdevapi.connection-attributes`

  An X DevAPI-specific comma-delimited list of user-defined "key=value" pairs, in addition to standard X Protocol-defined "key=value" pairs, to be passed to MySQL Server for display as connection attributes in the 'PERFORMANCE_SCHEMA' tables 'session_account_connect_attrs' and 'session_connect_attrs'. Example usage: "xdevapi.connection-attributes=key1=value1,key2=value2" or "xdevapi.connection-attributes=[key1=value1,key2=value2]". This functionality is available for use with MySQL Server version 8.0.16 or later only. Earlier versions of X Protocol do not support connection attributes, causing this configuration option to be ignored. For situations where Session creation/initialization speed is critical, setting "xdevapi.connection-attributes=false" will cause connection attribute processing to be bypassed.

Since Version	8.0.16

- `xdevapi.dns-srv`

  X DevAPI-specific option for instructing the driver use the given host name to lookup for DNS SRV records and use the resulting list of hosts in a multi-host failover connection. Note that a single host name and no port must be provided when this option is enabled.

Default Value	false
Since Version	8.0.19

- `xdevapi.fallback-to-system-keystore`

  X DevAPI-specific switch to specify whether in the absence of a set value for 'xdevapi.ssl-keystore' (or 'clientCertificateKeyStoreUrl'), Connector/J falls back to using the system-wide key store defined through the system properties 'javax.net.ssl.keyStore*'. If not specified, the value of 'fallbackToSystemKeyStore' is used.

Default Value	true

Since Version	8.0.22

- `xdevapi.fallback-to-system-truststore`

  X DevAPI-specific switch to specify whether in the absence of a set value for 'xdevapi.ssl-truststore' (or 'trustCertificateKeyStoreUrl'), Connector/J falls back to using the system-wide default trust store or one defined through the system properties 'javax.net.ssl.trustStore*'. If not specified, the value of 'fallbackToSystemTrustStore' is used.

Default Value	true
Since Version	8.0.22

- `xdevapi.ssl-keystore`

  X DevAPI-specific URL for the client certificate key store. If not specified, use 'clientCertificateKeyStoreUrl' value.

Since Version	8.0.22

- `xdevapi.ssl-keystore-password`

  X DevAPI-specific password for the client certificate key store. If not specified, use 'clientCertificateKeyStorePassword' value.

Since Version	8.0.22

- `xdevapi.ssl-keystore-type`

  X DevAPI-specific type of the client certificate key store. If not specified, use 'clientCertificateKeyStoreType' value.

Default Value	JKS
Since Version	8.0.22

- `xdevapi.ssl-mode`

  X DevAPI-specific SSL mode setting. If not specified, use 'sslMode'. Because the "PREFERRED" mode is not applicable to X Protocol, if 'xdevapi.ssl-mode' is not set and 'sslMode' is set to "PREFERRED", 'xdevapi.ssl-mode' is set to "REQUIRED".

Default Value	REQUIRED
Since Version	8.0.7

- `xdevapi.ssl-truststore`

  X DevAPI-specific URL for the trusted CA certificates key store. If not specified, use 'trustCertificateKeyStoreUrl' value.

Since Version	6.0.6

- `xdevapi.ssl-truststore-password`

  X DevAPI-specific password for the trusted CA certificates key store. If not specified, use 'trustCertificateKeyStorePassword' value.

Since Version	6.0.6

- `xdevapi.ssl-truststore-type`

X DevAPI-specific type of the trusted CA certificates key store. If not specified, use 'trustCertificateKeyStoreType' value.

Default Value	JKS
Since Version	6.0.6

- `xdevapi.tls-ciphersuites`

  X DevAPI-specific property overriding the cipher suites enabled for use on the underlying SSL sockets. If not specified, the value of 'enabledSSLCipherSuites' is used.

Since Version	8.0.19

- `xdevapi.tls-versions`

  X DevAPI-specific property that takes a list of TLS protocols to allow when creating secure sessions. Overrides the TLS protocols enabled in the underlying SSL socket. If not specified, then the value of 'tlsVersions' is used instead. Allowed and default values are "TLSv1.2" and "TLSv1.3".

Since Version	8.0.19

6.4 JDBC API Implementation Notes

MySQL Connector/J, as a rigorous implementation of the JDBC API, passes all of the tests in the publicly available version of Oracle's JDBC compliance test suite. The JDBC specification is flexible on how certain functionality should be implemented. This section gives details on an interface-by-interface level about implementation decisions that might affect how you code applications with MySQL Connector/J.

- **BLOB**

  You can emulate BLOBs with locators by adding the property `emulateLocators=true` to your JDBC URL. Using this method, the driver will delay loading the actual BLOB data until you retrieve the other data and then use retrieval methods (`getInputStream()`, `getBytes()`, and so forth) on the BLOB data stream.

  You must use a column alias with the value of the column to the actual name of the BLOB, for example:

  ```
  SELECT id, 'data' as blob_data from blobtable
  ```

  You must also follow these rules:

  - The `SELECT` must reference only one table. The table must have a primary key.
  - The `SELECT` must alias the original BLOB column name, specified as a string, to an alternate name.
  - The `SELECT` must cover all columns that make up the primary key.

  The BLOB implementation does not allow in-place modification (they are copies, as reported by the `DatabaseMetaData.locatorsUpdateCopies()` method). Because of this, use the corresponding `PreparedStatement.setBlob()` or `ResultSet.updateBlob()` (in the case of updatable result sets) methods to save changes back to the database.

- **Connection**

  The `isClosed()` method does not ping the server to determine if it is available. In accordance with the JDBC specification, it only returns true if `closed()` has been called on the connection. If you

need to determine if the connection is still valid, issue a simple query, such as `SELECT 1`. The driver will throw an exception if the connection is no longer valid.

- **DatabaseMetaData**

  Foreign key information (`getImportedKeys()`/`getExportedKeys()` and `getCrossReference()`) is only available from `InnoDB` tables. The driver uses `SHOW CREATE TABLE` to retrieve this information, so if any other storage engines add support for foreign keys, the driver would transparently support them as well.

- **PreparedStatement**

  Two variants of prepared statements are implemented by Connector/J, the client-side and the server-side prepared statements. Client-side prepared statements are used by default because early MySQL versions did not support the prepared statement feature or had problems with its implementation. Server-side prepared statements and binary-encoded result sets are used when the server supports them. To enable usage of server-side prepared statements, set `useServerPrepStmts=true`.

  Be careful when using a server-side prepared statement with **large** parameters that are set using `setBinaryStream()`, `setAsciiStream()`, `setUnicodeStream()`, `setCharacterStream()`, `setNCharacterStream()`, `setBlob()`, `setClob()`, or `setNCLob()`. To re-execute the statement with any large parameter changed to a nonlarge parameter, call `clearParameters()` and set all parameters again. The reason for this is as follows:

  - During both server-side prepared statements and client-side emulation, large data is exchanged only when `PreparedStatement.execute()` is called.
  - Once that has been done, the stream used to read the data on the client side is closed (as per the JDBC spec), and cannot be read from again.
  - If a parameter changes from large to nonlarge, the driver must reset the server-side state of the prepared statement to allow the parameter that is being changed to take the place of the prior large value. This removes all of the large data that has already been sent to the server, thus requiring the data to be re-sent, using the `setBinaryStream()`, `setAsciiStream()`, `setUnicodeStream()`, `setCharacterStream()`, `setNCharacterStream()`, `setBlob()`, `setClob()`, or `setNCLob()` method.

  Consequently, to change the type of a parameter to a nonlarge one, you must call `clearParameters()` and set all parameters of the prepared statement again before it can be re-executed.

- **ResultSet**

  By default, ResultSets are completely retrieved and stored in memory. In most cases this is the most efficient way to operate and, due to the design of the MySQL network protocol, is easier to implement. If you are working with ResultSets that have a large number of rows or large values and cannot allocate heap space in your JVM for the memory required, you can tell the driver to stream the results back one row at a time.

  To enable this functionality, create a `Statement` instance in the following manner:

```
stmt = conn.createStatement(java.sql.ResultSet.TYPE_FORWARD_ONLY,
              java.sql.ResultSet.CONCUR_READ_ONLY);
```

```
stmt.setFetchSize(Integer.MIN_VALUE);
```

The combination of a forward-only, read-only result set, with a fetch size of `Integer.MIN_VALUE` serves as a signal to the driver to stream result sets row-by-row. After this, any result sets created with the statement will be retrieved row-by-row.

There are some caveats with this approach. You must read all of the rows in the result set (or close it) before you can issue any other queries on the connection, or an exception will be thrown.

The earliest the locks these statements hold can be released (whether they be `MyISAM` table-level locks or row-level locks in some other storage engine such as `InnoDB`) is when the statement completes.

If the statement is within scope of a transaction, then locks are released when the transaction completes (which implies that the statement needs to complete first). As with most other databases, statements are not complete until all the results pending on the statement are read or the active result set for the statement is closed.

Therefore, if using streaming results, process them as quickly as possible if you want to maintain concurrent access to the tables referenced by the statement producing the result set.

Another alternative is to use cursor-based streaming to retrieve a set number of rows each time. This can be done by setting the connection property `useCursorFetch` to true, and then calling `setFetchSize(int)` with `int` being the desired number of rows to be fetched each time:

```
conn = DriverManager.getConnection("jdbc:mysql://localhost/?useCursorFetch=true", "user", "s3cr3t");
stmt = conn.createStatement();
stmt.setFetchSize(100);
rs = stmt.executeQuery("SELECT * FROM your_table_here");
```

- **Statement**

  Connector/J includes support for both `Statement.cancel()` and `Statement.setQueryTimeout()`. Both require a separate connection to issue the `KILL QUERY` statement. In the case of `setQueryTimeout()`, the implementation creates an additional thread to handle the timeout functionality.

  > **Note**
  >
  > Failures to cancel the statement for `setQueryTimeout()` may manifest themselves as `RuntimeException` rather than failing silently, as there is currently no way to unblock the thread that is executing the query being cancelled due to timeout expiration and have it throw the exception instead.

  MySQL does not support SQL cursors, and the JDBC driver does not emulate them, so `setCursorName()` has no effect.

  Connector/J also supplies two additional methods:

  - `setLocalInfileInputStream()` sets an `InputStream` instance that will be used to send data to the MySQL server for a `LOAD DATA LOCAL INFILE` statement rather than a `FileInputStream` or `URLInputStream` that represents the path given as an argument to the statement.

    This stream will be read to completion upon execution of a `LOAD DATA LOCAL INFILE` statement, and will automatically be closed by the driver, so it needs to be reset before each call

to `execute*()` that would cause the MySQL server to request data to fulfill the request for `LOAD DATA LOCAL INFILE`.

If this value is set to `NULL`, the driver will revert to using a `FileInputStream` or `URLInputStream` as required.

- `getLocalInfileInputStream()` returns the `InputStream` instance that will be used to send data in response to a `LOAD DATA LOCAL INFILE` statement.

  This method returns `NULL` if no such stream has been set using `setLocalInfileInputStream()`.

6.5 Java, JDBC, and MySQL Types

MySQL Connector/J is flexible in the way it handles conversions between MySQL data types and Java data types.

In general, any MySQL data type can be converted to a `java.lang.String`, and any numeric type can be converted to any of the Java numeric types, although round-off, overflow, or loss of precision may occur.

Connector/J issues warnings or throws `DataTruncation` exceptions as is required by the JDBC specification, unless the connection was configured not to do so by using the property `jdbcCompliantTruncation` and setting it to `false`.

The conversions that are always guaranteed to work are listed in the following table. The first column lists one or more MySQL data types, and the second column lists one or more Java types to which the MySQL types can be converted.

Table 6.19 Possible Conversions Between MySQL and Java Data Types

These MySQL Data Types	Can always be converted to these Java types
`CHAR, VARCHAR, BLOB, TEXT, ENUM, and SET`	`java.lang.String, java.io.InputStream, java.io.Reader, java.sql.Blob, java.sql.Clob`
`FLOAT, REAL, DOUBLE PRECISION, NUMERIC, DECIMAL, TINYINT, SMALLINT, MEDIUMINT, INTEGER, BIGINT`	`java.lang.String, java.lang.Short, java.lang.Integer, java.lang.Long, java.lang.Double, java.math.BigDecimal`
`DATE, TIME, DATETIME, TIMESTAMP`	`java.lang.String, java.sql.Date, java.sql.Timestamp`

> **Note**
>
> Round-off, overflow or loss of precision may occur if you choose a Java numeric data type that has less precision or capacity than the MySQL data type you are converting to/from.

The `ResultSet.getObject()` method uses the type conversions between MySQL and Java types, following the JDBC specification where appropriate. The values returned by `ResultSetMetaData.GetColumnTypeName()`and `ResultSetMetaData.GetColumnClassName()` are shown in the table below. For more information on the JDBC types, see the reference on the java.sql.Types class.

Table 6.20 MySQL Types and Return Values for ResultSetMetaData.GetColumnTypeName()and ResultSetMetaData.GetColumnClassName()

MySQL Type Name	Return value of `GetColumnTypeName`	Return value of `GetColumnClassName`
`BIT(1)`	`BIT`	`java.lang.Boolean`

MySQL Type Name	Return value of `GetColumnTypeName`	Return value of `GetColumnClassName`
`BIT( > 1)`	`BIT`	`byte[]`
`TINYINT(1) SIGNED, BOOLEAN`	If `tinyInt1isBit=true` and `transformedBitIsBoolean=false`: `BIT` If `tinyInt1isBit=true` and `transformedBitIsBoolean=true`: `BOOLEAN` If `tinyInt1isBit=false`: `TINYINT`	If `tinyInt1isBit=true` and `transformedBitIsBoolean=false`: `java.lang.Boolean` If `tinyInt1isBit=true` and `transformedBitIsBoolean=true`: `java.lang.Boolean` If `tinyInt1isBit=false`: `java.lang.Integer`
`TINYINT( > 1) SIGNED`	`TINYINT`	`java.lang.Integer`
`TINYINT( any ) UNSIGNED`	`TINYINT UNSIGNED`	`java.lang.Integer`
`SMALLINT[(M)] [UNSIGNED]`	`SMALLINT [UNSIGNED]`	`java.lang.Integer` (regardless of whether it is `UNSIGNED` or not)
`MEDIUMINT[(M)] [UNSIGNED]`	`MEDIUMINT [UNSIGNED]`	`java.lang.Integer` (regardless of whether it is `UNSIGNED` or not)
`INT,INTEGER[(M)]`	`INTEGER`	`java.lang.Integer`
`INT,INTEGER[(M)] UNSIGNED`	`INTEGER UNSIGNED`	`java.lang.Long`
`BIGINT[(M)]`	`BIGINT`	`java.lang.Long`
`BIGINT[(M)] UNSIGNED`	`BIGINT UNSIGNED`	`java.math.BigInteger`
`FLOAT[(M,D)]`	`FLOAT`	`java.lang.Float`
`DOUBLE[(M,B)] [UNSIGNED]`	`DOUBLE`	`java.lang.Double` (regardless of whether it is `UNSIGNED` or not)
`DECIMAL[(M[,D])] [UNSIGNED]`	`DECIMAL`	`java.math.BigDecimal` (regardless of whether it is `UNSIGNED` or not)
`DATE`	`DATE`	`java.sql.Date`
`DATETIME`	`DATETIME`	`java.time.LocalDateTime`
`TIMESTAMP[(M)]`	`TIMESTAMP`	`java.sql.Timestamp`
`TIME`	`TIME`	`java.sql.Time`
`YEAR[(2|4)]`	`YEAR`	If `yearIsDateType` configuration property is set to `false`, then the returned object type is `java.sql.Short`. If set to `true` (the default), then the returned object is of type `java.sql.Date`.
`CHAR(M)`	`CHAR`	`java.lang.String`
`VARCHAR(M)`	`VARCHAR`	`java.lang.String`
`BINARY(M)`, `CHAR(M) BINARY`	`BINARY`	`byte[]`

MySQL Type Name	Return value of `GetColumnTypeName`	Return value of `GetColumnClassName`
`VARBINARY(M)`, `VARCHAR(M) BINARY`	`VARBINARY`	`byte[]`
`BLOB`	`BLOB`	`byte[]`
`TINYBLOB`	`TINYBLOB`	`byte[]`
`MEDIUMBLOB`	`MEDIUMBLOB`	`byte[]`
`LONGBLOB`	`LONGBLOB`	`byte[]`
`TEXT`	`TEXT`	`java.lang.String`
`TINYTEXT`	`TINYTEXT`	`java.lang.String`
`MEDIUMTEXT`	`MEDIUMTEXT`	`java.lang.String`
`LONGTEXT`	`LONGTEXT`	`java.lang.String`
`JSON`	`JSON`	`java.lang.String`
`GEOMETRY`	`GEOMETRY`	`byte[]`
`ENUM('value1','value2',...)`	`CHAR`	`java.lang.String`
`SET('value1','value2',...)`	`CHAR`	`java.lang.String`

6.6 Handling of Date-Time Values

6.6.1 Preserving Time Instants

Background

A time instant is a specific moment on a time-line. A time instant is said to be preserved when it always refers to the same point in time when its value is being stored to or retrieved from a database, no matter what time zones the database server and the clients are operating in.

`TIMESTAMP` is the only MySQL data type designed to store instants. To preserve time instants, the server applies time zone conversions in incoming or outgoing time values when needed. Incoming values are converted by server from the connection session's time zone to Coordinated Universal Time (UTC) for storage, and outgoing values are converted from UTC to the session time zone. Starting from MySQL 8.0.19, you can also specify a time zone offset when storing `TIMESTAMP` values (see The DATE, DATETIME, and TIMESTAMP Types for details), in which case the `TIMESTAMP` values are converted to the UTC from the specified offset instead of the session time zone. But, once stored, the original offset information is no longer preserved.

The situation is less straightforward with the `DATETIME` data type: it does not represent an instant and, when no time zone offset is specified, there is no time zone conversion for `DATETIME` values, so they are stored and retrieved as they are. However, with a specified time zone offset, the input value is converted to the session time zone before it is stored; the result is that, when retrieved in a different session with a different time zone offset as the specified one, the `DATETIME` value becomes different from the original input value.

Because MySQL data types other than `TIMESTAMP` (and the Java wrapper classes for those other MySQL data types) do not represent true time instants; mixing up instant-representing and non-instant-representing date-time types when storing and retrieving values might give rise to unexpected results. For example:

- When storing `java.sql.Timestamp` to, for example, a `DATETIME` column, you might not get back the same instant value when retrieving it into a client that is in a different time zone than the one the client was in when storing the value.

- When storing, for example, a `java.time.LocalDateTime` to a `TIMESTAMP` column, you might not be storing the correct UTC-based value for it, because the time zone for the value is actually undefined.

Therefore, do not pass instant date-time types (`java.util.Calendar`, `java.util.Date`, `java.time.OffsetDateTime`, `java.sql.Timestamp`) to non-instant date-time types (for example, `java.sql.DATE`, `java.time.LocalDate`, `java.time.LocalTime`, `java.time.OffsetTime`) or vice versa, when working with the server.

The rest of the section discusses how to preserve time instants when working with Connector/J.

Preserving Instants with Connector/J

The scenario: Let us assume that an application is running on a certain application server and is connecting to a MySQL server using Connector/J. Certain events take place in a connection session, for which timestamps are generated, and the event timestamps are associated with the JVM time zone of the application server. These timestamps are to be stored onto a MySQL Server, and are also to be retrieved from it later.

The challenge: The timestamps' instant values need to be preserved when they are saved onto or retrieved from the server using Connector/J. Because the MySQL Server always assumes implicitly that a time instant value references to the connection session time zone (which is set by the session `time_zone` variable) when being saved to or retrieved form the server, a time instant value is properly preserved only in the following situations:

1. When Connector/J is running in the same time zone as the MySQL Server (i.e., the server's session time zone is the same as the JVM's time zone), time instants are naturally preserved, and no time zone conversion is needed. Note that in this case, time instants are really preserved only if the server and the JVM continue to run always in the same time zone in the future.

2. When Connector/J is running in a different time zone from that of the MySQL Server (i.e., the JVM's time zone is different from the server's session time zone), Connector/.J performs one of the following:

   a. Queries the value of the session time zone from the server, and converts the event timestamps between the session time zone and the JVM time zone.

   b. Changes the server's session time zone to that of the JVM time zone, after which no time zone conversion will be required.

   c. Changes the server session time zone to a desired time zone specified by the user, and then converts the timestamps between the JVM time zone and the user-specified time zone.

We identify the above solutions for time instant preservation as Solution 1, 2a, 2b, and 2c. To achieve these solutions, the following connection properties have been introduced in Connector/J since release 8.0.23:

- `preserveInstants={true|false}`: Whether to attempt to preserve time instant values by adjusting timestamps.

  - When it is `false`, no conversions are attempted; a timestamp is sent to the server as-is for storage, and its visual presentation, not the actual time instant is preserved. When it is retrieved from the server by Connector/J, different time zones might be associated with it, as the retrieval might happen in different JVM time zones. For example: For example:

    - Time zones: UTC for JVM, UTC+1 for server session

    - Original timestamp from client (in UTC): `2020-01-01 01:00:00`

    - Timestamp sent to server by Connector/J: `2020-01-01 01:00:00` (no conversion)

    - Timestamp values stored internally on the server: `2020-01-01 00:00:00 UTC` (after internal conversion of `2020-01-01 00:00:00 UTC+1` to UTC)
    - Timestamp value retrieved later into a server section (in UTC+1): `2020-01-01 01:00:00` (after internal conversion of `2020-01-01 00:00:00` from UTC to UTC+1)
    - Timestamp values constructed by Connector/J in some other JVM time zone then before (say, in UTC+3): `2020-01-01 01:00:00`
    - Comment: Time instant is not preserved
  - When it is `true`, Connector/J attempts to preserve the time instants by performing the conversions in a manner defined by the connection properties `connectionTimeZone` and `forceConnectionTimeZoneToSession`.

    When storing a value, the conversion is performed only if the target data type, either the explicit one or the default one, is `TIMESTAMP`. When retrieving a value, the conversion is performed only if the source column has the `TIMESTAMP`, `DATETIME`, or a character data type and the target class is an instant-preserving one, like `java.sql.Timestamp` or `java.time.OffsetDateTime`.

- `connectionTimeZone={LOCAL|SERVER|`*`user-defined-time-zone`*`}`: Specifies how the server's session time zone (in reference to which the timestamps are saved onto the server) is to be determined by Connector/J. It takes on one of the following values:
  - `LOCAL`: Connector/J assumes that the server's session time zone either (a) is the same as the JVM time zone for Connector/J, or (b) should be set as the same as the JVM time zone for Connector/J. Connector/J takes the situation as (a) or (b) depending on the value of the connection property `forceConnectionTimeZoneToSession`.
  - SERVER: Connector/J should query the session's time zone from the server, instead of making any assumptions about it. If the session time zone actually turns out to be different from Connector/J's JVM time zone and `preserveInstants=true`, Connector/J performs time zone conversion between the session time zone and the JVM time zone.
  - *`user-defined-time-zone`*: Connector/J assumes that the server's session time zone either (a) is the same as the user-defined time zone, or (b) should be set as the user-defined time zone. Connector/J takes the situation as (a) or (b) depending on the value of the connection property `forceConnectionTimeZoneToSession`.

    > **Note**
    >
    > For Connector/J 8.0.23 and later, `serverTimezone` is an alias for `connectionTimeZone`. For Connector/J 8.0.22 and earlier, `serverTimezone` was used to override the session time zone setting on the server.

- `forceConnectionTimeZoneToSession={true|false}`: Controls whether the session `time_zone` variable is to be set to the value specified in `connectionTimeZone`.

Now, here are the connection properties values to be used for achieving the Solutions defined above for preserving time instants:

- Solution 1: Use either **preserveInstants=false** or **connectionTimeZone=LOCAL& forceConnectionTimeZoneToSession=false**. Because it can be safely assumed that the server session time zone is the same as Connector/J' s JVM timezone, no query of the server's session time zone occurs, and no time zone conversion occurs. For example:
  - Time zones: UTC+1 for both the JVM and the server session
  - Original timestamp from client (in UTC+1): `2020-01-01 01:00:00`

- Timestamp sent to server by Connector/J: `2020-01-01 01:00:00` (no conversion needed)
- Timestamp values stored internally on the server: `2020-01-01 00:00:00 UTC` (after internal conversion from UTC+1 to UTC)
- Timestamp value retrieved later into a server time session in UTC+1 that Connector/J connects to: `2020-01-01 01:00:00` (after internal conversion from UTC to UTC+1)
- Timestamp value constructed by Connector/J in the same JVM time zone as before (UTC+1) and returned to an application: `2020-01-01 01:00:00`
- Comment: Time instant is preserved without conversion.

  > **Note**
  >
  > This setting corresponds to the default behavior of Connector/J 5.1

- Solution 2a: Use **preserveInstants=true&connectionTimeZone=SERVER** . Connector/J then queries the value of the session time zone from the server, and converts the event timestamps between the session time zone and the JVM time zone. For example:
  - Time zones: UTC+2 for JVM, UTC+1 for server session
  - Original timestamp from client (in UTC+2): `2020-01-01 02:00:00`
  - Timestamp sent to server by Connector/J: `2020-01-01 01:00:00` (after conversion from UTC+2 to UTC+1)
  - Timestamp value stored internally on the server: `2020-01-01 00:00:00 UTC` (after internal conversion from UTC+1 to UTC)
  - Timestamp value retrieved later into a server session in UTC+1: `2020-01-01 01:00:00` (after internal conversion from UTC to UTC+1)
  - Timestamp values constructed by Connector/J in the same JVM time zone as before (UTC+2) and returned to an application: `2020-01-01 02:00:00` (after conversion from UTC+1 to UTC+2)
  - Timestamp values constructed by Connector/J in another JVM time zone (say, UTC+3) and returned to an application: `2020-01-01 03:00:00` (after conversion from UTC+1 to UTC+3)
  - Comment: Time instant is preserved.

    > **Notes**
    >
    > - This setting corresponds to the default behavior of Connector/J 8.0.22 and before and to the behavior of Connector/J 5.1 with `useLegacyDatetimeCode=false`.

- Solution 2b: Use **connectionTimeZone=LOCAL& forceConnectionTimeZoneToSession=true**. Connector/J then changes the server's session time zone to that of the JVM time zone, after which no timezone conversions are required when storing or achieving the timestamps. For example:
  - Time zones: UTC+1 for JVM, UTC+2 for server session originally, but now modified to UTC+1 by Connector/J
  - Original timestamp from client (in UTC+1): `2020-01-01 01:00:00`
  - Timestamp sent to server by Connector/J: `2020-01-01 01:00:00` (no conversion)
  - Timestamp values stored internally on the server: `2020-01-01 00:00:00` (after internal conversion from UTC+1 to UTC)

- Timestamp values retrieved later into a server session (in UTC+1, as set by Connector/J): `2020-01-01 01:00:00` (after internal conversion from UTC to UTC+1)
- Timestamp value constructed by Connector/J in the same JVM time zone as before (UTC+1): `2020-01-01 01:00:00` (no conversion needed)
- Timestamp values retrieved later into a server session (time zone modified to, say, UTC+3, by Connector/J): `2020-01-01 03:00:00` (after internal conversion from UTC to UTC+3)
- Timestamp value constructed by Connector/J in the JVM time zone of UTC+3: `2020-01-01 03:00:00` (no conversion needed)
- Comment: Time instant is preserved without conversion by Connector/J, because the session time zone is changed by Connector/J to its JVM's value.

  Warnings

  - Altering the session time zone affects the results of MySQL functions such as `NOW()`, `CURTIME()`, or `CURDATE()`—if you do not want those functions to be affected, do not use this setting.
  - If you use this setting on different clients in different time zones, the clients are going to modify their connection session's time zones to different values; if you want to keep the same visual date-time value representation for the same time instant for all the clients and in all their sessions, store the values to a `DATETIME` instead of a `TIMESTAMP` column and use non-instant Java classes for them, for example, `java.time.LocalDateTime`.

- Solution 2c: Use **preserveInstants=true&connectionTimeZone=*`user-defined-time-zone`*& forceConnectionTimeZoneToSession=true**. Connector/J then changes the server's session time zone to the user-defined time zone, and converts the timestamps between the user-defined time zone and the JVM time zone. A typical use case for this setting is when the session time zone value on the server is known to be unrecognizable by Connector/J (e.g., `CST` or `CEST`). For example:
  - Time zones: UTC+2 for JVM, `CET` for server session originally, but now modified to user-specified `Europe/Berlin` by Connector/J
  - Original timestamp from client (in UTC+2): `2020-01-01 02:00:00`
  - Timestamp sent to server by Connector/J: `2020-01-01 01:00:00` (after conversion between JVM time zone (UTC+2) and user-defined time zone (`Europe/Berlin`=UTC+1))
  - Timestamp values stored internally on the server: `2020-01-01 00:00:00` (after internal conversion from UTC+1 to UTC)
  - Timestamp value retrieved into a server session (time zone modified to `Europe/Berlin` (=UTC +1) by Connector/J): `2020-01-01 01:00:00` (after internal conversion from UTC to UTC+1)
  - Timestamp value constructed by Connector/J in the same JVM time zone as before (UTC+2) and returned to an application: `2020-01-01 02:00:00` (after conversion between user-defined time zone (UTC+1) and JVM time zone (UTC+2)).
  - Comment: Time instant is preserved with conversion and with the session time zone being changed by Connector/J according to a user-defined value.

  As an alternative to this solution, the user might want the same conversion of the timestamps between the JVM time zone and the user-defined time zone as described above, without actually correcting the unrecognizable time zone value on the server. To do so, use, `preserveInstants=true&connectionTimeZone=user-defined-time-zone&`

`forceConnectionTimeZoneToSession=false`. This achieves the same result of preserving the time instant.

> **Warnings**
>
> See the warnings above for Solution 2b.

6.6.2 Fractional Seconds

While a `java.sql.TIME` instance, according to the JDBC specification, is not supposed to contain fractional seconds by design, because `java.sql.TIME` is a wrapper around `java.util.Date`, it is possible to store fractional seconds in a `java.sql.TIME` instance. However, when Connector/J inserted a `java.sql.TIME` into the server as a MySQL `TIME` value, the fractional seconds were always truncated. To allow the fractional seconds to be sent to the server, a connection property, `sendFractionalSecondsForTime`, has been introduced in release 8.0.23: when the property is `true` (which is the default value), the fractional seconds for `java.sql.TIME` are sent to the server; otherwise, the fractional seconds are truncated.

Also, the connection property `sendFractionalSeconds` has become a global control for the sending of fractional seconds for ALL date-time types since release 8.0.23. As a result, if `sendFractionalSeconds=false`, fractional seconds are not sent irrespective of the value of `sendFractionalSecondsForTime`.

6.6.3 Handling of YEAR Values

How a value in a MySQL YEAR column is handled is controlled by the connection property yearIsDateType:

- If yearIsDateType is `true` (the default), YEAR is mapped to the Java data type `java.sql.Date`.
- If yearIsDateType is `false`, YEAR is mapped to the Java data type `java.sql.Short`.

Connector/J follows the same rules that govern how values are inserted by a `mysql` client; see explanations in The YEAR Type for details.

Connector/J handles the retrieval of zero values from a YEAR column differently than a `mysql` client. Treatments of zero values depend on whether they are strings or numbers, and on the value of yearIsDateType:

- If a string value of `'0'`, `'00'`, or `'000'` is entered into a YEAR column, when retrieved by Connector/J:
  - If yearIsDateType is true, the retrieved value is equivalent to January 1, 2000 00:00:00.000.
  - If yearIsDateType is false, the retrieved value is `2000`
- If a numeric value of `0`, `00`, `000`, or `0000` is entered into a YEAR column, when retrieved by Connector/J,
  - If yearIsDateType is true, the retrieved value is equivalent to January 1, 2000 00:00:00.000.
  - If yearIsDateType is false, the retrieved value is `0`

6.7 Using Character Sets and Unicode

All strings sent from the JDBC driver to the server are converted automatically from native Java Unicode form to the connection's character encoding, including all queries sent using `Statement.execute()`, `Statement.executeUpdate()`, and `Statement.executeQuery()`, as well as all `PreparedStatement` and `CallableStatement` parameters, *excluding* parameters set using the following methods:

- `setBlob()`
- `setBytes()`
- `setClob()`
- `setNClob()`
- `setAsciiStream()`
- `setBinaryStream()`
- `setCharacterStream()`
- `setNCharacterStream()`
- `setUnicodeStream()`

Number of Encodings Per Connection

Connector/J supports a single character encoding between the client and the server, and any number of character encodings for data returned by the server to the client in `ResultSets`.

Setting the Character Encoding

For Connector/J 8.0.25 and earlier: The character encoding between the client and the server is automatically detected upon connection (provided that the Connector/J connection properties `characterEncoding` and `connectionCollation` are not set). The encoding on the server is specified using the system variable `character_set_server` (for more information, see Server Character Set and Collation), and the driver automatically uses the encoding. For example, to use the 4-byte UTF-8 character set with Connector/J, configure the MySQL server with `character_set_server=utf8mb4`, and leave `characterEncoding` and `connectionCollation` out of the Connector/J connection string. Connector/J will then autodetect the UTF-8 setting. To override the automatically detected encoding on the client side, use the `characterEncoding` property in the connection URL to the server.

For Connector/J 8.0.26 and later: There are two phases during the connection initialization in which the character encoding and collation are set.

- *Pre-Authentication Phase:* In this phase, the character encoding between the client and the server is determined by the settings of the Connector/J connection properties, in the following order of priority:
  - `passwordCharacterEncoding`
  - `connectionCollation`
  - `characterEncoding`
  - Set to `UTF8` (corresponds to `utf8mb4` on MySQL servers), if none of the properties above is set
- *Post-Authentication Phase*: In this phase, the character encoding between the client and the server for the rest of the session is determined by the settings of the Connector/J connection properties, in the following order of priority:
  - `connectionCollation`
  - `characterEncoding`
  - Set to `UTF8` (corresponds to `utf8mb4` on MySQL servers), if none of the properties above is set

  This means Connector/J needs to issue a SET NAMES Statement to change the character set and collation that were established in the pre-authentication phase

only if `passwordCharacterEncoding` is set, but its setting is different from that of `connectionCollation`, or different from that of `characterEncoding` (when `connectionCollation` is not set), or different from `utf8mb4` (when both `connectionCollation` and `characterEncoding` are not set).

Custom Character Sets and Collations

To support the use of custom character sets and collations on the server, set the Connector/J connection property `detectCustomCollations` to `true`, and provide the mapping between the custom character sets and the Java character encodings by supplying the `customCharsetMapping` connection property with a comma-delimited list of *`custom_charset:java_encoding`* pairs (for example: `customCharsetMapping=charset1:UTF-8,charset2:Cp1252`).

MySQL to Java Encoding Name Translations

Use Java-style names when specifying character encodings. The following table lists MySQL character set names and their corresponding Java-style names:

Table 6.21 MySQL to Java Encoding Name Translations

MySQL Character Set Name	Java-Style Character Encoding Name
ascii	US-ASCII
big5	Big5
gbk	GBK
sjis	SJIS or Cp932
cp932	Cp932 or MS932
gb2312	EUC_CN
ujis	EUC_JP
euckr	EUC_KR
latin1	Cp1252
latin2	ISO8859_2
greek	ISO8859_7
hebrew	ISO8859_8
cp866	Cp866
tis620	TIS620
cp1250	Cp1250
cp1251	Cp1251
cp1257	Cp1257
macroman	MacRoman
macce	MacCentralEurope
utf8mb4	UTF-8
ucs2	UnicodeBig

Notes

- When `UTF-8` is used for `characterEncoding` in the connection string, it maps to the MySQL character set name `utf8mb4`.
- If the connection option `connectionCollation` is also set alongside `characterEncoding` and is incompatible with it, `characterEncoding` will be overridden with the encoding corresponding to `connectionCollation`.

- Because there is no Java-style character set name for `utfmb3` that you can use with the connection option `charaterEncoding`, the only way to use `utf8mb3` as your connection character set is to use a `utf8mb3` collation (for example, `utf8_general_ci`) for the connection option `connectionCollation`, which forces a `utf8mb3` character set to be used, as explained in the last bullet.

Warning

Do not issue the query SET NAMES with Connector/J, as the driver will not detect that the character set has been changed by the query, and will continue to use the character set configured when the connection was first set up.

6.8 Using Query Attributes

Connector/J supports Query Attributes when it has been enabled on the server by installing the `query_attributes` component (see Prerequisites for Using Query Attributes for details).

Attributes are set for a query by using the `setAttribute()` method of the `JdbcStatement` interface. Here is the method's signature:

```
JdbcStatement.setAttribute(String name, Object value)
```

Here is an example of using the query attributes with a `JdbcStatement`:

Example 6.1 Using Query Attributes with a Plain Statement

```
conn = DriverManager.getConnection("jdbc:mysql://localhost/test", "myuser", "password");

Statement stmt = conn.createStatement();

JdbcStatement jdbcStmt = (JdbcStatement) stmt;

jdbcStmt.executeUpdate("CREATE TABLE t11 (c1 CHAR(20), c2 CHAR(20))");

jdbcStmt.setAttribute("attr1", "cat");
jdbcStmt.setAttribute("attr2", "mat");
jdbcStmt.executeUpdate("INSERT INTO t11 (c1, c2) VALUES(\n" +
    " mysql_query_attribute_string('attr1'),\n" +
    " mysql_query_attribute_string('attr2')\n" +
    " );");

ResultSet rs = stmt.executeQuery("SELECT * from t11");

while(rs.next()) {
    String col1 = rs.getString(1);
    String col2 = rs.getString(2);
    System.out.println("The "+col1+" is on the "+col2);
    }
```

While query attributes are cleared on the server after each query, they are kept on the side of Connector/J, so they can be resent for the next query. To clear the attributes, use the `clearAttributes()` method of the `JdbcStatement` interface:

```
JdbcStatement.clearAttributes()
```

The following example (a continuation of the code in Example 6.1, “Using Query Attributes with a Plain Statement”) shows how the attributes are preserved for a statement until it is cleared :

Example 6.2 Preservation of Query Attributes

```
/* Continuing from the code in the last example, where query attributes have
already been set and used */
```

```
rs = stmt.executeQuery("SELECT c2 FROM t11 where " +
                "c1 = mysql_query_attribute_string('attr1')");

        if (rs.next()) {
            String col1 = rs.getString(1);
            System.out.println("It is on the "+col1);
        }

        // Prints "It is on the mat"

        jdbcStmt.clearAttributes();
        rs = stmt.executeQuery("SELECT c2 FROM t11 where " +
                "c1 = mysql_query_attribute_string('attr1')");

        if (rs.next()) {
            String col1 = rs.getString(1);
            System.out.println("It is on the "+col1);
        }

        else {
            System.out.println("No results!");
        }
        // Prints "No results!" as attribute string attr1 is empty
```

Attributes can also be set for client-side and server-side prepared statements, using the `setAttribute()` method:

Example 6.3 Using Query Attributes with a Prepared Statement

```
conn = DriverManager.getConnection("jdbc:mysql://localhost/test", "myuser", "password");

PreparedStatement ps = conn.prepareStatement(
                "select ?, c2 from t11 where c1 = mysql_query_attribute_string('attr1')");
ps.setString(1, "It is on a ");

JdbcStatement jdbcPs = (JdbcStatement) ps;
jdbcPs.setAttribute("attr1", "cat");
rs = ps.executeQuery();
if (rs.next()) {
            System.out.println(rs.getString(1)+" "+ rs.getString(2));
}
```

Not all MySQL data types are supported by the `setAttribute()` method; only the following MySQL data types are supported and are directly mapped to from specific Java objects or their subclasses:

Table 6.22 Data Type Mappings for Query Attributes

MySQL Data Type	Java Object
`MYSQL_TYPE_STRING`	`java.lang.String`
`MYSQL_TYPE_TINY`	`java.lang.Boolean`, `java.lang.Byte`
`MYSQL_TYPE_SHORT`	`java.lang.Short`
`MYSQL_TYPE_LONG`	`java.lang.Integer`
`MYSQL_TYPE_LONGLONG`	`java.lang.Long`, `java.math.BigInteger`
`MYSQL_TYPE_FLOAT`	`java.lang.Float`
`MYSQL_TYPE_DOUBLE`	`java.lang.Double`, `java.math.BigDecimal`
`MYSQL_TYPE_DATE`	`java.sql.Date`, `java.time.LocalDate`
`MYSQL_TYPE_TIME`	`java.sql.Time`, `java.time.LocalTime`, `java.time.OffsetTime`, `java.time.Duration`
`MYSQL_TYPE_DATETIME`	`java.time.LocalDateTime`
`MYSQL_TYPE_TIMESTAMP`	`java.sql.Timestamp`, `java.time.Instant`, `java.time.OffsetDateTime`,

MySQL Data Type	Java Object
	`java.time.ZonedDateTime`, `java.util.Date`, `java.util.Calendar`

When there is no direct mapping from a Java object type to any MySQL data type, the attribute is set with a string value that comes from converting the supplied object to a `String` using the `.toString()` method.

6.9 Connecting Securely Using SSL

Connector/J can encrypt all data communicated between the JDBC driver and the server (except for the initial handshake) using SSL. There is a performance penalty for enabling connection encryption, the severity of which depends on multiple factors including (but not limited to) the size of the query, the amount of data returned, the server hardware, the SSL library used, the network bandwidth, and so on.

The system works through two Java keystore files: one file contains the certificate information for the server (`truststore` in the examples below), and another contains the keys and certificate for the client (`keystore` in the examples below). All Java keystore files are protected by the password supplied to the `keytool` when you created the files. You need the file names and the associated passwords to create an SSL connection.

For SSL support to work, you must have the following:

- A MySQL server that supports SSL, and compiled and configured to do so. For more information, see Using Encrypted Connections and Configuring SSL Library Support.
- A signed client certificate, if using mutual (two-way) authentication.

By default, Connector/J establishes secure connections with the MySQL servers. Note that MySQL servers 5.7 and up, when compiled with OpenSSL, can automatically generate missing SSL files at startup and configure the SSL connection accordingly.

For 8.0.12 and earlier: As long as the server is correctly configured to use SSL, there is no need to configure anything on the Connector/J client to use encrypted connections (the exception is when Connector/J is connecting to very old server versions like 5.6.25 and earlier or 5.7.5 and earlier, in which case the client must set the connection property `useSSL=true` in order to use encrypted connections). The client can demand SSL to be used by setting the connection property `requireSSL=true`; the connection then fails if the server is not configured to use SSL. Without `requireSSL=true`, the connection just falls back to non-encrypted mode if the server is not configured to use SSL.

For 8.0.13 and later: As long as the server is correctly configured to use SSL, there is no need to configure anything on the Connector/J client to use encrypted connections. The client can demand SSL to be used by setting the connection property `sslMode=REQUIRED`, `VERIFY_CA`, or `VERIFY_IDENTITY`; the connection then fails if the server is not configured to use SSL. With `sslMode=PREFERRED`, the connection just falls back to non-encrypted mode if the server is not configured to use SSL. For X-Protocol connections, the connection property `xdevapi.ssl-mode` specifies the SSL Mode setting, just like `sslMode` does for MySQL-protocol connections (except that `PREFERRED` is not supported by X Protocol); if not explicitly set, `xdevapi.ssl-mode` takes up the value of `sslMode` (if `xdevapi.ssl-mode` is not set and `sslMode` is set to `PREFERRED`, `xdevapi.ssl-mode` is set to `REQUIRED`).

For additional security, you can setup the client for a one-way (server or client) or two-way (server and client) SSL authentication, allowing the client or the server to authenticate each other's identity.

TLS versions: The allowable versions of TLS protocol can be restricted using the connection properties `tlsVersions` and, for X DevAPI connections and for release 8.0.19 and later, `xdevapi.tls-versions` (when `xdevapi.tls-versions` is not specified, it takes up the value of `tlsVersions`). If no such restrictions have been specified, Connector/J attempts to connect to the server with the TLSv1.2 and TLSv1.3.

Notes

- *Since Connector/J 8.0.28,* the connection property `enabledTLSProtocols` has been renamed to `tlsVersions`, and `enabledSSLCipherSuites` has been renamed to `tlsCiphersuites`; the original names remain as aliases.
- *For Connector/J 8.0.26 and later:* TLSv1 and TLSv1.1 were deprecated in Connector/J 8.0.26 and removed in release 8.0.28; the removed values are considered invalid for use with connection options and session settings. Connections can be made using the more-secure TLSv1.2 and TLSv1.3 protocols. Using TLSv1.3 requires that the server be compiled with OpenSSL 1.1.1 or higher and Connector/J be run with a JVM that supports TLSv1.3 (for example, Oracle Java 8u261 and above).
- *For Connector/J 8.0.18 and earlier when connecting to MySQL Community Server 5.6 and 5.7 using the JDBC API:* Due to compatibility issues with MySQL Server compiled with yaSSL, Connector/J does not enable connections with TLSv1.2 and higher by default. When connecting to servers that restrict connections to use those higher TLS versions, enable them explicitly by setting the Connector/J connection property `enabledTLSProtocols` (e.g., set `enabledTLSProtocols=TLSv1.2,TLSv1.3`).

Cipher Suites: Since release 8.0.19, the cipher suites usable by Connector/J are pre-restricted by a properties file that can be found at `src/main/resources/com/mysql/cj/TlsSettings.properties` inside the `src` folder on the source tree or in the platform-independent distribution archive (in `.tar.gz` or `.zip` format) for Connector/J. The file contains four sections, listing in each the mandatory, approved, deprecated, and unacceptable ciphers. Only suites listed in the first three sections can be used. The last section (unacceptable) defines patterns or masks that blocklist unsafe cipher suites. Practically, with the allowlist already given in the first three sections, the blocklist patterns in the forth section are redundant; but they are there as an extra safeguard against unwanted ciphers. The allowlist and blocklist of cipher suites apply to both JDBC and X DevAPI connections.

The allowable cipher suites for SSL connections can be restricted using the connection properties `tlsCiphersuites` and, for X DevAPI connections and for release 8.0.19 and later, `xdevapi.tls-ciphersuites` (when `xdevapi.tls-ciphersuites` is not specified, it takes up the value of `tlsCiphersuites`). If no such restrictions have been specified, Connector/J attempts to establish SSL connections with any allowlisted cipher suites that the server accepts.

6.9.1 Setting up Server Authentication

Server authentication via server certificate verification is enabled when the Connector/J connection property `sslMode` is set to `VERIFY_CA` or `VERIFY_IDENTITY`. If `sslMode` is not set, server authentication via server certificate verification is enabled when the legacy properties `useSSL` AND `verifyServerCertificate` are both true.

Certificates signed by a trusted CA. When server authentication via server certificate verification is enabled, if no additional configurations are made regarding server authentication, Java verifies the server certificate using its default trusted CA certificates, usually from `$JAVA_HOME/lib/security/cacerts`.

Using self-signed certificates. It is pretty common though for MySQL server certificates to be self-signed or signed by a self-signed CA certificate; the auto-generated certificates and keys created by the MySQL server are based on the latter—that is, the server generates all required keys and a self-signed CA certificate that is used to sign a server and a client certificate. The server then configures itself to use the CA certificate and the server certificate. Although the client certificate file is placed in the same directory, it is not used by the server.

To verify the server certificate, Connector/J needs to be able to read the certificate that signed it, that is, the server certificate that signed itself or the self-signed CA certificate. This can be accomplished

by either importing the certificate (`ca.pem` or any other certificate) into the Java default truststore (although tampering the default truststore is not recommended) or by importing it into a custom Java truststore file and configuring the Connector/J driver accordingly. Use Java's keytool (typically located in the `bin` subdirectory of your JDK or JRE installation) to import the server certificates:

```
$> keytool -importcert -alias MySQLCACert -file ca.pem \
    -keystore truststore -storepass mypassword
```

Supply the proper arguments for the command options. If the truststore file does not already exist, a new one will be created; otherwise the certificate will be added to the existing file. Interaction with `keytool` looks like this:

```
Owner: CN=MySQL_Server_5.7.17_Auto_Generated_CA_Certificate
Issuer: CN=MySQL_Server_5.7.17_Auto_Generated_CA_Certificate
Serial number: 1
Valid from: Thu Feb 16 11:42:43 EST 2017 until: Sun Feb 14 11:42:43 EST 2027
Certificate fingerprints:
  MD5:  18:87:97:37:EA:CB:0B:5A:24:AB:27:76:45:A4:78:C1
  SHA1: 2B:0D:D9:69:2C:99:BF:1E:2A:25:4E:8D:2D:38:B8:70:66:47:FA:ED
  SHA256: C3:29:67:1B:E5:37:06:F7:A9:93:DF:C7:B3:27:5E:09:C7:FD:EE:2D:18:86:F4:9C:40:D8:26:CB:DA:95:A0:
Signature algorithm name: SHA256withRSA
Subject Public Key Algorithm: 2048-bit RSA key
Version: 1
Trust this certificate? [no]:  yes
Certificate was added to keystore
```

The output of the command shows all details about the imported certificate. Make sure you remember the password you have supplied. Also, be mindful that the password will have to be written as plain text in your Connector/J configuration file or application source code.

The next step is to configure Java or Connector/J to read the truststore you just created or modified. This can be done by using one of the following three methods:

1. Using the Java command line arguments:

   ```
   -Djavax.net.ssl.trustStore=path_to_truststore_file
   -Djavax.net.ssl.trustStorePassword=mypassword
   ```

2. Setting the system properties directly in the client code:

   ```
   System.setProperty("javax.net.ssl.trustStore","path_to_truststore_file");
   System.setProperty("javax.net.ssl.trustStorePassword","mypassword");
   ```

3. Setting the Connector/J connection properties:

   ```
   trustCertificateKeyStoreUrl=file:path_to_truststore_file
   trustCertificateKeyStorePassword=mypassword
   ```

Notice that when used together, the connection properties override the values set by the other two methods. Also, whatever values set with connection properties are used in that connection only, while values set using the system-wide values are used for all connections (unless overridden by the connection properties). Setting the connection property `fallbackToSystemTrustStore` to `false` prevents Connector/J from falling back to the system-wide truststore setup you created using method (1) or (2) when method (3) is not used.

With the above setup and the server authentication enabled, all connections established are going to be SSL-encrypted, with the server being authenticated in the SSL handshake process, and the client can now safely trust the server it is connecting to.

For X-Protocol connections, the connection properties `xdevapi.ssl-truststore`, `xdevapi.ssl-truststore-type`, `xdevapi.ssl-truststore-password`, and `xdevapi.ssl-fallbackToSystemTrustStore` specify the truststore settings, just like `trustCertificateKeyStoreUrl`, `trustCertificateKeyStoreType`, `trustCertificateKeyStorePassword`amd `fallbackToSystemTrustStore` do for MySQL-protocol connections; if not explicitly set, `xdevapi.ssl-truststore`, `xdevapi.ssl-`

`truststore-type`, `xdevapi.ssl-truststore-password`, and `xdevapi.ssl-fallbackToSystemTrustStore` take up the values of `trustCertificateKeyStoreUrl`, `trustCertificateKeyStoreType`, `trustCertificateKeyStorePassword`, and `fallbackToSystemTrustStore` respectively.

Service Identity Verification. Beyond server authentication via server certificate verification, when `sslMode` is set to `VERIFY_IDENTITY`, Connector/J also performs host name identity verification by checking whether the host name that it uses for connecting matches the Common Name value in the server certificate.

6.9.2 Setting up Client Authentication

The server may want to authenticate a client and require the client to provide an SSL certificate to it, which it verifies against its known certificate authorities or performs additional checks on the client identity if needed (see CREATE USER SSL/TLS Options for details). In that case, Connector/J needs to have access to the client certificate, so it can be sent to the server while establishing new database connections. This is done using the Java keystore files.

To allow client authentication, the client connecting to the server must have its own set of keys and an SSL certificate. The client certificate must be signed so that the server can verify it. While you can have the client certificates signed by official certificate authorities, it is more common to use an intermediate, private, CA certificate to sign client certificates. Such an intermediate CA certificate may be self-signed or signed by a trusted root CA. The requirement is that the server knows a CA certificate that is capable of validating the client certificate.

Some MySQL server builds are able to generate SSL keys and certificates for communication encryption, including a certificate and a private key (contained in the `client-cert.pem` and `client-key.pem` files), which can be used by any client. This SSL certificate is already signed by the self-signed CA certificate `ca.pem`, which the server may have already been configured to use.

If you do not want to use the client keys and certificate files generated by the server, you can also generate new ones using the procedures described in Creating SSL and RSA Certificates and Keys. Notice that, according to the setup of the server, you may have to reuse the already existing CA certificate the server is configured to work with to sign the new client certificate, instead of creating a new one.

Once you have the client private key and certificate files you want to use, you need to import them into a Java keystore so that they can be used by the Java SSL library and Connector/J. The following instructions explain how to create the keystore file:

- Convert the client key and certificate files to a PKCS #12 archive:

```
$> openssl pkcs12 -export -in client-cert.pem -inkey client-key.pem \
  -name "mysqlclient" -passout pass:mypassword -out client-keystore.p12
```

- Import the client key and certificate into a Java keystore:

```
$> keytool -importkeystore -srckeystore client-keystore.p12 -srcstoretype pkcs12 \
 -srcstorepass mypassword -destkeystore keystore -deststoretype JKS -deststorepass mypassword
```

  Supply the proper arguments for the command options. If the keystore file does not already exist, a new one will be created; otherwise the certificate will be added to the existing file. Output by `keytool` looks like this:

```
Entry for alias mysqlclient successfully imported.
Import command completed:  1 entries successfully imported, 0 entries failed or cancelled
```

  Make sure you remember the password you have chosen. Also, be mindful that the password will have to be written as plain text in your Connector/J configuration file or application source code.

After the step, you can delete the PKCS #12 archive (*`client-keystore.p12`* in the example).

The next step is to configure Java or Connector/J so that it reads the keystore you just created or modified. This can be done by using one of the following three methods:

1. Using the Java command line arguments:

```
-Djavax.net.ssl.keyStore=path_to_keystore_file
-Djavax.net.ssl.keyStorePassword=mypassword
```

2. Setting the system properties directly in the client code:

```
System.setProperty("javax.net.ssl.keyStore","path_to_keystore_file");
System.setProperty("javax.net.ssl.keyStorePassword","mypassword");
```

3. Through Connector/J connection properties:

```
clientCertificateKeyStoreUrl=file:path_to_truststore_file
clientCertificateKeyStorePassword=mypassword
```

Notice that when used together, the connection properties override the values set by the other two methods. Also, whatever values set with connection properties are used in that connection only, while values set using the system-wide values are used for all connections (unless overridden by the connection properties). Setting the connection property `fallbackToSystemKeyStore` to `false` prevents Connector/J from falling back to the system-wide keystore setup you created using method (1) or (2) when method (3) is not used.

With the above setups, all connections established are going to be SSL-encrypted with the client being authenticated in the SSL handshake process, and the server can now safely trust the client that is requesting a connection to it.

For X-Protocol connections, the connection properties `xdevapi.ssl-keystore`, `xdevapi.ssl-keystore-type`, `xdevapi.ssl-keystore-password`, and `xdevapi.ssl-fallbackToSystemKeyStore` specify the keystore settings, just like `trustCertificateKeyStoreUrl`, `trustCertificateKeyStoreType`, `trustCertificateKeyStorePassword`, and `fallbackToSystemTKeyStore` do for MySQL-protocol connections; if not explicitly set, `xdevapi.ssl-keystore`, `xdevapi.ssl-keystore-type`, `xdevapi.ssl-keystore-password`, and `xdevapi.ssl-fallbackToSystemKeyStore` take up the values of `clientCertificateKeyStoreUrl`, `clientCertificateKeyStoreType`, `clientCertificateKeyStorePassword`, and `fallbackToSystemKeyStore` respectively.

6.9.3 Setting up 2-Way Authentication

Apply the steps outlined in both Section 6.9.1, “Setting up Server Authentication” and Section 6.9.2, “Setting up Client Authentication” to set up a mutual, two-way authentication process in which the server and the client authenticate each other before establishing a connection.

Although the typical setup described above uses the same CA certificate in both ends for mutual authentication, it does not have to be the case. The only requirements are that the CA certificate configured in the server must be able to validate the client certificate and the CA certificate imported into the client truststore must be able to validate the server certificate; the two CA certificates used on the two ends can be distinct.

6.9.4 JSSE in FIPS Mode

When using a Java 8 to 12 JREs, if JSSE is configured to use FIPS mode, attempts to connect to a MySQL Server may fail in some cases with a `KeyManagementException`, complaining that "FIPS mode: only SunJSSE `TrustManagers` may be used." This happens because, in that case, a custom `TrustManager` implemented by Connector/J that supports the different `sslMode` options is invoked but is eventually rejected by the default implementation of SunJSSE.

The issue can be overcome by telling Connector/J not to use its custom `TrustManager` implementation, but use your own security providers instead. This can be done by setting the following connection properties:

- `fipsCompliantJsse`: Set to `true` to overcome the above-mentioned issue with FIPS mode.

  > **Note**
  >
  > When set to true, Connector/J always performs server certificate validation (even if `sslMode` is set to `PREFERRED` or `REQUIRED`), which means a truststore must be configured with the connection proprieties described below, or the fallback system-wide truststore must be enabled.

- `KeyManagerFactoryProvider`: The name of the a Java Security Provider that provides a `javax.net.ssl.KeyManagerFactory` implementation.
- `trustManagerFactoryProvider`: The name of the a Java Security Provider that provides a `javax.net.ssl.TrustManagerFactory` implementation.
- `keyStoreProvider`: The name of the a Java Security Provider that provides a `java.security.KeyStore` implementation, supporting the key stores types specified with `clientCertificateKeyStoreType` and `trustCertificateKeyStoreType`.

6.9.5 Debugging an SSL Connection

JSSE provides debugging information to `stdout` when you set the system property `-Djavax.net.debug=all`. Java then tells you what keystores and truststores are being used, as well as what is going on during the SSL handshake and certificate exchange. That will be helpful when you are trying to debug a failed SSL connection.

6.10 Connecting Using Unix Domain Sockets

Connector/J does not natively support connections to MySQL Servers with Unix domain sockets. However, there is provision for using 3rd-party libraries that supply the function via a pluggable socket factory. Such a custom factory should implement the `com.mysql.cj.protocol.SocketFactory` interface or the legacy `com.mysql.jdbc.SocketFactory` interface of Connector/J. Follow these requirements when you use such a custom socket factory for Unix sockets :

- The MySQL Server must be configured with the system variable `--socket` (for native protocol connections using the JDBC API) or `--mysqlx-socket` (for X Protocol connections using the X DevAPI), which must contain the file path of the Unix socket file.
- The fully-qualified class name of the custom factory should be passed to Connector/J via the connection property `socketFactory`. For example, with the junixsocket library, set:

  ```
  socketFactory=org.newsclub.net.mysql.AFUNIXDatabaseSocketFactory
  ```

  You might also need to pass other parameters to the custom factory as connection properties. For example, for the junixsocket library, provide the file path of the socket file with the property `junixsocket.file`:

  ```
  junixsocket.file=path_to_socket_file
  ```

6.11 Connecting Using Named Pipes

> **Important**
>
> Minimal permissions on named pipes are granted to clients that use them to connect to the server. Connector/J, however, can only use named pipes when granted full access on them. As a workaround, the MySQL Server that Connector/J wants to connect to must be started with the system variable `named_pipe_full_access_group`, which specifies a Windows local group containing the user by which the client application JVM (and thus Connector/J) is being executed; see the description for `named_pipe_full_access_group` for more details.

> **Note**
>
> Support for named pipes is not available for X Protocol connections.

Connector/J also supports access to MySQL using named pipes on Windows platforms with the `NamedPipeSocketFactory` as a plugin-sockets factory. If you do not use a `namedPipePath` property, the default of `'\\.\pipe\MySQL'` is used. If you use the `NamedPipeSocketFactory`, the host name and port number values in the JDBC URL are ignored. To enable this feature, set the `socketFactory` property:

```
socketFactory=com.mysql.cj.protocol.NamedPipeSocketFactory
```

Set this property, as well as the path of the named pipe, with the following connection URL:

```
jdbc:mysql:///test?socketFactory=com.mysql.cj.protocol.NamedPipeSocketFactory&namedPipePath=\\.\pipe\My
```

To create your own socket factories, follow the sample code in `com.mysql.cj.protocol.NamedPipeSocketFactory` or `com.mysql.cj.protocol.StandardSocketFactory`.

An alternate approach is to use the following two properties in connection URLs for establishing named pipe connections on Windows platforms:

- `(protocol=pipe)` for named pipes (default value for the property is `tcp`).
- `(path=`*`path_to_pipe`*`)` for path of named pipes. Default value for the path is `\\.\pipe\MySQL`.

The “address-equals” or “key-value” form of host specification (see Single host [22] for details) greatly simplifies the URL for a named pipe connection on Windows. For example, to use the default named pipe of “`\\.\pipe\MySQL`,” just specify:

```
jdbc:mysql://address=(protocol=pipe)/test
```

To use the custom named pipe of “`\\.\pipe\MySQL80`” :

```
jdbc:mysql://address=(protocol=pipe)(path=\\.\pipe\MySQL80)/test
```

With `(protocol=pipe)`, the `NamedPipeSocketFactory` is automatically selected.

Named pipes only work when connecting to a MySQL server on the same physical machine where the JDBC driver is running. In simple performance tests, named pipe access is between 30%-50% faster than the standard TCP/IP access. However, this varies per system, and named pipes are slower than TCP/IP in many Windows configurations.

6.12 Connecting Using Various Authentication Methods

6.12.1 Connecting Using PAM Authentication

Java applications using Connector/J can connect to MySQL servers that use the pluggable authentication module (PAM) authentication scheme.

For PAM authentication to work, you must have the following:

- A MySQL server that supports PAM authentication. See PAM Pluggable Authentication for more information. Connector/J implements the same cleartext authentication method as in Client-Side Cleartext Pluggable Authentication.
- SSL capability, as explained in Section 6.9, “Connecting Securely Using SSL”. Because the PAM authentication scheme sends the original password to the server, the connection to the server must be encrypted.

PAM authentication support is enabled by default in Connector/J 8.3, so no extra configuration is needed.

To disable the PAM authentication feature, specify `mysql_clear_password` (the method) or `com.mysql.cj.protocol.a.authentication.MysqlClearPasswordPlugin` (the class name) in the comma-separated list of arguments for the `disabledAuthenticationPlugins` connection option. See Section 6.3, “Configuration Properties” for details about that connection option.

6.12.2 Connecting Using Kerberos

Kerberos is a ticket-based server-client mutual authentication protocol that is supported by the MySQL Server (commercial versions only) .

Support for Kerberos is implemented by Connector/J using the GSS-API, JAAS API, and JCA API; providers for each of these APIs must be available on the Java Virtual Machine running your application that uses Kerberos authentication. Using non-default providers can lead to unexpected results.

Kerberos Authentication Workflow

The main usage of Kerberos authentication in MySQL is to allow users to create connections without having to specify a user name and password in the connection string. For that to work, Connector/J must be configured with the connection property setting `defaultAuthenticationPlugin=authentication_kerberos_client` and then the MySQL user name may be extracted from the Kerberos principal associated to the locally cached Ticket-Granting Ticket (TGT). Notice that a MySQL user name differs from a Kerberos principal in not containing a realm part; therefore, Connector/J cuts all the characters in the principle after the “@” sign and uses it as the MySQL user name.

If there is no TGT available in the local Kerberos cache, Connector/J uses the OS login user name as the MySQL user name. A user name specified in the connection string always takes precedence over names obtained by any other means for the MySQL user.

The MySQL user name is then sent to the MySQL server for validation. Non-existing users cause the server to return an error. Existing users are allowed to proceed with the authentication process, and the authentication mechanism that follows depends on how the MySQL user was created:

- For users created with the authentication plugin `authentication_kerberos`, MySQL server sends the corresponding Kerberos realm back to Connector/J, which, in turn, uses it to construct the Kerberos principal that identifies the user on the Kerberos server. One of three things may then happen:
  - The newly constructed Kerberos principal matches the Kerberos principal associated to the locally cached TGT; this TGT is then sent to the Kerberos server to obtain the desired MySQL Service Ticket, and the authentication proceeds.
  - The newly constructed Kerberos principal does not match the Kerberos principal associated to the locally cached TGT, or there is no local Kerberos cache; this Kerberos principal, as well as the password that may have been specified in the connection string (or an empty string if none was specified), is sent to the Kerberos server to obtain first a valid TGT, and then the desired MySQL Service Ticket; and the authentication proceeds.
  - An error is thrown if Connector/J is unable to obtain the correct Kerberos configurations, unable to communicate with the Kerberos server, or unable to perform either of the two steps above.
- For users defined with a plugin different from `authentication_kerberos`, the server requests Connector/J to use another authentication method.

Client-side Kerberos configurations

In order to operate properly with the Kerberos server, Connector/J requires either a system-wide Kerberos configuration, or these local system property settings for the JVM:

- `-Djava.security.krb5.kdc=[the KDC host name]`
- `-Djava.security.krb5.realm=[the default Kerberos realm]`

Debug Information

The process of configuring Connector/J to use Kerberos authentication is not always straightforward. Enabling logging in the internal Java providers can help find potential problems. That can be done by setting these system properties:

- `-Dsun.security.krb5.debug=true`
- `-Dsun.security.jgss.debug=true`

6.12.3 Connecting Using Multifactor Authentication

Multifactor authentication (MFA) is the use of multiple authentication factors during an authentication process. MySQL Server supports MFA for up to three authentication factors.

Connection to MySQL Server with MFA is supported by Connector/J. When authenticating user accounts that require multiple passwords, up to three passwords can be specified using the Connector/J connection properties `password1`, `password2`, and `password3` . This is a sample connection string that uses the three connection properties for passwords:

```
jdbc:mysql://localhost/db?user=johndoe&password1=password&passsword2=password&password3=password
```

The following apply when using the connection properties for passwords:

- `password1`, `password2`, and `password3` are passwords for authentication factors 1, 2, and 3, respectively, as described in Getting Started with Multifactor Authentication.
- If any of the authentication factors (say, factor *N*) does not require a password, the corresponding password (`password`*N*) is ignored, even if supplied.
- Not specifying the corresponding password for an authentication factor that requires a password is equivalent to supplying an empty password for the factor.
- password and `password1` are taken as synonyms except when both are supplied, in which case `password1` overrides password.

6.12.4 Connecting Using Fast Identity Online (FIDO) Authentication

> **Note**
>
> Support for the `authentication_fido pluggin` has been deprecated since release 8.2.0

Fast Identity Online (FIDO) authentication enables user authentication for MySQL Server using devices such as smart cards, security keys, and biometric readers. FIDO enables passwordless authentication, and can be used for MySQL accounts that use multifactor authentication. See FIDO Pluggable Authentication for details.

Connector/J supports FIDO authentication. To use the feature, a custom implementation of the `com.mysql.cj.callback.MysqlCallbackHandler` interface must be created (see the documentation for `com.mysql.cj.callback.FidoAuthenticationCallback` for details), and the full class name of the implementation must be provided to Connector/J using the connection property authenticationFidoCallbackHandler.

6.12.5 Connecting Using Web Authentication (WebAuthn) Authentication

Web Authentication (WebAuthn) enables user authentication for MySQL Server using devices such as smart cards, security keys, and biometric readers. WebAuthn enables passwordless authentication,

and can be used for MySQL accounts that use multifactor authentication. It is supported by MySQL Enterprise Edition and Connector/J since release 8.2.0—see WebAuthn Pluggable Authentication for details.

> **Note**
>
> The `authentication_fido` plugin, an earlier implementation for supporting FIDO authentication, has now been deprecated. The `authentication_webauthn` plugin is a new implementation that supports some newer FIDO2 features.

The following explains how to use WebAuthn authentication with Connector/J. It assumes there is a MySQL server running and configured to support WebAuthn authentication, with the authentication plugin `authentication_webauthn` loaded and the system variable `authentication_webauthn_rp_id` properly configured. Although not always the case, FIDO authentication often works with multifactor authentication, so additional configuration might be necessary but, typically, a default MySQL installation is multifactor authentication ready.

Create a MySQL User

Create the MySQL user to be linked to the FIDO device. Use the `mysql` client with a root user:

```
mysql > CREATE USER 'johndoe'@'%' IDENTIFIED WITH caching_sha2_password BY 's3cr3t' AND IDENTIFIED WITH aut
Query OK, 0 rows affected (0,02 sec)
```

Register the FIDO device by the user you just created. This is accomplished by running the `mysql` client on the same system the device is installed, which might require installing the `mysql` client in your working machine or moving the FIDO device to the system where the MySQL Server is running. In either case, issue the following command (additional command options to connect to the right server might be needed):

```
$ mysql --user=johndoe --password1 --register-factor=2
Enter password: <type "s3cr3t">
Please insert FIDO device and follow the instruction.  Depending on the device, you may have to perform ges
1. Perform gesture action (Skip this step if you are prompted to enter device PIN).
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 12
Server version: 8.2.0-commercial MySQL Enterprise Server - Commercial

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql >
```

Get 3rd-party Dependencies

MySQL Connector/J is a JDBC Type 4 driver, which is a 100% pure Java implementation, However, there is no pure Java library supporting the authentication devices that Connector/J can use. Therefore, developers need to implement the code that handles the interaction with the authentication devices, for which the following 3rd-party libraries are needed.

- The `libfido2` native library, which must be installed in the system where the application will run.
- Some Java bindings, for example Java Native Interface (JNI) or Java Native Access (JNA). In the following example, Java Native Access (JNA) is used to implement our minimal Java bindings over the `libfido2` library.

Implement the Native Bindings

Create a simple class (called `FidoAssertion` below) that implements the minimal set of bindings between Java and the `libfido2` native library (consult the `libfido2` manuals if needed):

```
import com.sun.jna.Library;
import com.sun.jna.Native;
import com.sun.jna.Pointer;
import com.sun.jna.PointerType;
import com.sun.jna.ptr.IntByReference;
import com.sun.jna.ptr.PointerByReference;

public class FidoAssertion {

    private interface LibFido2 extends Library {
        public static int FIDO_OK = 0;
        static class FidoAssertT extends PointerType {}
        static class FidoDevInfoT extends PointerType {}
        static class FidoDevT extends PointerType {}
        LibFido2 INSTANCE = Native.load("fido2", LibFido2.class);
        int fido_assert_allow_cred(FidoAssertT assrt, byte[] ptr, int len);
        int fido_assert_authdata_len(FidoAssertT assrt, int idx);
        Pointer fido_assert_authdata_ptr(FidoAssertT assrt, int idx);
        void fido_assert_free(PointerByReference assrt);
        FidoAssertT fido_assert_new();
        int fido_assert_count(FidoAssertT assrt);
        int fido_assert_set_clientdata_hash(FidoAssertT assrt, byte[] ptr, int len);
        int fido_assert_set_rp(FidoAssertT assrt, String id);
        int fido_assert_sig_len(FidoAssertT assrt, int idx);
        Pointer fido_assert_sig_ptr(FidoAssertT assrt, int idx);
        int fido_dev_close(FidoDevT dev);
        void fido_dev_free(PointerByReference dev);
        int fido_dev_get_assert(FidoDevT dev, FidoAssertT assrt, String pin);
        void fido_dev_info_free(PointerByReference devlist, int n);
        int fido_dev_info_manifest(FidoDevInfoT devlist, int ilen, IntByReference olen);
        FidoDevInfoT fido_dev_info_new(int n);
        String fido_dev_info_path(FidoDevInfoT di);
        FidoDevInfoT fido_dev_info_ptr(FidoDevInfoT devList, int size);
        FidoDevT fido_dev_new();
        int fido_dev_open(FidoDevT dev, String path);
        boolean fido_dev_supports_credman(FidoDevT dev);
        void fido_init(int flags);
    }

    private LibFido2.FidoAssertT fidoAssert;
    private LibFido2.FidoDevT fidoDev;
    private byte[] clientDataHash;
    private String relyingPartyId;
    private byte[] credentialId;
    private boolean supportsCredMan = false;

    public FidoAssertion() {
        LibFido2.INSTANCE.fido_init(0);
        initializeFidoDevice();
    }

    private void initializeFidoDevice() {
        LibFido2.FidoDevInfoT fidoDevInfo = LibFido2.INSTANCE.fido_dev_info_new(1);
        IntByReference olen = new IntByReference();
        int r = LibFido2.INSTANCE.fido_dev_info_manifest(fidoDevInfo, 1, olen);
        if (r != LibFido2.FIDO_OK) {
            throw new RuntimeException("Failed locating FIDO devices.");
        }
        LibFido2.FidoDevInfoT dev = LibFido2.INSTANCE.fido_dev_info_ptr(fidoDevInfo, 0);
        String path = LibFido2.INSTANCE.fido_dev_info_path(dev);

        LibFido2.INSTANCE.fido_dev_info_free(new PointerByReference(fidoDevInfo.getPointer()), 1);

        this.fidoDev = LibFido2.INSTANCE.fido_dev_new();
        r = LibFido2.INSTANCE.fido_dev_open(this.fidoDev, path);
        if (r != LibFido2.FIDO_OK) {
            throw new RuntimeException("Failed opening the FIDO device.");
```

```
        }

        this.supportsCredMan = LibFido2.INSTANCE.fido_dev_supports_credman(this.fidoDev);
    }

    boolean supportsCredentialManagement() {
        return this.supportsCredMan;
    }

    void setClienDataHash(byte[] clientDataHash) {
        this.clientDataHash = clientDataHash;
    }

    void setRelyingPartyId(String relyingPartyId) {
        this.relyingPartyId = relyingPartyId;
    }

    void setCredentialId(byte[] credentialId) {
        this.credentialId = credentialId;
    }

    void computeAssertions() {
        int r;
        this.fidoAssert = LibFido2.INSTANCE.fido_assert_new();

        // Set the Relying Party Id.
        r = LibFido2.INSTANCE.fido_assert_set_rp(this.fidoAssert, this.relyingPartyId);
        if (r != LibFido2.FIDO_OK) {
            throw new RuntimeException("Failed setting the relying party id.");
        }

        // Set the Client Data Hash.
        r = LibFido2.INSTANCE.fido_assert_set_clientdata_hash(this.fidoAssert, this.clientDataHash, this.cl
        if (r != LibFido2.FIDO_OK) {
            throw new RuntimeException("Failed setting the client data hash.");
        }

        // Set the Credential Id. Not applicable when resident keys are used.
        if (this.credentialId.length > 0) {
            r = LibFido2.INSTANCE.fido_assert_allow_cred(this.fidoAssert, this.credentialId, this.credentia
            if (r != LibFido2.FIDO_OK) {
                throw new RuntimeException("Failed setting the credential id.");
            }
        }

        // Obtain the assertion(s) from the FIDO device.
        r = LibFido2.INSTANCE.fido_dev_get_assert(this.fidoDev, this.fidoAssert, null);
        if (r != LibFido2.FIDO_OK) {
            throw new RuntimeException("Failed obtaining the assertion(s) from the FIDO device.");
        }
    }

    public int getAssertCount() {
        int assertCount = LibFido2.INSTANCE.fido_assert_count(this.fidoAssert);
        return assertCount;
    }

    public byte[] getAuthenticatorData(int idx) {
        int authDataLen = LibFido2.INSTANCE.fido_assert_authdata_len(this.fidoAssert, idx);
        Pointer authData = LibFido2.INSTANCE.fido_assert_authdata_ptr(this.fidoAssert, idx);
        byte[] authenticatorData = authData.getByteArray(0, authDataLen);
        return authenticatorData;
    }

    public byte[] getSignature(int idx) {
        int sigLen = LibFido2.INSTANCE.fido_assert_sig_len(this.fidoAssert, idx);
        Pointer sigData = LibFido2.INSTANCE.fido_assert_sig_ptr(this.fidoAssert, idx);
        byte[] signature = sigData.getByteArray(0, sigLen);
        return signature;
    }

    public void freeResources() {
```

```
        LibFido2.INSTANCE.fido_dev_close(this.fidoDev);
        LibFido2.INSTANCE.fido_dev_free(new PointerByReference(this.fidoDev.getPointer()));
        LibFido2.INSTANCE.fido_assert_free(new PointerByReference(this.fidoAssert.getPointer()));
    }
}
```

Compile the class with a Java 8 compiler (or above).

```
$ javac -classpath *:. FidoAssertion.java
```

Implement the Authentication Callback

MySQL Connector/J uses a pluggable callback class that exchanges data between the authentication process and the interaction with the authentication device. This class must be an instance of the interface `com.mysql.cj.callback.MysqlCallbackHandler`, which defines one single method: `void handle(MysqlCallback cb);`. The `MysqlCallback` argument this method takes is an instance of `com.mysql.cj.callback.WebAuthnAuthenticationCallback` and it contains all the data required by the FIDO assertion code implemented earlier. Likewise, it also takes the output from the FIDO device (authenticator data and signatures) to the running authentication process.

Here is one possible implementation of the `WebAuthnAuthenticationCallback`.

```
import com.mysql.cj.callback.MysqlCallback;
import com.mysql.cj.callback.MysqlCallbackHandler;
import com.mysql.cj.callback.WebAuthnAuthenticationCallback;

public class AuthenticationWebAuthnCallbackHandler implements MysqlCallbackHandler {
    @Override
    public void handle(MysqlCallback cb) {
        if (!WebAuthnAuthenticationCallback.class.isAssignableFrom(cb.getClass())) {
            return;
        }

        WebAuthnAuthenticationCallback webAuthnAuthCallback = (WebAuthnAuthenticationCallback) cb;

        FidoAssertion libFido2Assertion = new FidoAssertion();
        webAuthnAuthCallback.setSupportsCredentialManagement(libFido2Assertion.supportsCredentialManage

        libFido2Assertion.setClienDataHash(webAuthnAuthCallback.getClientDataHash());
        libFido2Assertion.setRelyingPartyId(webAuthnAuthCallback.getRelyingPartyId());
        libFido2Assertion.setCredentialId(webAuthnAuthCallback.getCredentialId());

        System.out.println("Please perform the gesture action on your FIDO device.");
        libFido2Assertion.computeAssertions();

        for (int i = 0; i < libFido2Assertion.getAssertCount(); i++) {
            webAuthnAuthCallback.addAuthenticatorData(libFido2Assertion.getAuthenticatorData(i));
            webAuthnAuthCallback.addSignature(libFido2Assertion.getSignature(i));
        }

        libFido2Assertion.freeResources();
    }
}
```

Notice how this implementation is responsible for asking the user to perform the gesture action. In a real use case, this would eventually trigger an event that would, for example, open a pop-up message to the user.

Compile this code:

```
$ javac -classpath *:.  AuthenticationWebAuthnCallbackHandler.java
```

The name of this class must be supplied to Connector/J through the connection property `authenticationWebAuthnCallbackHandler`.

Implement the Application

Implement the client application. The following implementation is just a proof of concept that creates a MySQL connection to the MySQL server with the user created earlier and checks if the connection was

established successfully. Notice that FIDO authentication requires some sort of human interactions, so this is not a solution to apply for a typical three-tier architecture, where there is usually a single database user configured in the application server and connections to the database are established from a remote machine.

Here is a simple client application code:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.util.Properties;

import com.mysql.cj.conf.PropertyKey;

public class AuthenticationWebAuthnApp {
    private static final String HOST = "localhost";
    private static final String PORT = "3306";
    private static final String USER = "johndoe";
    private static final String PASS = "s3cr3t";

    public static void main(String[] args) throws Exception {
        Properties props = new Properties();
        props.setProperty(PropertyKey.authenticationWebAuthnCallbackHandler.getKeyName(), AuthenticationWeb

        String url = "jdbc:mysql://" + USER + ":" + PASS + "@" + HOST + ":" + PORT + "/";

        try (Connection conn = DriverManager.getConnection(url, props)) {
            ResultSet rs = conn.createStatement().executeQuery("SELECT CURRENT_USER()");
            rs.next();
            System.out.println(rs.getString(1) + " AUTHENTICATED SUCCESSFULLY!");
        }
    }
}
```

Compile the code:

```
$ javac -classpath *:.  AuthenticationWebAuthnApp.java
```

Run the code:

```
$ /usr/lib/jvm/jdk-17/bin/java -classpath *:. AuthenticationWebAuthnApp
Please perform the gesture action on your FIDO device.
johndoe@% AUTHENTICATED SUCCESSFULLY!
```

6.13 Using Source/Replica Replication with ReplicationConnection

See Section 9.4, “Configuring Source/Replica Replication with Connector/J” for details on the topic.

6.14 Support for DNS SRV Records

Connector/J supports the use of DNS SRV records for connections. For information about DNS SRV support in MySQL, see Connecting to the Server Using DNS SRV Records.

When multiple MySQL instances provide the same service for your applications, DNS SRV records can be used to provide failover, load balancing, and replication services. They eliminate the need for clients to identify each possible host in the connection string, or for connections to be handled by an additional software component. Here is a summary for Connector/J's support for DNS SRV records:

- These new schemas in the connection URLs enable DNS SRV record support:
  - `jdbc:mysql+srv:` For ordinary and basic failover JDBC connections that make use of DNS SRV records.
  - `jdbc:mysql+srv:loadbalance:` For load-balancing JDBC connections that make use of DNS SRV records.

  - `jdbc:mysql+srv:replication:` For replication JDBC connections that make use of DNS SRV records.
  - `mysqlx+srv:` For X DevAPI connections that make use of DNS SRV records.
- Besides using the new schemas in the connection URLs, DNS SRV record support can be enabled or disabled using the two new connection properties, `dnsSrv` and `xdevapi.dns-srv`, for JDBC and X DevAPI connections respectively. For example, this connection URL enables DNS SRV record support:

  ```
  mysqlx://johndoe:secret@_mysql._tcp.mycompany.local/db?xdevapi.dns-srv=true
  ```

  However, using the DNS SRV schema with the DNS SRV connection properties set to `false` results in an error; for example:

  ```
  mysqlx+srv://johndoe:secret@_mysql._tcp.mycompany.local/db?xdevapi.dns-srv=false
  # The connection URL causes Connector/J to throw an error
  ```

Here are some requirements and restrictions on the DNS SRV record support by Connector/J:

- Connector/J throws an exception if multiple hosts are specified in the connection URL for a DNS SRV connection (except for a replication set up, created using `jdbc:mysql+srv:replication`, which requires exactly one source and one replica server to be specified).
- Connector/J throws an exception if a port number is specified in the connection URL for a DNS SRV connection.
- DNS SRV records are supported only for TCP/IP connections. Connector/J throws an exception if you attempt to enable DNS SRV record support Windows named pipe connections.

DNS SRV Record Support for Load Balancing and Failover. For load-balancing and failover connections, Connector/J uses the `priority` field of the DNS SRV records to decide on the priorities for connection attempts for hosts.

DNS SRV Record Support for Connection Pooling. In an X DevAPI connection pooling setup, Connector/J re-queries the DNS SRV records regularly and phases out gracefully any connections whose hosts no longer appear in the records, and readmits the connections into the pool when their hosts reappear in the records.

Looking up DNS SRV Records. It is the users' responsibility to provide a full service host name; Connector/J does not append any prefix nor validate the host name structure. The following are examples of valid service host name patterns:

- `foo.domain.local`
- `_mysql._tcp.foo.domain.local`
- `_mysqlx._tcp.foo.domain.local`
- `_readonly._tcp.foo.domain.local`
- `_readwrite._tcp.foo.domain.local`

See *Connections Using DNS SRV Records* in the X DevAPI User Guide for details.

6.15 Client Session State Tracker

Connector/J can receive information on client session state changes tracked by the server if the tracking has been enabled on the server. The reception of the information is enabled by setting the Connector/J connection property `trackSessionState` to `true` (default value is `false` for the property).

When the function is enabled, information on session state changes received from the server are stored inside the `SessionStateChanges` object, accessible through a `ServerSessionStateController` and its `getSessionStateChanges()` method:

```
ServerSessionStateChanges ssc =
  MysqlConnection.getServerSessionStateController().getSessionStateChanges();
```

In `SessionStateChanges` is a list of `SessoinStateChange` objects, accessible by the `getSessionStateChangesList()` method:

```
List<SessionStateChange> sscList = ssc.getSessionStateChangesList();
```

Each `SessionStateChange` has the fields `type` and `values`, accessible by the `getType()` and `getValues()` methods. The types and their corresponding values are described below:

Table 6.23 SessionStateChange Type and Values

Type	Number of Values in the value List	Values
`SESSION_TRACK_SYSTEM_VARIABLES`	2	The name of the changed system variable and its new value
`SESSION_TRACK_SCHEMA`	1	The new schema name
`SESSION_TRACK_STATE_CHANGE`	1	"1" or "0"
`SESSION_TRACK_GTIDS`	1	List of GTIDs as reported by server
`SESSION_TRACK_TRANSACTION_CHARACTERISTICS`	1	Transaction characteristics statement
`SESSION_TRACK_TRANSACTION_STATE`	1	Transaction state record

Connector/J receives changes only from the most recent OK packet sent by the server. With `getSessionStateChanges()`, some changes returned by the intermediate queries issued by Connector/J could be missed. However, the session state change information can also be received using a `SessionStateChangesListener`, which has to be registered with a `ServerSessionStateController` using the `addSessionStateChangesListener()` method. The following example implements `SessionStateChangesListener` in a class, which also provides a method to print the change information:

```
class SSCListener implements SessionStateChangesListener {
        ServerSessionStateChanges changes = null;

        public void handleSessionStateChanges(ServerSessionStateChanges ch) {
            this.changes = ch;
            for (SessionStateChange change : ch.getSessionStateChangesList()) {
                printChange(change);
            }
        }

        private void printChange(SessionStateChange change) {
            System.out.print(change.getType() + " == > ");
            int pos = 0;
            if (change.getType() == ServerSessionStateController.SESSION_TRACK_SYSTEM_VARIABLES) {
                // There are two values with this change type, the system variable name and its new value
                System.out.print(change.getValues().get(pos++) + "=");
            }
            System.out.println(change.getValues().get(pos));
        }
    }
SessionStateChangesListener listener = new SSCListener();
MysqlConnection.getServerSessionStateController().addSessionStateChangesListener(listener);
```

With a registered `SessionStateChangesListener`, users have access to all intermediate results, though the listener might slow down the delivery of query results. That is because the listener is

invoked immediately after the OK packet is consumed by Connector/J, before the `ResultSet` is constructed.

6.16 Mapping MySQL Error Numbers to JDBC SQLState Codes

The table below provides a mapping of the MySQL error numbers to JDBC `SQLState` values.

Table 6.24 Mapping of MySQL Error Numbers to SQLStates

MySQL Error Number	MySQL Error Name	SQL Standard SQLState
1022	ER_DUP_KEY	23000
1037	ER_OUTOFMEMORY	HY001
1038	ER_OUT_OF_SORTMEMORY	HY001
1040	ER_CON_COUNT_ERROR	08004
1042	ER_BAD_HOST_ERROR	08S01
1043	ER_HANDSHAKE_ERROR	08S01
1044	ER_DBACCESS_DENIED_ERROR	42000
1045	ER_ACCESS_DENIED_ERROR	28000
1046	ER_NO_DB_ERROR	3D000
1047	ER_UNKNOWN_COM_ERROR	08S01
1048	ER_BAD_NULL_ERROR	23000
1049	ER_BAD_DB_ERROR	42000
1050	ER_TABLE_EXISTS_ERROR	42S01
1051	ER_BAD_TABLE_ERROR	42S02
1052	ER_NON_UNIQ_ERROR	23000
1053	ER_SERVER_SHUTDOWN	08S01
1054	ER_BAD_FIELD_ERROR	42S22
1055	ER_WRONG_FIELD_WITH_GROUP	42000
1056	ER_WRONG_GROUP_FIELD	42000
1057	ER_WRONG_SUM_SELECT	42000
1058	ER_WRONG_VALUE_COUNT	21S01
1059	ER_TOO_LONG_IDENT	42000
1060	ER_DUP_FIELDNAME	42S21
1061	ER_DUP_KEYNAME	42000
1062	ER_DUP_ENTRY	23000
1063	ER_WRONG_FIELD_SPEC	42000
1064	ER_PARSE_ERROR	42000
1065	ER_EMPTY_QUERY	42000
1066	ER_NONUNIQ_TABLE	42000
1067	ER_INVALID_DEFAULT	42000
1068	ER_MULTIPLE_PRI_KEY	42000
1069	ER_TOO_MANY_KEYS	42000
1070	ER_TOO_MANY_KEY_PARTS	42000
1071	ER_TOO_LONG_KEY	42000

MySQL Error Number	MySQL Error Name	SQL Standard SQLState
1072	ER_KEY_COLUMN_DOES_NOT_EXITS	42000
1073	ER_BLOB_USED_AS_KEY	42000
1074	ER_TOO_BIG_FIELDLENGTH	42000
1075	ER_WRONG_AUTO_KEY	42000
1080	ER_FORCING_CLOSE	08S01
1081	ER_IPSOCK_ERROR	08S01
1082	ER_NO_SUCH_INDEX	42S12
1083	ER_WRONG_FIELD_TERMINATORS	42000
1084	ER_BLOBS_AND_NO_TERMINATED	42000
1090	ER_CANT_REMOVE_ALL_FIELDS	42000
1091	ER_CANT_DROP_FIELD_OR_KEY	42000
1101	ER_BLOB_CANT_HAVE_DEFAULT	42000
1102	ER_WRONG_DB_NAME	42000
1103	ER_WRONG_TABLE_NAME	42000
1104	ER_TOO_BIG_SELECT	42000
1106	ER_UNKNOWN_PROCEDURE	42000
1107	ER_WRONG_PARAMCOUNT_TO_PROCEDURE	42000
1109	ER_UNKNOWN_TABLE	42S02
1110	ER_FIELD_SPECIFIED_TWICE	42000
1112	ER_UNSUPPORTED_EXTENSION	42000
1113	ER_TABLE_MUST_HAVE_COLUMNS	42000
1115	ER_UNKNOWN_CHARACTER_SET	42000
1118	ER_TOO_BIG_ROWSIZE	42000
1120	ER_WRONG_OUTER_JOIN	42000
1121	ER_NULL_COLUMN_IN_INDEX	42000
1131	ER_PASSWORD_ANONYMOUS_USER	42000
1132	ER_PASSWORD_NOT_ALLOWED	42000
1133	ER_PASSWORD_NO_MATCH	42000
1136	ER_WRONG_VALUE_COUNT_ON_ROW	21S01
1138	ER_INVALID_USE_OF_NULL	22004
1139	ER_REGEXP_ERROR	42000
1140	ER_MIX_OF_GROUP_FUNC_AND_FIELDS	42000
1141	ER_NONEXISTING_GRANT	42000
1142	ER_TABLEACCESS_DENIED_ERROR	42000
1143	ER_COLUMNACCESS_DENIED_ERROR	42000
1144	ER_ILLEGAL_GRANT_FOR_TABLE	42000
1145	ER_GRANT_WRONG_HOST_OR_USER	42000
1146	ER_NO_SUCH_TABLE	42S02
1147	ER_NONEXISTING_TABLE_GRANT	42000

MySQL Error Number	MySQL Error Name	SQL Standard SQLState
1148	ER_NOT_ALLOWED_COMMAND	42000
1149	ER_SYNTAX_ERROR	42000
1152	ER_ABORTING_CONNECTION	08S01
1153	ER_NET_PACKET_TOO_LARGE	08S01
1154	ER_NET_READ_ERROR_FROM_PIPE	08S01
1155	ER_NET_FCNTL_ERROR	08S01
1156	ER_NET_PACKETS_OUT_OF_ORDER	08S01
1157	ER_NET_UNCOMPRESS_ERROR	08S01
1158	ER_NET_READ_ERROR	08S01
1159	ER_NET_READ_INTERRUPTED	08S01
1160	ER_NET_ERROR_ON_WRITE	08S01
1161	ER_NET_WRITE_INTERRUPTED	08S01
1162	ER_TOO_LONG_STRING	42000
1163	ER_TABLE_CANT_HANDLE_BLOB	42000
1164	ER_TABLE_CANT_HANDLE_AUTO_INCREMENT	42000
1166	ER_WRONG_COLUMN_NAME	42000
1167	ER_WRONG_KEY_COLUMN	42000
1169	ER_DUP_UNIQUE	23000
1170	ER_BLOB_KEY_WITHOUT_LENGTH	42000
1171	ER_PRIMARY_CANT_HAVE_NULL	42000
1172	ER_TOO_MANY_ROWS	42000
1173	ER_REQUIRES_PRIMARY_KEY	42000
1176	ER_KEY_DOES_NOT_EXITS	42000
1177	ER_CHECK_NO_SUCH_TABLE	42000
1178	ER_CHECK_NOT_IMPLEMENTED	42000
1179	ER_CANT_DO_THIS_DURING_AN_TRANSACTION	25000
1184	ER_NEW_ABORTING_CONNECTION	08S01
1189	ER_SOURCE_NET_READ	08S01
1190	ER_SOURCE_NET_WRITE	08S01
1203	ER_TOO_MANY_USER_CONNECTIONS	42000
1205	ER_LOCK_WAIT_TIMEOUT	40001
1207	ER_READ_ONLY_TRANSACTION	25000
1211	ER_NO_PERMISSION_TO_CREATE_USER	42000
1213	ER_LOCK_DEADLOCK	40001
1216	ER_NO_REFERENCED_ROW	23000
1217	ER_ROW_IS_REFERENCED	23000
1218	ER_CONNECT_TO_SOURCE	08S01
1222	ER_WRONG_NUMBER_OF_COLUMNS_IN_SELECT	21000
1226	ER_USER_LIMIT_REACHED	42000

MySQL Error Number	MySQL Error Name	SQL Standard SQLState
1227	ER_SPECIFIC_ACCESS_DENIED_ERROR	42000
1230	ER_NO_DEFAULT	42000
1231	ER_WRONG_VALUE_FOR_VAR	42000
1232	ER_WRONG_TYPE_FOR_VAR	42000
1234	ER_CANT_USE_OPTION_HERE	42000
1235	ER_NOT_SUPPORTED_YET	42000
1239	ER_WRONG_FK_DEF	42000
1241	ER_OPERAND_COLUMNS	21000
1242	ER_SUBQUERY_NO_1_ROW	21000
1247	ER_ILLEGAL_REFERENCE	42S22
1248	ER_DERIVED_MUST_HAVE_ALIAS	42000
1249	ER_SELECT_REDUCED	01000
1250	ER_TABLENAME_NOT_ALLOWED_HERE	42000
1251	ER_NOT_SUPPORTED_AUTH_MODE	08004
1252	ER_SPATIAL_CANT_HAVE_NULL	42000
1253	ER_COLLATION_CHARSET_MISMATCH	42000
1261	ER_WARN_TOO_FEW_RECORDS	01000
1262	ER_WARN_TOO_MANY_RECORDS	01000
1263	ER_WARN_NULL_TO_NOTNULL	22004
1264	ER_WARN_DATA_OUT_OF_RANGE	22003
1265	ER_WARN_DATA_TRUNCATED	01000
1280	ER_WRONG_NAME_FOR_INDEX	42000
1281	ER_WRONG_NAME_FOR_CATALOG	42000
1286	ER_UNKNOWN_STORAGE_ENGINE	42000
1292	ER_TRUNCATED_WRONG_VALUE	22007
1303	ER_SP_NO_RECURSIVE_CREATE	2F003
1304	ER_SP_ALREADY_EXISTS	42000
1305	ER_SP_DOES_NOT_EXIST	42000
1308	ER_SP_LILABEL_MISMATCH	42000
1309	ER_SP_LABEL_REDEFINE	42000
1310	ER_SP_LABEL_MISMATCH	42000
1311	ER_SP_UNINIT_VAR	01000
1312	ER_SP_BADSELECT	0A000
1313	ER_SP_BADRETURN	42000
1314	ER_SP_BADSTATEMENT	0A000
1315	ER_UPDATE_LOG_DEPRECATED_IGNORED	42000
1316	ER_UPDATE_LOG_DEPRECATED_TRANSLATED	42000
1317	ER_QUERY_INTERRUPTED	70100
1318	ER_SP_WRONG_NO_OF_ARGS	42000

MySQL Error Number	MySQL Error Name	SQL Standard SQLState
1319	ER_SP_COND_MISMATCH	42000
1320	ER_SP_NORETURN	42000
1321	ER_SP_NORETURNEND	2F005
1322	ER_SP_BAD_CURSOR_QUERY	42000
1323	ER_SP_BAD_CURSOR_SELECT	42000
1324	ER_SP_CURSOR_MISMATCH	42000
1325	ER_SP_CURSOR_ALREADY_OPEN	24000
1326	ER_SP_CURSOR_NOT_OPEN	24000
1327	ER_SP_UNDECLARED_VAR	42000
1329	ER_SP_FETCH_NO_DATA	02000
1330	ER_SP_DUP_PARAM	42000
1331	ER_SP_DUP_VAR	42000
1332	ER_SP_DUP_COND	42000
1333	ER_SP_DUP_CURS	42000
1335	ER_SP_SUBSELECT_NYI	0A000
1336	ER_STMT_NOT_ALLOWED_IN_SF_OR_TRG	0A000
1337	ER_SP_VARCOND_AFTER_CURSHNDLR	42000
1338	ER_SP_CURSOR_AFTER_HANDLER	42000
1339	ER_SP_CASE_NOT_FOUND	20000
1365	ER_DIVISION_BY_ZERO	22012
1367	ER_ILLEGAL_VALUE_FOR_TYPE	22007
1370	ER_PROCACCESS_DENIED_ERROR	42000
1397	ER_XAER_NOTA	XAE04
1398	ER_XAER_INVAL	XAE05
1399	ER_XAER_RMFAIL	XAE07
1400	ER_XAER_OUTSIDE	XAE09
1401	ER_XA_RMERR	XAE03
1402	ER_XA_RBROLLBACK	XA100
1403	ER_NONEXISTING_PROC_GRANT	42000
1406	ER_DATA_TOO_LONG	22001
1407	ER_SP_BAD_SQLSTATE	42000
1410	ER_CANT_CREATE_USER_WITH_GRANT	42000
1413	ER_SP_DUP_HANDLER	42000
1414	ER_SP_NOT_VAR_ARG	42000
1415	ER_SP_NO_RETSET	0A000
1416	ER_CANT_CREATE_GEOMETRY_OBJECT	22003
1425	ER_TOO_BIG_SCALE	42000
1426	ER_TOO_BIG_PRECISION	42000
1427	ER_M_BIGGER_THAN_D	42000

MySQL Error Number	MySQL Error Name	SQL Standard SQLState
1437	ER_TOO_LONG_BODY	42000
1439	ER_TOO_BIG_DISPLAYWIDTH	42000
1440	ER_XAER_DUPID	XAE08
1441	ER_DATETIME_FUNCTION_OVERFLOW	22008
1451	ER_ROW_IS_REFERENCED_2	23000
1452	ER_NO_REFERENCED_ROW_2	23000
1453	ER_SP_BAD_VAR_SHADOW	42000
1458	ER_SP_WRONG_NAME	42000
1460	ER_SP_NO_AGGREGATE	42000
1461	ER_MAX_PREPARED_STMT_COUNT_REACHED	42000
1463	ER_NON_GROUPING_FIELD_USED	42000
1557	ER_FOREIGN_DUPLICATE_KEY	23000
1568	ER_CANT_CHANGE_TX_ISOLATION	25001
1582	ER_WRONG_PARAMCOUNT_TO_NATIVE_FCT	42000
1583	ER_WRONG_PARAMETERS_TO_NATIVE_FCT	42000
1584	ER_WRONG_PARAMETERS_TO_STORED_FCT	42000
1586	ER_DUP_ENTRY_WITH_KEY_NAME	23000
1613	ER_XA_RBTIMEOUT	XA106
1614	ER_XA_RBDEADLOCK	XA102
1630	ER_FUNC_INEXISTENT_NAME_COLLISION	42000
1641	ER_DUP_SIGNAL_SET	42000
1642	ER_SIGNAL_WARN	01000
1643	ER_SIGNAL_NOT_FOUND	02000
1645	ER_RESIGNAL_WITHOUT_ACTIVE_HANDLER	0K000
1687	ER_SPATIAL_MUST_HAVE_GEOM_COL	42000
1690	ER_DATA_OUT_OF_RANGE	22003
1698	ER_ACCESS_DENIED_NO_PASSWORD_ERROR	28000
1701	ER_TRUNCATE_ILLEGAL_FK	42000
1758	ER_DA_INVALID_CONDITION_NUMBER	35000
1761	ER_FOREIGN_DUPLICATE_KEY_WITH_CHILD_INFO	23000
1762	ER_FOREIGN_DUPLICATE_KEY_WITHOUT_CHILD_INFO	23000
1792	ER_CANT_EXECUTE_IN_READ_ONLY_TRANSACTION	25006
1845	ER_ALTER_OPERATION_NOT_SUPPORTED	0A000
1846	ER_ALTER_OPERATION_NOT_SUPPORTED_REASON	0A000
1859	ER_DUP_UNKNOWN_IN_INDEX	23000
1873	ER_ACCESS_DENIED_CHANGE_USER_ERROR	28000
1887	ER_GET_STACKED_DA_WITHOUT_ACTIVE_HANDLER	0Z002
1903	ER_INVALID_ARGUMENT_FOR_LOGARITHM	2201E

Chapter 7 JDBC Concepts

Table of Contents

7.1 Connecting to MySQL Using the JDBC `DriverManager` Interface 107
7.2 Using JDBC `Statement` Objects to Execute SQL 108
7.3 Using JDBC `CallableStatements` to Execute Stored Procedures 109
7.4 Retrieving `AUTO_INCREMENT` Column Values through JDBC 112

This section provides some general JDBC background.

7.1 Connecting to MySQL Using the JDBC `DriverManager` Interface

When you are using JDBC outside of an application server, the `DriverManager` class manages the establishment of connections.

Specify to the `DriverManager` which JDBC drivers to try to make Connections with. The easiest way to do this is to use `Class.forName()` on the class that implements the `java.sql.Driver` interface. With MySQL Connector/J, the name of this class is `com.mysql.cj.jdbc.Driver`. With this method, you could use an external configuration file to supply the driver class name and driver parameters to use when connecting to a database.

The following section of Java code shows how you might register MySQL Connector/J from the `main()` method of your application. If testing this code, first read the installation section at Chapter 4, *Connector/J Installation*, to make sure you have connector installed correctly and the `CLASSPATH` set up. Also, ensure that MySQL is configured to accept external TCP/IP connections.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

// Notice, do not import com.mysql.cj.jdbc.*
// or you will have problems!

public class LoadDriver {
    public static void main(String[] args) {
        try {
            // The newInstance() call is a work around for some
            // broken Java implementations

            Class.forName("com.mysql.cj.jdbc.Driver").newInstance();
        } catch (Exception ex) {
            // handle the error
        }
    }
}
```

After the driver has been registered with the `DriverManager`, you can obtain a `Connection` instance that is connected to a particular database by calling `DriverManager.getConnection()`:

Example 7.1 Connector/J: Obtaining a connection from the `DriverManager`

If you have not already done so, please review the portion of Section 7.1, “Connecting to MySQL Using the JDBC `DriverManager` Interface” above before working with the example below.

This example shows how you can obtain a `Connection` instance from the `DriverManager`. There are a few different signatures for the `getConnection()` method. Consult the API documentation that comes with your JDK for more specific information on how to use them.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
```

```
Connection conn = null;
...
try {
    conn =
       DriverManager.getConnection("jdbc:mysql://localhost/test?" +
                                   "user=minty&password=greatsqldb");

    // Do something with the Connection

   ...
} catch (SQLException ex) {
    // handle any errors
    System.out.println("SQLException: " + ex.getMessage());
    System.out.println("SQLState: " + ex.getSQLState());
    System.out.println("VendorError: " + ex.getErrorCode());
}
```

Once a Connection is established, it can be used to create Statement and PreparedStatement objects, as well as retrieve metadata about the database. This is explained in the following sections.

When the user for the connection is unspecified, Connector/J's implementations of the authentication plugins use by default the name of the OS user who runs the application for authentication with the MySQL server (except when the Kerberos authentication plugin is being used; see Section 6.12.2, "Connecting Using Kerberos" for details).

> **Note**
>
> A user name is considered unspecified only when the following conditions are all met:
>
> 1. The method DriverManager.getConnection(String url, String user, String password) is not used.
> 2. The connection property user is not used in, for example, the connection URL,or elsewhere.
> 3. The user is not mentioned in the authority of the connection URL, as in jdbc:mysql://localhost:3306/test, or jdbc:mysql://@localhost:3306/test.
>
> Notice if (1) or (2) is not true and an empty string is passed, the user name is an empty string then, and is not considered unspecified.

7.2 Using JDBC Statement Objects to Execute SQL

Statement objects allow you to execute basic SQL queries and retrieve the results through the ResultSet class, which is described later.

To create a Statement instance, you call the createStatement() method on the Connection object you have retrieved using one of the DriverManager.getConnection() or DataSource.getConnection() methods described earlier.

Once you have a Statement instance, you can execute a SELECT query by calling the executeQuery(String) method with the SQL you want to use.

To update data in the database, use the executeUpdate(String SQL) method. This method returns the number of rows matched by the update statement, not the number of rows that were modified.

If you do not know ahead of time whether the SQL statement will be a SELECT or an UPDATE/INSERT, then you can use the execute(String SQL) method. This method will return true if the SQL query was a SELECT, or false if it was an UPDATE, INSERT, or DELETE statement. If the statement was a SELECT query, you can retrieve the results by calling the getResultSet() method. If the statement

was an UPDATE, INSERT, or DELETE statement, you can retrieve the affected rows count by calling getUpdateCount() on the Statement instance.

Example 7.2 Connector/J: Using java.sql.Statement to execute a SELECT query

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;
import java.sql.ResultSet;

// assume that conn is an already created JDBC connection (see previous examples)

Statement stmt = null;
ResultSet rs = null;

try {
    stmt = conn.createStatement();
    rs = stmt.executeQuery("SELECT foo FROM bar");

    // or alternatively, if you don't know ahead of time that
    // the query will be a SELECT...

    if (stmt.execute("SELECT foo FROM bar")) {
        rs = stmt.getResultSet();
    }

    // Now do something with the ResultSet ....
}
catch (SQLException ex){
    // handle any errors
    System.out.println("SQLException: " + ex.getMessage());
    System.out.println("SQLState: " + ex.getSQLState());
    System.out.println("VendorError: " + ex.getErrorCode());
}
finally {
    // it is a good idea to release
    // resources in a finally{} block
    // in reverse-order of their creation
    // if they are no-longer needed

    if (rs != null) {
        try {
            rs.close();
        } catch (SQLException sqlEx) { } // ignore

        rs = null;
    }

    if (stmt != null) {
        try {
            stmt.close();
        } catch (SQLException sqlEx) { } // ignore

        stmt = null;
    }
}
```

7.3 Using JDBC CallableStatements to Execute Stored Procedures

Connector/J fully implements the java.sql.CallableStatement interface.

For more information on MySQL stored procedures, please refer to Using Stored Routines.

Connector/J exposes stored procedure functionality through JDBC's CallableStatement interface.

The following example shows a stored procedure that returns the value of inOutParam incremented by 1, and the string passed in using inputParam as a ResultSet:

Example 7.3 Connector/J: Calling Stored Procedures

```
CREATE PROCEDURE demoSp(IN inputParam VARCHAR(255), \
                        INOUT inOutParam INT)
BEGIN
    DECLARE z INT;
    SET z = inOutParam + 1;
    SET inOutParam = z;

    SELECT inputParam;

    SELECT CONCAT('zyxw', inputParam);
END
```

To use the demoSp procedure with Connector/J, follow these steps:

1. Prepare the callable statement by using Connection.prepareCall().

   Notice that you have to use JDBC escape syntax, and that the parentheses surrounding the parameter placeholders are not optional:

   Example 7.4 Connector/J: Using Connection.prepareCall()

   ```
   import java.sql.CallableStatement;

   ...

       //
       // Prepare a call to the stored procedure 'demoSp'
       // with two parameters
       //
       // Notice the use of JDBC-escape syntax ({call ...})
       //

       CallableStatement cStmt = conn.prepareCall("{call demoSp(?, ?)}");



       cStmt.setString(1, "abcdefg");
   ```

   > **Note**
   >
   > Connection.prepareCall() is an expensive method, due to the metadata retrieval that the driver performs to support output parameters. For performance reasons, minimize unnecessary calls to Connection.prepareCall() by reusing CallableStatement instances in your code.

2. Register the output parameters (if any exist)

   To retrieve the values of output parameters (parameters specified as OUT or INOUT when you created the stored procedure), JDBC requires that they be specified before statement execution using the various registerOutputParameter() methods in the CallableStatement interface:

   Example 7.5 Connector/J: Registering output parameters

   ```
   import java.sql.Types;
   ...
   //
   // Connector/J supports both named and indexed
   // output parameters. You can register output
   // parameters using either method, as well
   // as retrieve output parameters using either
   // method, regardless of what method was
   // used to register them.
   //
   // The following examples show how to use
   ```

```
// the various methods of registering
// output parameters (you should of course
// use only one registration per parameter).
//

//
// Registers the second parameter as output, and
// uses the type 'INTEGER' for values returned from
// getObject()
//

cStmt.registerOutParameter(2, Types.INTEGER);

//
// Registers the named parameter 'inOutParam', and
// uses the type 'INTEGER' for values returned from
// getObject()
//

cStmt.registerOutParameter("inOutParam", Types.INTEGER);
...
```

3. Set the input parameters (if any exist)

   Input and in/out parameters are set as for PreparedStatement objects. However, CallableStatement also supports setting parameters by name:

   Example 7.6 Connector/J: Setting CallableStatement input parameters

```
...

    //
    // Set a parameter by index
    //

    cStmt.setString(1, "abcdefg");

    //
    // Alternatively, set a parameter using
    // the parameter name
    //

    cStmt.setString("inputParam", "abcdefg");

    //
    // Set the 'in/out' parameter using an index
    //

    cStmt.setInt(2, 1);

    //
    // Alternatively, set the 'in/out' parameter
    // by name
    //

    cStmt.setInt("inOutParam", 1);

...
```

4. Execute the CallableStatement, and retrieve any result sets or output parameters.

   Although CallableStatement supports calling any of the Statement execute methods (executeUpdate(), executeQuery() or execute()), the most flexible method to call is execute(), as you do not need to know ahead of time if the stored procedure returns result sets:

   Example 7.7 Connector/J: Retrieving results and output parameter values

```
...

    boolean hadResults = cStmt.execute();
```

```
    //
    // Process all returned result sets
    //

    while (hadResults) {
        ResultSet rs = cStmt.getResultSet();

        // process result set
        ...

        hadResults = cStmt.getMoreResults();
    }

    //
    // Retrieve output parameters
    //
    // Connector/J supports both index-based and
    // name-based retrieval
    //

    int outputValue = cStmt.getInt(2); // index-based

    outputValue = cStmt.getInt("inOutParam"); // name-based

...
```

7.4 Retrieving AUTO_INCREMENT Column Values through JDBC

`getGeneratedKeys()` is the preferred method to use if you need to retrieve `AUTO_INCREMENT` keys and through JDBC; this is illustrated in the first example below. The second example shows how you can retrieve the same value using a standard `SELECT LAST_INSERT_ID()` query. The final example shows how updatable result sets can retrieve the `AUTO_INCREMENT` value when using the `insertRow()` method.

Example 7.8 Connector/J: Retrieving `AUTO_INCREMENT` column values using `Statement.getGeneratedKeys()`

```
Statement stmt = null;
ResultSet rs = null;

try {

    //
    // Create a Statement instance that we can use for
    // 'normal' result sets assuming you have a
    // Connection 'conn' to a MySQL database already
    // available

    stmt = conn.createStatement();

    //
    // Issue the DDL queries for the table for this example
    //

    stmt.executeUpdate("DROP TABLE IF EXISTS autoIncTutorial");
    stmt.executeUpdate(
            "CREATE TABLE autoIncTutorial ("
            + "priKey INT NOT NULL AUTO_INCREMENT, "
            + "dataField VARCHAR(64), PRIMARY KEY (priKey))");

    //
    // Insert one row that will generate an AUTO INCREMENT
    // key in the 'priKey' field
    //

    stmt.executeUpdate(
            "INSERT INTO autoIncTutorial (dataField) "
            + "values ('Can I Get the Auto Increment Field?')",
            Statement.RETURN_GENERATED_KEYS);
```

```
    //
    // Example of using Statement.getGeneratedKeys()
    // to retrieve the value of an auto-increment
    // value
    //

    int autoIncKeyFromApi = -1;

    rs = stmt.getGeneratedKeys();

    if (rs.next()) {
        autoIncKeyFromApi = rs.getInt(1);
    } else {

        // throw an exception from here
    }

    System.out.println("Key returned from getGeneratedKeys():"
        + autoIncKeyFromApi);
} finally {

    if (rs != null) {
        try {
            rs.close();
        } catch (SQLException ex) {
            // ignore
        }
    }

    if (stmt != null) {
        try {
            stmt.close();
        } catch (SQLException ex) {
            // ignore
        }
    }
}
```

Example 7.9 Connector/J: Retrieving `AUTO_INCREMENT` column values using `SELECT LAST_INSERT_ID()`

```
Statement stmt = null;
ResultSet rs = null;

try {

    //
    // Create a Statement instance that we can use for
    // 'normal' result sets.

    stmt = conn.createStatement();

    //
    // Issue the DDL queries for the table for this example
    //

    stmt.executeUpdate("DROP TABLE IF EXISTS autoIncTutorial");
    stmt.executeUpdate(
            "CREATE TABLE autoIncTutorial ("
            + "priKey INT NOT NULL AUTO_INCREMENT, "
            + "dataField VARCHAR(64), PRIMARY KEY (priKey))");

    //
    // Insert one row that will generate an AUTO INCREMENT
    // key in the 'priKey' field
    //

    stmt.executeUpdate(
            "INSERT INTO autoIncTutorial (dataField) "
            + "values ('Can I Get the Auto Increment Field?')");
```

```
    //
    // Use the MySQL LAST_INSERT_ID()
    // function to do the same thing as getGeneratedKeys()
    //

    int autoIncKeyFromFunc = -1;
    rs = stmt.executeQuery("SELECT LAST_INSERT_ID()");

    if (rs.next()) {
        autoIncKeyFromFunc = rs.getInt(1);
    } else {
        // throw an exception from here
    }

    System.out.println("Key returned from " +
                       "'SELECT LAST_INSERT_ID()': " +
                       autoIncKeyFromFunc);

} finally {

    if (rs != null) {
        try {
            rs.close();
        } catch (SQLException ex) {
            // ignore
        }
    }

    if (stmt != null) {
        try {
            stmt.close();
        } catch (SQLException ex) {
            // ignore
        }
    }
}
```

Example 7.10 Connector/J: Retrieving `AUTO_INCREMENT` column values in `Updatable ResultSets`

```
Statement stmt = null;
ResultSet rs = null;

try {

    //
    // Create a Statement instance that we can use for
    // 'normal' result sets as well as an 'updatable'
    // one, assuming you have a Connection 'conn' to
    // a MySQL database already available
    //

    stmt = conn.createStatement(java.sql.ResultSet.TYPE_FORWARD_ONLY,
                                java.sql.ResultSet.CONCUR_UPDATABLE);

    //
    // Issue the DDL queries for the table for this example
    //

    stmt.executeUpdate("DROP TABLE IF EXISTS autoIncTutorial");
    stmt.executeUpdate(
            "CREATE TABLE autoIncTutorial ("
            + "priKey INT NOT NULL AUTO_INCREMENT, "
            + "dataField VARCHAR(64), PRIMARY KEY (priKey))");

    //
    // Example of retrieving an AUTO INCREMENT key
    // from an updatable result set
    //

    rs = stmt.executeQuery("SELECT priKey, dataField "
       + "FROM autoIncTutorial");
```

```
    rs.moveToInsertRow();

    rs.updateString("dataField", "AUTO INCREMENT here?");
    rs.insertRow();

    //
    // the driver adds rows at the end
    //

    rs.last();

    //
    // We should now be on the row we just inserted
    //

    int autoIncKeyFromRS = rs.getInt("priKey");

    System.out.println("Key returned for inserted row: "
        + autoIncKeyFromRS);

} finally {

    if (rs != null) {
        try {
            rs.close();
        } catch (SQLException ex) {
            // ignore
        }
    }

    if (stmt != null) {
        try {
            stmt.close();
        } catch (SQLException ex) {
            // ignore
        }
    }
}
```

Running the preceding example code should produce the following output:

```
Key returned from getGeneratedKeys(): 1
Key returned from SELECT LAST_INSERT_ID(): 1
Key returned for inserted row: 1
```

At times, it can be tricky to use the `SELECT LAST_INSERT_ID()` query, as that function's value is scoped to a connection. So, if some other query happens on the same connection, the value is overwritten. On the other hand, the `getGeneratedKeys()` method is scoped by the `Statement` instance, so it can be used even if other queries happen on the same connection, but not on the same `Statement` instance.

Chapter 8 Connection Pooling with Connector/J

Connection pooling is a technique of creating and managing a pool of connections that are ready for use by any thread that needs them. Connection pooling can greatly increase the performance of your Java application, while reducing overall resource usage.

How Connection Pooling Works

Most applications only need a thread to have access to a JDBC connection when they are actively processing a transaction, which often takes only milliseconds to complete. When not processing a transaction, the connection sits idle. Connection pooling enables the idle connection to be used by some other thread to do useful work.

In practice, when a thread needs to do work against a MySQL or other database with JDBC, it requests a connection from the pool. When the thread is finished using the connection, it returns it to the pool, so that it can be used by any other threads.

When the connection is loaned out from the pool, it is used exclusively by the thread that requested it. From a programming point of view, it is the same as if your thread called `DriverManager.getConnection()` every time it needed a JDBC connection. With connection pooling, your thread may end up using either a new connection or an already-existing connection.

Benefits of Connection Pooling

The main benefits to connection pooling are:

- Reduced connection creation time.

  Although this is not usually an issue with the quick connection setup that MySQL offers compared to other databases, creating new JDBC connections still incurs networking and JDBC driver overhead that will be avoided if connections are recycled.

- Simplified programming model.

  When using connection pooling, each individual thread can act as though it has created its own JDBC connection, allowing you to use straightforward JDBC programming techniques.

- Controlled resource usage.

  If you create a new connection every time a thread needs one rather than using connection pooling, your application's resource usage can be wasteful, and it could lead to unpredictable behaviors for your application when it is under a heavy load.

Using Connection Pooling with Connector/J

The concept of connection pooling in JDBC has been standardized through the JDBC 2.0 Optional interfaces, and all major application servers have implementations of these APIs that work with MySQL Connector/J.

Generally, you configure a connection pool in your application server configuration files, and access it through the Java Naming and Directory Interface (JNDI). The following code shows how you might use a connection pool from an application deployed in a J2EE application server:

Example 8.1 Connector/J: Using a connection pool with a J2EE application server

```
import java.sql.Connection;
import java.sql.SQLException;
import java.sql.Statement;

import javax.naming.InitialContext;
import javax.sql.DataSource;
```

```
public class MyServletJspOrEjb {

    public void doSomething() throws Exception {
        /*
         * Create a JNDI Initial context to be able to
         *  lookup  the DataSource
         *
         * In production-level code, this should be cached as
         * an instance or static variable, as it can
         * be quite expensive to create a JNDI context.
         *
         * Note: This code only works when you are using servlets
         * or EJBs in a J2EE application server. If you are
         * using connection pooling in standalone Java code, you
         * will have to create/configure datasources using whatever
         * mechanisms your particular connection pooling library
         * provides.
         */

        InitialContext ctx = new InitialContext();

         /*
          * Lookup the DataSource, which will be backed by a pool
          * that the application server provides. DataSource instances
          * are also a good candidate for caching as an instance
          * variable, as JNDI lookups can be expensive as well.
          */

        DataSource ds =
          (DataSource)ctx.lookup("java:comp/env/jdbc/MySQLDB");

        /*
         * The following code is what would actually be in your
         * Servlet, JSP or EJB 'service' method...where you need
         * to work with a JDBC connection.
         */

        Connection conn = null;
        Statement stmt = null;

        try {
            conn = ds.getConnection();

            /*
             * Now, use normal JDBC programming to work with
             * MySQL, making sure to close each resource when you're
             * finished with it, which permits the connection pool
             * resources to be recovered as quickly as possible
             */

            stmt = conn.createStatement();
            stmt.execute("SOME SQL QUERY");

            stmt.close();
            stmt = null;

            conn.close();
            conn = null;
        } finally {
            /*
             * close any jdbc instances here that weren't
             * explicitly closed during normal code path, so
             * that we don't 'leak' resources...
             */

            if (stmt != null) {
                try {
                    stmt.close();
                } catch (sqlexception sqlex) {
                    // ignore, as we can't do anything about it here
                }
```

```
                stmt = null;
            }

            if (conn != null) {
                try {
                    conn.close();
                } catch (sqlexception sqlex) {
                    // ignore, as we can't do anything about it here
                }

                conn = null;
            }
        }
    }
}
```

As shown in the example above, after obtaining the JNDI `InitialContext`, and looking up the `DataSource`, the rest of the code follows familiar JDBC conventions.

When using connection pooling, always make sure that connections, and anything created by them (such as statements or result sets) are closed. This rule applies no matter what happens in your code (exceptions, flow-of-control, and so forth). When these objects are closed, they can be re-used; otherwise, they will be stranded, which means that the MySQL server resources they represent (such as buffers, locks, or sockets) are tied up for some time, or in the worst case can be tied up forever.

Sizing the Connection Pool

Each connection to MySQL has overhead (memory, CPU, context switches, and so forth) on both the client and server side. Every connection limits how many resources there are available to your application as well as the MySQL server. Many of these resources will be used whether or not the connection is actually doing any useful work! Connection pools can be tuned to maximize performance, while keeping resource utilization below the point where your application will start to fail rather than just run slower.

The optimal size for the connection pool depends on anticipated load and average database transaction time. In practice, the optimal connection pool size can be smaller than you might expect. If you take Oracle's Java Petstore blueprint application for example, a connection pool of 15-20 connections can serve a relatively moderate load (600 concurrent users) using MySQL and Tomcat with acceptable response times.

To correctly size a connection pool for your application, create load test scripts with tools such as Apache JMeter or The Grinder, and load test your application.

An easy way to determine a starting point is to configure your connection pool's maximum number of connections to be unbounded, run a load test, and measure the largest amount of concurrently used connections. You can then work backward from there to determine what values of minimum and maximum pooled connections give the best performance for your particular application.

Validating Connections

MySQL Connector/J can validate the connection by executing a lightweight ping against a server. In the case of load-balanced connections, this is performed against all active pooled internal connections that are retained. This is beneficial to Java applications using connection pools, as the pool can use this feature to validate connections. Depending on your connection pool and configuration, this validation can be carried out at different times:

1. Before the pool returns a connection to the application.
2. When the application returns a connection to the pool.
3. During periodic checks of idle connections.

To use this feature, specify a validation query in your connection pool that starts with `/* ping */`. Note that the syntax must be exactly as specified. This will cause the driver send a ping to the server and return a dummy lightweight result set. When using a `ReplicationConnection` or `LoadBalancedConnection`, the ping will be sent across all active connections.

It is critical that the syntax be specified correctly. The syntax needs to be exact for reasons of efficiency, as this test is done for every statement that is executed:

```
protected static final String PING_MARKER = "/* ping */";
...
if (sql.charAt(0) == '/') {
if (sql.startsWith(PING_MARKER)) {
doPingInstead();
...
```

None of the following snippets will work, because the ping syntax is sensitive to whitespace, capitalization, and placement:

```
sql = "/* PING */ SELECT 1";
sql = "SELECT 1 /* ping*/";
sql = "/*ping*/ SELECT 1";
sql = " /* ping */ SELECT 1";
sql = "/*to ping or not to ping*/ SELECT 1";
```

All of the previous statements will issue a normal `SELECT` statement and will **not** be transformed into the lightweight ping. Further, for load-balanced connections, the statement will be executed against one connection in the internal pool, rather than validating each underlying physical connection. This results in the non-active physical connections assuming a stale state, and they may die. If Connector/J then re-balances, it might select a dead connection, resulting in an exception being passed to the application. To help prevent this, you can use `loadBalanceValidateConnectionOnSwapServer` to validate the connection before use.

If your Connector/J deployment uses a connection pool that allows you to specify a validation query, take advantage of it, but ensure that the query starts *exactly* with `/* ping */`. This is particularly important if you are using the load-balancing or replication-aware features of Connector/J, as it will help keep alive connections which otherwise will go stale and die, causing problems later.

Chapter 9 Multi-Host Connections

Table of Contents

9.1 Configuring Server Failover for Connections Using JDBC 121
9.2 Configuring Server Failover for Connections Using X DevAPI 124
9.3 Configuring Load Balancing with Connector/J 124
9.4 Configuring Source/Replica Replication with Connector/J 127
9.5 Advanced Load-balancing and Failover Configuration 130

The following sections discuss a number of topics that involve multi-host connections, namely, server load-balancing, failover, and replication.

Developers should know the following things about multi-host connections that are managed through Connector/J:

- Each multi-host connection is a wrapper of the underlying physical connections.
- Each of the underlying physical connections has its own session. Sessions cannot be tracked, shared, or copied, given the MySQL architecture.
- Every switch between physical connections means a switch between sessions.
- Within a transaction boundary, there are no switches between physical connections. Beyond a transaction boundary, there is no guarantee that a switch does not occur.

> **Note**
>
> If an application reuses session-scope data (for example, variables, SSPs) beyond a transaction boundary, failures are possible, as a switch between the physical connections (which is also a switch between sessions) might occur. Therefore, the application should re-prepare the session data and also restart the last transaction in case of an exception, or it should re-prepare session data for each new transaction if it does not want to deal with exception handling.

9.1 Configuring Server Failover for Connections Using JDBC

MySQL Connector/J supports server failover. A failover happens when connection-related errors occur for an underlying, active connection. The connection errors are, by default, propagated to the client, which has to handle them by, for example, recreating the working objects (`Statement`, `ResultSet`, etc.) and restarting the processes. Sometimes, the driver might eventually fall back to the original host automatically before the client application continues to run, in which case the host switch is transparent and the client application will not even notice it.

A connection using failover support works just like a standard connection: the client does not experience any disruptions in the failover process. This means the client can rely on the same connection instance even if two successive statements might be executed on two different physical hosts. However, this does not mean the client does not have to deal with the exception that triggered the server switch.

The failover is configured at the initial setup stage of the server connection by the connection URL (see explanations for its format here):

```
jdbc:mysql://[primary host][:port],[secondary host 1][:port][,[secondary host 2][:port]]...[/[database]
```

```
[?propertyName1=propertyValue1[&propertyName2=propertyValue2]...]
```

The host list in the connection URL comprises of two types of hosts, the primary and the secondary. When starting a new connection, the driver always tries to connect to the primary host first and, if required, fails over to the secondary hosts on the list sequentially when communication problems are experienced. Even if the initial connection to the primary host fails and the driver gets connected to a secondary host, the primary host never loses its special status: for example, it can be configured with an access mode distinct from those of the secondary hosts, and it can be put on a higher priority when a host is to be picked during a failover process.

The failover support is configured by the following connection properties (their functions are explained in the paragraphs below):

- `failOverReadOnly`
- `secondsBeforeRetrySource`
- `queriesBeforeRetrySource`
- `retriesAllDown`
- `autoReconnect`
- `autoReconnectForPools`

Configuring Connection Access Mode

As with any standard connection, the initial connection to the primary host is in read/write mode. However, if the driver fails to establish the initial connection to the primary host and it automatically switches to the next host on the list, the access mode now depends on the value of the property `failOverReadOnly`, which is “true” by default. The same happens if the driver is initially connected to the primary host and, because of some connection failure, it fails over to a secondary host. Every time the connection falls back to the primary host, its access mode will be read/write, irrespective of whether or not the primary host has been connected to before. The connection access mode can be changed any time at runtime by calling the method `Connection.setReadOnly(boolean)`, which partially overrides the property `failOverReadOnly`. When `failOverReadOnly=false` and the access mode is explicitly set to either true or false, it becomes the mode for every connection after a host switch, no matter what host type are being connected to; but, if `failOverReadOnly=true`, changing the access mode to read/write is only possible if the driver is connecting to the primary host; however, even if the access mode cannot be changed for the current connection, the driver remembers the client's last intention and, when falling back to the primary host, that is the mode that will be used. For an illustration, see the following successions of events with a two-host connection.

- Sequence A, with `failOverReadOnly=true`:
  1. Connects to primary host in read/write mode
  2. Sets `Connection.setReadOnly(true)`; primary host now in read-only mode
  3. Failover event; connects to secondary host in read-only mode
  4. Sets `Connection.setReadOnly(false)`; secondary host remains in read-only mode
  5. Falls back to primary host; connection now in read/write mode
- Sequence B, with `failOverReadOnly=false`
  1. Connects to primary host in read/write mode
  2. Sets `Connection.setReadOnly(true)`; primary host now in read-only mode
  3. Failover event; connects to secondary host in read-only mode

4. Set `Connection.setReadOnly(false)`; connection to secondary host switches to read/write mode
5. Falls back to primary host; connection now in read/write mode

The difference between the two scenarios is in step 4: the access mode for the secondary host in sequence A does not change at that step, but the driver remembers and uses the set mode when falling back to the primary host, which would be read-only otherwise; but in sequence B, the access mode for the secondary host changes immediately.

Configuring Fallback to Primary Host

As already mentioned, the primary host is special in the failover arrangement when it comes to the host's access mode. Additionally, the driver tries to fall back to the primary host as soon as possible by default, even if no communication exception occurs. Two properties, `secondsBeforeRetrySource` and `queriesBeforeRetrySource`, determine when the driver is ready to retry a reconnection to the primary host (the `Source` in the property names stands for the primary host of our connection URL, which is not necessarily a source host in a replication setup):

- `secondsBeforeRetrySource` determines how much time the driver waits before trying to fall back to the primary host
- `queriesBeforeRetrySource` determines the number of queries that are executed before the driver tries to fall back to the primary host. Note that for the driver, each call to a `Statement.execute*()` method increments the query execution counter; therefore, when calls are made to `Statement.executeBatch()` or if `allowMultiQueries` or `rewriteBatchStatements` are enabled, the driver may not have an accurate count of the actual number of queries executed on the server. Also, the driver calls the `Statement.execute*()` methods internally in several occasions. All these mean you can only use `queriesBeforeRetrySource` only as a coarse specification for when to fall back to the primary host.

In general, an attempt to fallback to the primary host is made when at least one of the conditions specified by the two properties is met, and the attempt always takes place at transaction boundaries. However, if auto-commit is turned off, the check happens only when the method `Connection.commit()` or `Connection.rollback()` is called. The automatic fallback to the primary host can be turned off by setting simultaneously `secondsBeforeRetrySource` and `queriesBeforeRetrySource` to “0”. Setting only one of the properties to “0” only disables one part of the check.

Configuring Reconnection Attempts

When establishing a new connection or when a failover event occurs, the driver tries to connect successively to the next candidate on the host list. When the end of the list has been reached, it restarts all over again from the beginning of the list; however, the primary host is skipped over, if (a) NOT all the secondary hosts have already been tested at least once, AND (b) the fallback conditions defined by `secondsBeforeRetrySource` and `queriesBeforeRetrySource` are not yet fulfilled. Each run-through of the whole host list, (which is not necessarily completed at the end of the host list) counts as a single connection attempt. The driver tries as many connection attempts as specified by the value of the property `retriesAllDown`.

Seamless Reconnection

Although not recommended, you can make the driver perform failovers without invalidating the active `Statement` or `ResultSet` instances by setting either the parameter `autoReconnect` or `autoReconnectForPools` to `true`. This allows the client to continue using the same object instances after a failover event, without taking any exceptional measures. This, however, may lead to unexpected results: for example, if the driver is connected to the primary host with read/write access

mode and it fails-over to a secondary host in read-only mode, further attempts to issue data-changing queries will result in errors, and the client will not be aware of that. This limitation is particularly relevant when using data streaming: after the failover, the `ResultSet` looks to be alright, but the underlying connection may have changed already, and no backing cursor is available anymore.

Configuring Server Failover Using JDBC with DNS SRV

See Section 6.14, “Support for DNS SRV Records” for details.

9.2 Configuring Server Failover for Connections Using X DevAPI

When using the X Protocol, Connector/J supports a client-side failover feature for establishing a Session. If multiple hosts are specified in the connection URL, when Connector/J fails to connect to a listed host, it tries to connect to another one. This is a sample X DevAPI URL for configuring client-side failover:

```
mysqlx://sandy:mypassword@[host1:33060,host2:33061]/test
```

With the client-side failover configured, when there is a failure to establish a connection, Connector/J keeps attempting to connect to a host on the host list. The order in which the hosts are attempted for connection is as follows:

- *For connections with the `priority` property set for each host in the connection URL,* hosts are attempted according to the set priorities for the hosts, which are specified by any numbers between 0 to 100, with a larger number indicating a higher priority for connection. For example:

  ```
  mysqlx://sandy:mypassword@[(address=host1:33060,priority=2),(address=host2:33061,priority=1)]/test
  ```

  In this example, `host1` is always attempted before `host2` when new sessions are created.

  Priorities should either be set for all or no hosts.

- *For connections with the `priority` property NOT set for each host in the connection URL,* hosts are attempted one after another in a random order.

Notice that the server failover feature for X DevAPI only allows for a failover when Connector/J is trying to establish a connection, but not during operations after a connection has already been made.

Connection Pooling Using X DevAPI. When using connection pooling with X DevAPI, Connector/J keeps track of any host it failed to connect to and, for a short waiting period after the failure, avoids connecting to it during the creation or retrieval of a `Session`. However, if all other hosts have already been tried, those excluded hosts will be retried without waiting. Once all hosts have been tried and no connections can be established, Connector/J throws a `com.mysql.cj.exceptions.CJCommunicationsException` and returns the message `Unable to connect to any of the target hosts`.

Configuring Server Failover Using X DevAPI with DNS SRV

See Section 6.14, “Support for DNS SRV Records” for details.

9.3 Configuring Load Balancing with Connector/J

Connector/J has long provided an effective means to distribute read/write load across multiple MySQL server instances for Cluster or source-source replication deployments. You can dynamically configure load-balanced connections, with no service outage. In-process transactions are not lost, and no application exceptions are generated if any application is trying to use that particular server instance.

The load balancing is configured at the initial setup stage of the server connection by the following connection URL, which has a similar format as the general JDBC URL for MySQL connection, but a specialized scheme:

```
jdbc:mysql:loadbalance://[host1][:port],[host2][:port][,[host3][:port]]...[/[database]] »
[?propertyName1=propertyValue1[&propertyName2=propertyValue2]...]
```

There are two configuration properties associated with this functionality:

- `loadBalanceConnectionGroup` – This provides the ability to group connections from different sources. This allows you to manage these JDBC sources within a single class loader in any combination you choose. If they use the same configuration, and you want to manage them as a logical single group, give them the same name. This is the key property for management: if you do not define a name (string) for `loadBalanceConnectionGroup`, you cannot manage the connections. All load-balanced connections sharing the same `loadBalanceConnectionGroup` value, regardless of how the application creates them, will be managed together.
- `ha.enableJMX` – The ability to manage the connections is exposed when you define a `loadBalanceConnectionGroup`; but if you want to manage this externally, enable JMX by setting this property to `true`. This enables a JMX implementation, which exposes the management and monitoring operations of a connection group. Further, start your application with the `-Dcom.sun.management.jmxremote` JVM flag. You can then perform connect and perform operations using a JMX client such as `jconsole`.

Once a connection has been made using the correct connection properties, a number of monitoring properties are available:

- Current active host count.
- Current active physical connection count.
- Current active logical connection count.
- Total logical connections created.
- Total transaction count.

The following management operations can also be performed:

- Add host.
- Remove host.

The JMX interface, `com.mysql.cj.jdbc.jmx.LoadBalanceConnectionGroupManagerMBean`, has the following methods:

- `int getActiveHostCount(String group);`
- `int getTotalHostCount(String group);`
- `long getTotalLogicalConnectionCount(String group);`
- `long getActiveLogicalConnectionCount(String group);`
- `long getActivePhysicalConnectionCount(String group);`
- `long getTotalPhysicalConnectionCount(String group);`
- `long getTotalTransactionCount(String group);`
- `void removeHost(String group, String host) throws SQLException;`
- `void stopNewConnectionsToHost(String group, String host) throws SQLException;`
- `void addHost(String group, String host, boolean forExisting);`
- `String getActiveHostsList(String group);`

- `String getRegisteredConnectionGroups();`

The `getRegisteredConnectionGroups()` method returns the names of all connection groups defined in that class loader.

You can test this setup with the following code:

```
public class Test {

    private static String URL = "jdbc:mysql:loadbalance://" +
        "localhost:3306,localhost:3310/test?" +
        "loadBalanceConnectionGroup=first&ha.enableJMX=true";

    public static void main(String[] args) throws Exception {
        new Thread(new Repeater()).start();
        new Thread(new Repeater()).start();
        new Thread(new Repeater()).start();
    }

    static Connection getNewConnection() throws SQLException, ClassNotFoundException {
        Class.forName("com.mysql.cj.jdbc.Driver");
        return DriverManager.getConnection(URL, "root", "");
    }

    static void executeSimpleTransaction(Connection c, int conn, int trans){
        try {
            c.setAutoCommit(false);
            Statement s = c.createStatement();
            s.executeQuery("SELECT SLEEP(1) /* Connection: " + conn + ", transaction: " + trans + " */");
            c.commit();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    public static class Repeater implements Runnable {
        public void run() {
            for(int i=0; i < 100; i++){
                try {
                    Connection c = getNewConnection();
                    for(int j=0; j < 10; j++){
                        executeSimpleTransaction(c, i, j);
                        Thread.sleep(Math.round(100 * Math.random()));
                    }
                    c.close();
                    Thread.sleep(100);
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        }
    }
}
```

After compiling, the application can be started with the `-Dcom.sun.management.jmxremote` flag, to enable remote management. `jconsole` can then be started. The `Test` main class will be listed by `jconsole`. Select this and click **Connect**. You can then navigate to the `com.mysql.cj.jdbc.jmx.LoadBalanceConnectionGroupManager` bean. At this point, you can click on various operations and examine the returned result.

If you now had an additional instance of MySQL running on port 3309, you could ensure that Connector/J starts using it by using the `addHost()`, which is exposed in `jconsole`. Note that these operations can be performed dynamically without having to stop the application running.

For further information on the combination of load balancing and failover, see Section 9.5, “Advanced Load-balancing and Failover Configuration”.

Configuring Load Balancing with DNS SRV

See Section 6.14, “Support for DNS SRV Records” for details.

9.4 Configuring Source/Replica Replication with Connector/J

This section describe a number of features of Connector/J's support for replication-aware deployments.

The replication is configured at the initial setup stage of the server connection by the connection URL, which has a similar format as the general JDBC URL for MySQL connection, but a specialized scheme:

```
jdbc:mysql:replication://[source host][:port],[replica host 1][:port][,[replica host 2][:port]]...[/[da
[?propertyName1=propertyValue1[&propertyName2=propertyValue2]...]
```

Users may specify the property `allowSourceDownConnections=true` to allow `Connection` objects to be created even though no source hosts are reachable. Such `Connection` objects report they are read-only, and `isSourceConnection()` returns false for them. The `Connection` tests for available source hosts when `Connection.setReadOnly(false)` is called, throwing an SQLException if it cannot establish a connection to a source, or switching to a source connection if the host is available.

Users may specify the property `allowReplicasDownConnections=true` to allow `Connection` objects to be created even though no replica hosts are reachable. A `Connection` then, at runtime, tests for available replica hosts when `Connection.setReadOnly(true)` is called (see explanation for the method below), throwing an SQLException if it cannot establish a connection to a replica, unless the property `readFromSourceWhenNoReplicas` is set to be “true” (see below for a description of the property).

Scaling out Read Load by Distributing Read Traffic to Replicas

Connector/J supports replication-aware connections. It can automatically send queries to a read/write source host, or a failover or round-robin loadbalanced set of replicas based on the state of `Connection.getReadOnly()`.

An application signals that it wants a transaction to be read-only by calling `Connection.setReadOnly(true)`. The replication-aware connection will use one of the replica connections, which are load-balanced per replica host using a round-robin scheme. A given connection is sticky to a replica until a transaction boundary command (a commit or rollback) is issued, or until the replica is removed from service. After calling `Connection.setReadOnly(true)`, if you want to allow connection to a source when no replicas are available, set the property `readFromSourceWhenNoReplicas` to “true.” Notice that the source host will be used in read-only state in those cases, as if it is a replica host. Also notice that setting `readFromSourceWhenNoReplicas=true` might result in an extra load for the source host in a transparent manner.

If you have a write transaction, or if you have a read that is time-sensitive (remember, replication in MySQL is asynchronous), set the connection to be not read-only, by calling `Connection.setReadOnly(false)` and the driver will ensure that further calls are sent to the source MySQL server. The driver takes care of propagating the current state of autocommit, isolation level, and catalog between all of the connections that it uses to accomplish this load balancing functionality.

To enable this functionality, use the specialized replication scheme (`jdbc:mysql:replication://`) when connecting to the server.

Here is a short example of how a replication-aware connection might be used in a standalone application:

```
import java.sql.Connection;
import java.sql.ResultSet;
```

```
import java.util.Properties;
import java.sql.DriverManager;

public class ReplicationDemo {

  public static void main(String[] args) throws Exception {

    Properties props = new Properties();

    // We want this for failover on the replicas
    props.put("autoReconnect", "true");

    // We want to load balance between the replicas
    props.put("roundRobinLoadBalance", "true");

    props.put("user", "foo");
    props.put("password", "password");

    //
    // Looks like a normal MySQL JDBC url, with a
    // comma-separated list of hosts, the first
    // being the 'source', the rest being any number
    // of replicas that the driver will load balance against
    //

    Connection conn =
        DriverManager.getConnection("jdbc:mysql:replication://source,replica1,replica2,replica3/test",
            props);

    //
    // Perform read/write work on the source
    // by setting the read-only flag to "false"
    //

    conn.setReadOnly(false);
    conn.setAutoCommit(false);
    conn.createStatement().executeUpdate("UPDATE some_table ....");
    conn.commit();

    //
    // Now, do a query from a replica, the driver automatically picks one
    // from the list
    //

    conn.setReadOnly(true);

    ResultSet rs =
      conn.createStatement().executeQuery("SELECT a,b FROM alt_table");

     .......
  }
}
```

Consider using the Load Balancing JDBC Pool (`lbpool`) tool, which provides a wrapper around the standard JDBC driver and enables you to use DB connection pools that includes checks for system failures and uneven load distribution. For more information, see Load Balancing JDBC Driver for MySQL (mysql-lbpool).

Support for Multiple-Source Replication Topographies

Connector/J supports multi-source replication topographies.

The connection URL for replication discussed earlier (i.e., in the format of `jdbc:mysql:replication://source,replica1,replica2,replica3/test`) assumes that the first (and only the first) host is the source host. Supporting deployments with an arbitrary number of sources and replicas requires the "address-equals" URL syntax for multiple host connection discussed in Section 6.2, “Connection URL Syntax”, with the property `type=[source|replica]`; for example:

```
jdbc:mysql:replication://address=(type=source)(host=source1host),address=(type=source)(host=source2host),ad
```

Connector/J uses a load-balanced connection internally for management of the source connections, which means that `ReplicationConnection`, when configured to use multiple sources, exposes the same options to balance load across source hosts as described in Section 9.3, “Configuring Load Balancing with Connector/J”.

Live Reconfiguration of Replication Topography

Connector/J also supports live management of replication host (single or multi-source) topographies. This enables users to promote replicas for Java applications without requiring an application restart.

The replication hosts are most effectively managed in the context of a replication connection group. A ReplicationConnectionGroup class represents a logical grouping of connections which can be managed together. There may be one or more such replication connection groups in a given Java class loader (there can be an application with two different JDBC resources needing to be managed independently). This key class exposes host management methods for replication connections, and `ReplicationConnection` objects register themselves with the appropriate `ReplicationConnectionGroup` if a value for the new `replicationConnectionGroup` property is specified. The `ReplicationConnectionGroup` object tracks these connections until they are closed, and it is used to manipulate the hosts associated with these connections.

Some important methods related to host management include:

- `getSourceHosts()`: Returns a collection of strings representing the hosts configured as source hosts
- `getReplicaHosts()`: Returns a collection of strings representing the hosts configured as replica hosts
- `addReplicaHost(String host)`: Adds new host to pool of possible replica hosts for selection at start of new read-only workload
- `promoteReplicaToSource(String host)`: Removes the host from the pool of potential replica hosts for future read-only processes (existing read-only process is allowed to continue to completion) and adds the host to the pool of potential source hosts
- `removeReplicaHost(String host, boolean closeGently)`: Removes the host (host name match must be exact) from the list of configured replica hosts; if `closeGently` is false, existing connections which have this host as currently active will be closed hardly (application should expect exceptions)
- `removeSourceHost(String host, boolean closeGently)`: Same as `removeReplicaHost()`, but removes the host from the list of configured source hosts

Some useful management metrics include:

- `getConnectionCountWithHostAsReplica(String host)`: Returns the number of ReplicationConnection objects that have the given host configured as a possible replica host
- `getConnectionCountWithHostAsSource(String host)`: Returns the number of ReplicationConnection objects that have the given host configured as a possible source host
- `getNumberOfReplicasAdded()`: Returns the number of times a replica host has been dynamically added to the group pool
- `getNumberOfReplicasRemoved()`: Returns the number of times a replica host has been dynamically removed from the group pool
- `getNumberOfReplicaPromotions()`: Returns the number of times a replica host has been promoted to be a source host
- `getTotalConnectionCount()`: Returns the number of ReplicationConnection objects which have been registered with this group

- `getActiveConnectionCount()`: Returns the number of ReplicationConnection objects currently being managed by this group

ReplicationConnectionGroupManager

`com.mysql.cj.jdbc.ha.ReplicationConnectionGroupManager` provides access to the replication connection groups, together with some utility methods.

- `getConnectionGroup(String groupName)`: Returns the `ReplicationConnectionGroup` object matching the groupName provided

The other methods in `ReplicationConnectionGroupManager` mirror those of `ReplicationConnectionGroup`, except that the first argument is a String group name. These methods will operate on all matching ReplicationConnectionGroups, which are helpful for removing a server from service and have it decommissioned across all possible `ReplicationConnectionGroups`.

These methods might be useful for in-JVM management of replication hosts if an application triggers topography changes. For managing host configurations from outside the JVM, JMX can be used.

Using JMX for Managing Replication Hosts

When Connector/J is started with `ha.enableJMX=true` and a value set for the property `replicationConnectionGroup`, a JMX MBean will be registered, allowing manipulation of replication hosts by a JMX client. The MBean interface is defined in `com.mysql.cj.jdbc.jmx.ReplicationGroupManagerMBean`, and leverages the `ReplicationConnectionGroupManager` static methods:

```
public abstract void addReplicaHost(String groupFilter, String host) throws SQLException;
public abstract void removeReplicaHost(String groupFilter, String host) throws SQLException;
public abstract void promoteReplicaToSource(String groupFilter, String host) throws SQLException;
public abstract void removeSourceHost(String groupFilter, String host) throws SQLException;
public abstract String getSourceHostsList(String group);
public abstract String getReplicaHostsList(String group);
public abstract String getRegisteredConnectionGroups();
public abstract int getActiveSourceHostCount(String group);
public abstract int getActiveReplicaHostCount(String group);
public abstract int getReplicaPromotionCount(String group);
public abstract long getTotalLogicalConnectionCount(String group);
public abstract long getActiveLogicalConnectionCount(String group);
```

Configuring Source/Replica Replication with DNS SRV

See Section 6.14, “Support for DNS SRV Records” for details.

9.5 Advanced Load-balancing and Failover Configuration

Connector/J provides a useful load-balancing implementation for MySQL Cluster or multi-source deployments, as explained in Section 9.3, “Configuring Load Balancing with Connector/J” and Support for Multiple-Source Replication Topographies. This same implementation is used for balancing load between read-only replicas for replication-aware connections.

When trying to balance workload between multiple servers, the driver has to determine when it is safe to swap servers, doing so in the middle of a transaction, for example, could cause problems. It is important not to lose state information. For this reason, Connector/J will only try to pick a new server when one of the following happens:

1. At transaction boundaries (transactions are explicitly committed or rolled back).

2. A communication exception (SQL State starting with "08") is encountered.

3. When a `SQLException` matches conditions defined by user, using the extension points defined by the `loadBalanceSQLStateFailover`, `loadBalanceSQLExceptionSubclassFailover` or `loadBalanceExceptionChecker` properties.

The third condition revolves around three properties, which allow you to control which `SQLException`s trigger failover:

- `loadBalanceExceptionChecker` - The `loadBalanceExceptionChecker` property is really the key. This takes a fully-qualified class name which implements the new `com.mysql.cj.jdbc.ha.LoadBalanceExceptionChecker` interface. This interface is very simple, and you only need to implement the following method:

  ```
  public boolean shouldExceptionTriggerFailover(SQLException ex)
  ```

  A `SQLException` is passed in, and a boolean returned. A value of `true` triggers a failover, `false` does not.

  You can use this to implement your own custom logic. An example where this might be useful is when dealing with transient errors when using MySQL Cluster, where certain buffers may become overloaded. The following code snippet illustrates this:

  ```
  public class NdbLoadBalanceExceptionChecker
   extends StandardLoadBalanceExceptionChecker {

   public boolean shouldExceptionTriggerFailover(SQLException ex) {
    return super.shouldExceptionTriggerFailover(ex)
      ||  checkNdbException(ex);
   }

   private boolean checkNdbException(SQLException ex){
   // Have to parse the message since most NDB errors
   // are mapped to the same DEMC.
    return (ex.getMessage().startsWith("Lock wait timeout exceeded") ||
    (ex.getMessage().startsWith("Got temporary error")
    && ex.getMessage().endsWith("from NDB")));
   }
  }
  ```

  The code above extends `com.mysql.cj.jdbc.ha.StandardLoadBalanceExceptionChecker`, which is the default implementation. There are a few convenient shortcuts built into this, for those who want to have some level of control using properties, without writing Java code. This default implementation uses the two remaining properties: `loadBalanceSQLStateFailover` and `loadBalanceSQLExceptionSubclassFailover`.

- `loadBalanceSQLStateFailover` - allows you to define a comma-delimited list of `SQLState` code prefixes, against which a `SQLException` is compared. If the prefix matches, failover is triggered. So, for example, the following would trigger a failover if a given `SQLException` starts with "00", or is "12345":

  ```
  loadBalanceSQLStateFailover=00,12345
  ```

- `loadBalanceSQLExceptionSubclassFailover` - can be used in conjunction with `loadBalanceSQLStateFailover` or on its own. If you want certain subclasses of `SQLException` to trigger failover, simply provide a comma-delimited list of fully-qualified class or interface names to check against. For example, if you want all `SQLTransientConnectionExceptions` to trigger failover, you would specify:

  ```
  loadBalanceSQLExceptionSubclassFailover=java.sql.SQLTransientConnectionException
  ```

While the three failover conditions enumerated earlier suit most situations, if `autocommit` is enabled, Connector/J never re-balances, and continues using the same physical connection. This can be problematic, particularly when load-balancing is being used to distribute read-only load across multiple

replicas. However, Connector/J can be configured to re-balance after a certain number of statements are executed, when `autocommit` is enabled. This functionality is dependent upon the following properties:

- `loadBalanceAutoCommitStatementThreshold` – defines the number of matching statements which will trigger the driver to potentially swap physical server connections. The default value, 0, retains the behavior that connections with `autocommit` enabled are never balanced.
- `loadBalanceAutoCommitStatementRegex` – the regular expression against which statements must match. The default value, blank, matches all statements. So, for example, using the following properties will cause Connector/J to re-balance after every third statement that contains the string “test”:

```
loadBalanceAutoCommitStatementThreshold=3
loadBalanceAutoCommitStatementRegex=.*test.*
```

`loadBalanceAutoCommitStatementRegex` can prove useful in a number of situations. Your application may use temporary tables, server-side session state variables, or connection state, where letting the driver arbitrarily swap physical connections before processing is complete could cause data loss or other problems. This allows you to identify a trigger statement that is only executed when it is safe to swap physical connections.

Configuring Load Balancing and Failover with DNS SRV

See Section 6.14, “Support for DNS SRV Records” for details.

Chapter 10 Using the X DevAPI with Connector/J: Special Topics

Table of Contents

10.1 Connection Compression Using X DevAPI 133
10.2 Schema Validation 134

Connector/J 8.3 supports the X DevAPI, through which native support by MySQL for JSON, NoSQL, document collection, and other features are provided to Java applications. See Using MySQL as a Document Store, the X DevAPI User Guide, and the *Connector/J X DevAPI Reference* available at Connectors and APIs for details.

Information on using the X DevAPI with Connector/J can be found in different chapters in this manual. This chapter explores some special topics that are not covered elsewhere.

10.1 Connection Compression Using X DevAPI

Connector/J supports data compression for X DevAPI connections when working with MySQL Server 8.0.19 and later. General details about this feature can be found in Connection Compression with X Plugin. For details on how to configure connection compression for Connector/J, see the descriptions for the connection properties `xdevapi.compression`, `xdevapi.compression-algorithms`, and `xdevapi.compression-extensions` in Section 6.3, "Configuration Properties". The following is a summary of the feature:

The compression algorithms to be negotiated with the server and the priority of negotiation can be specified using the connection property `xdevapi.compression-algorithms`. It accepts a list of *`[algorithm-name]_[operation-mode]`*, separated by commas (,). If the property is not set, the default value of "`zstd_stream,lz4_message,deflate_stream`" is used. The priority for negotiation follows the order the algorithms appear in the list. Setting an empty string explicitly for the property means compression should be disabled for the connection.

> **Note**
>
> When specifying compression algorithms with `xdevapi.compression-algorithms`, the aliases `zstd`, `lz4`, and `deflate` can be used in place of `zstd_stream`, `lz4_message`, and `deflate_stream`, respectively.

Out of all the compression algorithms now supported by MySQL Server for X DevAPI connections, Connector/J provides out-of-the-box support for Deflate only; this is because none of the other compression algorithms (LZ4 and zstd, for now) are natively supported by the existing JREs. To support those algorithms, the client application must provide implementations for the corresponding deflate and inflate operations in the form of an `OutputStream` and an `InputStream` object, respectively. The easiest way to accomplish this is by using a third-party library such as the Apache Commons Compress library, which supports LZ4 and zstd. The connection option `xdevapi.compression-extensions` allows users to configure Connector/J to use any compression algorithm that is supported by MySQL Server, as long as there is a Java implementation for that algorithm. The option takes a list of triplets separated by commas (,), and each triplet in turn contains the following elements, separated by colons (:):

- The compression algorithm name, indicated by the identifier used by the server (see Connection Compression with X Plugin; aliases mentioned in the Note above can be used).
- A fully-qualified name of a class implementing the interface `java.io.InputStream` that will be used to *inflate* data compressed with the named algorithm.

- A fully-qualified name of a class implementing the interface `java.io.OutputStream` that will be used to *deflate* data using the named algorithm.

Here is an example that sets up the support for the algorithms `lz4_message` and `zstd_stream` using the Apache Commons Compress library:

```
String connStr = "jdbc:mysql://johndoe:secret@localhost:33060/mydb?"
    + "xdevapi.compression-extensions="
    + "lz4_message"+":" // LZ4 triplet
    + FramedLZ4CompressorInputStream.class.getName() + ":"
    + FramedLZ4CompressorOutputStream.class.getName() + ","
    + "zstd_stream"+":" // zstd triplet
    + ZstdCompressorInputStream.class.getName() + ":"
    + ZstdCompressorOutputStream.class.getName();
SessionFactory sessFact = new SessionFactory();
Session sess = sessFact.getSession(connStr);
Collection col = sess.getDefaultSchema().getCollection("myCollection");
// (...)
sess.close();
```

> **Note**
>
> *For Connector/J 8.0.21 and earlier:* The connection property `xdevapi.compression-extensions` described above is named `xdevapi.compression-algorithm` for Connector/J 8.0.21 and earlier, and the elements in each triplet should be separated by commas (,) instead of colons (:).

Negotiation for a compression algorithm is attempted by default (`xdevapi.compression`=`Preferred` by default), unless the connection property `xdevapi.compression` is set to `DISABLED`. The final choice of compression algorithm depends on what algorithms are enabled on the server. By default, because compression is not required, if the negotiation fails, the connection will not be compressed, but the client will still be able to communicate with the server; however, if the connection property `xdevapi.compression` is set to `REQUIRED`, the connection attempt fails with an error if no algorithm can be negotiated for successfully.

10.2 Schema Validation

When working with MySQL Server 8.0.19 and later: Schema validation can be configured for a `Collection`, so that documents in the `Collection` are validated against a schema before they can be inserted or updated. This is done by specifying a JSON Schema during `Collection` creation or modification; schema validation is then performed by the server at a document creation or update, and an error is returned if the document does not validate against the assigned schema. For more information on JSON schema validation in MySQL, see JSON Schema Validation Functions. This section describes how to configure schema validation for a `Collection` with Connector/J.

To configure schema validation during the creation of a `Collection`, pass to the `createCollection()` method a `CreateCollectionOptions` object, which has these fields:

- `reuse`: a boolean set by the `setReuseExisting` method. If it is `true`, when the `Collection` to be created already exists within the `Schema` that is to contain it, Connector/J returns success (without any attempt to apply JSON schema to the existing `Collection`); in the same case, Connector/J returns an error if the parameter is set to `false`. If `reuse` is not set, it is taken to be `false`.
- `validation`: a `Validation` object set by the `setValidation()` method. A `Validation` object in turns contains these fields:
  - `level`: a enumeration of the class `ValidationLevel`, set by the `setLevel()` method; it can be one of the following two values:
    - `STRICT`: Strict validation. Attempting to insert or modify a document that violates the validation schema results in a server error being raised.

  - `OFF`: No validation. Schema validation is turned off.

  If `level` is not set, it is taken as `OFF` for MySQL Server 8.0.19, and `STRICT` for 8.0.20 and later.

- `schema`: A string representing a JSON Schema to be used to validate a `Document` in the `Collection`; set by the `setSchema()` method.

  If `schema` is not provided but `level` is set to STRICT, the `Collection` is validated against the default schema `{"type" : "object"}`.

This is an example of how to configure schema validation at the creation of a `Collection`:

```
Collection coll = this.schema.createCollection(collName,
    new CreateCollectionOptions()
        .setReuseExisting(false)
        .setValidation(new Validation()
           .setLevel(ValidationLevel.STRICT)
           .setSchema(
               "{\"id\": \"http://json-schema.org/geo\","
               + "\"$schema\": \"http://json-schema.org/draft-06/schema#\","
               + "       \"description\": \"A geographical coordinate\","
               + "       \"type\": \"object\","
               + "       \"properties\": {"
               + "          \"latitude\": {"
               + "             \"type\": \"number\""
               + "          },"
               + "          \"longitude\": {"
               + "             \"type\": \"number\""
               + "          }"
               + "       },"
               + "       \"required\": [\"latitude\", \"longitude\"]"
               + "  }"
)));
```

The set fields are accessible by the corresponding getter methods.

To modify the schema validation configuration for a `Collection`, use the `modifyCollection()` method and pass to it a `ModifyCollectionOptions` object, which has the same fields as the `CreateCollectionOptions` object except for the `reuse` field, which does not exist for a `ModifyCollectionOptions` object. For the `Validation` object of a `ModifyCollectionOptions` object, users can set either its `level` or `schema`, or both. Here is an example of using the `modifyCollection()` to change the schema validation configuration:

```
schema.modifyCollection(collName,
    new ModifyCollectionOptions()
        .setValidation(new Validation()
           .setLevel(ValidationLevel.OFF)
           .setSchema(
               "{\"id\": \"http://json-schema.org/geo\","
               + "\"$schema\": \"http://json-schema.org/draft-06/schema#\","
               + "       \"description\": \"NEW geographical coordinate\","
               + "       \"type\": \"object\","
               + "       \"properties\": {"
               + "          \"latitude\": {"
               + "             \"type\": \"number\""
               + "          },"
               + "          \"longitude\": {"
               + "             \"type\": \"number\""
               + "          }"
               + "       },"
               + "       \"required\": [\"latitude\", \"longitude\"]"
               + "  }"
)));
```

If the Collection contains documents that do not validate against the new JSON schema supplied through `ModifyCollectionOptions`, the server will reject the schema modification with the error

`ERROR 5180 (HY000) Document is not valid according to the schema assigned to collection`.

> **Note**
>
> `createCollection()` and `modifyCollection()` are overloaded: they can be called without passing to them the `CreateCollectionOptions` or the `ModifyCollectionOptions`, respectively, in which case schema validation will not be applied to the `Collection`.

Chapter 11 Using the Connector/J Interceptor Classes

An interceptor is a software design pattern that provides a transparent way to extend or modify some aspect of a program, similar to a user exit. No recompiling is required. With Connector/J, the interceptors are enabled and disabled by updating the connection string to refer to different sets of interceptor classes that you instantiate.

The connection properties that control the interceptors are explained in Section 6.3, “Configuration Properties”:

- `connectionLifecycleInterceptors`, where you specify the fully qualified names of classes that implement the `com.mysql.cj.jdbc.interceptors.ConnectionLifecycleInterceptor` interface. In these kinds of interceptor classes, you might log events such as rollbacks, measure the time between transaction start and end, or count events such as calls to `setAutoCommit()`.
- `exceptionInterceptors`, where you specify the fully qualified names of classes that implement the `com.mysql.cj.exceptions.ExceptionInterceptor` interface. In these kinds of interceptor classes, you might add extra diagnostic information to exceptions that can have multiple causes or indicate a problem with server settings. `exceptionInterceptors` classes are called when handling an `Exception` thrown from Connector/J code.
- `queryInterceptors`, where you specify the fully qualified names of classes that implement the `com.mysql.cj.interceptors.QueryInterceptor` interface. In these kinds of interceptor classes, you might change or augment the processing done by certain kinds of statements, such as automatically checking for queried data in a `memcached` server, rewriting slow queries, logging information about statement execution, or route requests to remote servers.

Chapter 12 Using Logging Frameworks with SLF4J

Besides its default logger `com.mysql.cj.log.StandardLogger`, which logs to `stderr`, Connector/J supports the SLF4J logging facade, allowing end users of applications using Connector/J to plug in logging frameworks of their own choices at deployment time. Popular logging frameworks such as `java.util.logging`, `logback`, and `log4j` are supported by SLF4J. Follow these requirements to use a logging framework with SLF4J and Connector/J:

- In the development environment:
  - Install on your system `slf4j-api-x.y.z.jar` (available at https://www.slf4j.org/download.html) and add it to the Java classpath.
  - In the code of your application, obtain an `SLF4JLogger` as a `Log` instantiated within a `MysqlConnection` `Session`, and then use the `Log` instance for your logging.
- On the deployment system:
  - Install on your system `slf4j-api-x.y.z.jar` and add it to the Java classpath
  - Install on your system the SLF4J binding for the logging framework of your choice and add it to your Java classpath. SLF4J bindings are available at, for example, https://www.slf4j.org/manual.html#swapping.

    Note

    Do not put more than one SLF4J binding in you Java classpath. Switch from one logging framework to another by removing a binding and adding a new one to the classpath.
  - Install the logging framework of your choice on your system and add it to the Java classpath.
  - Configure the logging framework of your choice. This often consists of setting up appenders or handlers for log messages using a configuration file; see your logging framework's documentation for details.
  - When connecting the application to the MySQL Server, set the Connector/J connection property `logger` to `Slf4JLogger`.

The log category name used by Connector/J with SLF4J is `MySQL`. See the SLF4J user manual for more details about using SLF4J, including discussions on Maven dependency and bindings. Here is a sample code for using SLF4J with Connector/J:

```
import java.sql.DriverManager;
import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import com.mysql.cj.jdbc.JdbcConnection;
import com.mysql.cj.log.Log;

public class JDBCDemo {

  public static void main(String[] args) {

    Connection conn = null;
    Statement statement = null;
    ResultSet resultSet = null;
    Log logger = null;

  try {
     // Database parameters
     String url = "jdbc:mysql://myexample.com:3306/pets?logger=Slf4JLogger&explainSlowQueries=true";
     String user = "user";
```

```
    String password = "password";
    // create a connection to the database
    conn = DriverManager.getConnection(url, user, password);
    logger = ((JdbcConnection)conn).getSession().getLog();
  }
  catch (SQLException e) {
    System.err.println(e.getMessage());
    System.exit(1);
   }

  try {
    statement = conn.createStatement();
    resultSet = statement.executeQuery("SELECT * FROM pets.dogs");
    while(resultSet.next()){
      System.out.printf("%d\t%s\t%s\t %4$ty.%4$tm.%4$td \n",
      resultSet.getInt(1),
      resultSet.getString(2),
      resultSet.getString(3),
      resultSet.getDate(4));
    }
  }
  catch(SQLException e) {
    logger.logWarn("Warning: Select failed!");
  }

}

}
```

If you want to use, for example, Log4j 2.17.1 as your logging framework when running this program, put these JAR files in your Java classpath:

- `slf4j-api-2.0.3.jar` (SLF4J API module, available at, for example, https://search.maven.org/artifact/org.slf4j/slf4j-api/2.0.3/jar).
- `log4j-api-2.17.1.jar` and `log4j-core-2.17.1.jar` (Log4J library, available at, for example, https://search.maven.org/artifact/org.apache.logging.log4j/log4j-api/2.17.1/jar and https://search.maven.org/artifact/org.apache.logging.log4j/log4j-core/2.17.1/jar).
- `log4j-slf4j-impl-2.17.1.jar` (SLF4J's binding for Log4J 2.17.1, available at, for example, https://search.maven.org/artifact/org.apache.logging.log4j/log4j-slf4j-impl/2.17.1/jar).

Here is output of the program when the SELECT statement failed:

```
[2021-09-05 12:06:19,624] WARN     0[main] - WARN MySQL - Warning: Select failed!
```

Chapter 13 Using Connector/J with Tomcat

The following instructions are based on the instructions for Tomcat-5.x, available at http://tomcat.apache.org/tomcat-5.5-doc/jndi-datasource-examples-howto.html which is current at the time this document was written.

First, install the `.jar` file that comes with Connector/J in `$CATALINA_HOME/common/lib` so that it is available to all applications installed in the container.

Next, configure the JNDI DataSource by adding a declaration resource to `$CATALINA_HOME/conf/server.xml` in the context that defines your web application:

```
<Context ....>

...

<Resource name="jdbc/MySQLDB"
              auth="Container"
              type="javax.sql.DataSource"/>

<ResourceParams name="jdbc/MySQLDB">
  <parameter>
    <name>factory</name>
    <value>org.apache.commons.dbcp.BasicDataSourceFactory</value>
  </parameter>

  <parameter>
    <name>maxActive</name>
    <value>10</value>
  </parameter>

  <parameter>
    <name>maxIdle</name>
    <value>5</value>
  </parameter>

  <parameter>
    <name>validationQuery</name>
    <value>SELECT 1</value>
  </parameter>

  <parameter>
    <name>testOnBorrow</name>
    <value>true</value>
  </parameter>

  <parameter>
    <name>testWhileIdle</name>
    <value>true</value>
  </parameter>

  <parameter>
    <name>timeBetweenEvictionRunsMillis</name>
    <value>10000</value>
  </parameter>

  <parameter>
    <name>minEvictableIdleTimeMillis</name>
    <value>60000</value>
  </parameter>

  <parameter>
   <name>username</name>
   <value>someuser</value>
  </parameter>

  <parameter>
   <name>password</name>
   <value>somepass</value>
  </parameter>
```

```
    <parameter>
       <name>driverClassName</name>
       <value>com.mysql.cj.jdbc.Driver</value>
    </parameter>

    <parameter>
      <name>url</name>
      <value>jdbc:mysql://localhost:3306/test</value>
    </parameter>

  </ResourceParams>
</Context>
```

Connector/J introduces a facility whereby, rather than use a `validationQuery` value of `SELECT 1`, it is possible to use `validationQuery` with a value set to `/* ping */`. This sends a ping to the server which then returns a fake result set. This is a lighter weight solution. It also has the advantage that if using `ReplicationConnection` or `LoadBalancedConnection` type connections, the ping will be sent across all active connections. The following XML snippet illustrates how to select this option:

```
<parameter>
 <name>validationQuery</name>
 <value>/* ping */</value>
</parameter>
```

Note that `/* ping */` has to be specified exactly.

In general, follow the installation instructions that come with your version of Tomcat, as the way you configure datasources in Tomcat changes from time to time, and if you use the wrong syntax in your XML file, you will most likely end up with an exception similar to the following:

```
Error: java.sql.SQLException: Cannot load JDBC driver class 'null ' SQL
state: null
```

Note that the auto-loading of drivers having the `META-INF/service/java.sql.Driver` class in JDBC 4.0 and later causes an improper undeployment of the Connector/J driver in Tomcat on Windows. Namely, the Connector/J jar remains locked. This is an initialization problem that is not related to the driver. The possible workarounds, if viable, are as follows: use "`antiResourceLocking=true`" as a Tomcat Context attribute, or remove the `META-INF/` directory.

Chapter 14 Using Connector/J with Spring

Table of Contents

14.1 Using `JdbcTemplate` 144
14.2 Transactional JDBC Access 145
14.3 Connection Pooling with Spring 147

The Spring Framework is a Java-based application framework designed for assisting in application design by providing a way to configure components. The technique used by Spring is a well known design pattern called Dependency Injection (see Inversion of Control Containers and the Dependency Injection pattern). This article will focus on Java-oriented access to MySQL databases with Spring 2.0. For those wondering, there is a .NET port of Spring appropriately named Spring.NET.

Spring is not only a system for configuring components, but also includes support for aspect oriented programming (AOP). This is one of the main benefits and the foundation for Spring's resource and transaction management. Spring also provides utilities for integrating resource management with JDBC and Hibernate.

For the examples in this section the MySQL world sample database will be used. The first task is to set up a MySQL data source through Spring. Components within Spring use the “bean” terminology. For example, to configure a connection to a MySQL server supporting the world sample database, you might use:

```
<util:map id="dbProps">
    <entry key="db.driver" value="com.mysql.cj.jdbc.Driver"/>
    <entry key="db.jdbcurl" value="jdbc:mysql://localhost/world"/>
    <entry key="db.username" value="myuser"/>
    <entry key="db.password" value="mypass"/>
</util:map>
```

In the above example, we are assigning values to properties that will be used in the configuration. For the datasource configuration:

```
<bean id="dataSource"
       class="org.springframework.jdbc.datasource.DriverManagerDataSource">
    <property name="driverClassName" value="${db.driver}"/>
    <property name="url" value="${db.jdbcurl}"/>
    <property name="username" value="${db.username}"/>
    <property name="password" value="${db.password}"/>
</bean>
```

The placeholders are used to provide values for properties of this bean. This means that we can specify all the properties of the configuration in one place instead of entering the values for each property on each bean. We do, however, need one more bean to pull this all together. The last bean is responsible for actually replacing the placeholders with the property values.

```
<bean
 class="org.springframework.beans.factory.config.PropertyPlaceholderConfigurer">
    <property name="properties" ref="dbProps"/>
</bean>
```

Now that we have our MySQL data source configured and ready to go, we write some Java code to access it. The example below will retrieve three random cities and their corresponding country using the data source we configured with Spring.

```
// Create a new application context. this processes the Spring config
ApplicationContext ctx =
    new ClassPathXmlApplicationContext("ex1appContext.xml");
// Retrieve the data source from the application context
    DataSource ds = (DataSource) ctx.getBean("dataSource");
// Open a database connection using Spring's DataSourceUtils
Connection c = DataSourceUtils.getConnection(ds);
try {
    // retrieve a list of three random cities
    PreparedStatement ps = c.prepareStatement(
        "select City.Name as 'City', Country.Name as 'Country' " +
        "from City inner join Country on City.CountryCode = Country.Code " +
        "order by rand() limit 3");
    ResultSet rs = ps.executeQuery();
    while(rs.next()) {
        String city = rs.getString("City");
        String country = rs.getString("Country");
        System.out.printf("The city %s is in %s%n", city, country);
    }
} catch (SQLException ex) {
    // something has failed and we print a stack trace to analyse the error
    ex.printStackTrace();
    // ignore failure closing connection
    try { c.close(); } catch (SQLException e) { }
} finally {
    // properly release our connection
    DataSourceUtils.releaseConnection(c, ds);
}
```

This is very similar to normal JDBC access to MySQL with the main difference being that we are using DataSourceUtils instead of the DriverManager to create the connection.

While it may seem like a small difference, the implications are somewhat far reaching. Spring manages this resource in a way similar to a container managed data source in a J2EE application server. When a connection is opened, it can be subsequently accessed in other parts of the code if it is synchronized with a transaction. This makes it possible to treat different parts of your application as transactional instead of passing around a database connection.

14.1 Using `JdbcTemplate`

Spring makes extensive use of the Template method design pattern (see Template Method Pattern). Our immediate focus will be on the `JdbcTemplate` and related classes, specifically `NamedParameterJdbcTemplate`. The template classes handle obtaining and releasing a connection for data access when one is needed.

The next example shows how to use `NamedParameterJdbcTemplate` inside of a DAO (Data Access Object) class to retrieve a random city given a country code.

```
public class Ex2JdbcDao {
     /**
     * Data source reference which will be provided by Spring.
     */
     private DataSource dataSource;

     /**
     * Our query to find a random city given a country code. Notice
     * the ":country" parameter toward the end. This is called a
     * named parameter.
     */
     private String queryString = "select Name from City " +
        "where CountryCode = :country order by rand() limit 1";

     /**
     * Retrieve a random city using Spring JDBC access classes.
     */
     public String getRandomCityByCountryCode(String cntryCode) {
         // A template that permits using queries with named parameters
         NamedParameterJdbcTemplate template =
```

```
            new NamedParameterJdbcTemplate(dataSource);
            // A java.util.Map is used to provide values for the parameters
            Map params = new HashMap();
            params.put("country", cntryCode);
            // We query for an Object and specify what class we are expecting
            return (String)template.queryForObject(queryString, params, String.class);
        }

    /**
    * A JavaBean setter-style method to allow Spring to inject the data source.
    * @param dataSource
    */
    public void setDataSource(DataSource dataSource) {
        this.dataSource = dataSource;
    }
}
```

The focus in the above code is on the `getRandomCityByCountryCode()` method. We pass a country code and use the `NamedParameterJdbcTemplate` to query for a city. The country code is placed in a Map with the key "country", which is the parameter is named in the SQL query.

To access this code, you need to configure it with Spring by providing a reference to the data source.

```
<bean id="dao" class="code.Ex2JdbcDao">
    <property name="dataSource" ref="dataSource"/>
</bean>
```

At this point, we can just grab a reference to the DAO from Spring and call `getRandomCityByCountryCode()`.

```
    // Create the application context
    ApplicationContext ctx =
    new ClassPathXmlApplicationContext("ex2appContext.xml");
    // Obtain a reference to our DAO
    Ex2JdbcDao dao = (Ex2JdbcDao) ctx.getBean("dao");

    String countryCode = "USA";

    // Find a few random cities in the US
    for(int i = 0; i < 4; ++i)
        System.out.printf("A random city in %s is %s%n", countryCode,
            dao.getRandomCityByCountryCode(countryCode));
```

This example shows how to use Spring's JDBC classes to completely abstract away the use of traditional JDBC classes including `Connection` and `PreparedStatement`.

14.2 Transactional JDBC Access

Spring allows us to add transactions into our code without having to deal directly with the JDBC classes. For that purpose, Spring provides a transaction management package that not only replaces JDBC transaction management, but also enables declarative transaction management (configuration instead of code).

To use transactional database access, we will need to change the storage engine of the tables in the world database. The downloaded script explicitly creates MyISAM tables, which do not support transactional semantics. The InnoDB storage engine does support transactions and this is what we will be using. We can change the storage engine with the following statements.

```
ALTER TABLE City ENGINE=InnoDB;
ALTER TABLE Country ENGINE=InnoDB;
ALTER TABLE CountryLanguage ENGINE=InnoDB;
```

A good programming practice emphasized by Spring is separating interfaces and implementations. What this means is that we can create a Java interface and only use the operations on this interface

without any internal knowledge of what the actual implementation is. We will let Spring manage the implementation and with this it will manage the transactions for our implementation.

First you create a simple interface:

```
public interface Ex3Dao {
    Integer createCity(String name, String countryCode,
    String district, Integer population);
}
```

This interface contains one method that will create a new city record in the database and return the id of the new record. Next you need to create an implementation of this interface.

```
public class Ex3DaoImpl implements Ex3Dao {
    protected DataSource dataSource;
    protected SqlUpdate updateQuery;
    protected SqlFunction idQuery;

    public Integer createCity(String name, String countryCode,
        String district, Integer population) {
            updateQuery.update(new Object[] { name, countryCode,
                   district, population });
            return getLastId();
        }

    protected Integer getLastId() {
        return idQuery.run();
    }
}
```

You can see that we only operate on abstract query objects here and do not deal directly with the JDBC API. Also, this is the complete implementation. All of our transaction management will be dealt with in the configuration. To get the configuration started, we need to create the DAO.

```
<bean id="dao" class="code.Ex3DaoImpl">
    <property name="dataSource" ref="dataSource"/>
    <property name="updateQuery">...</property>
    <property name="idQuery">...</property>
</bean>
```

Now we need to set up the transaction configuration. The first thing we must do is create transaction manager to manage the data source and a specification of what transaction properties are required for the `dao` methods.

```
<bean id="transactionManager"
  class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
    <property name="dataSource" ref="dataSource"/>
</bean>

<tx:advice id="txAdvice" transaction-manager="transactionManager">
    <tx:attributes>
        <tx:method name="*"/>
    </tx:attributes>
</tx:advice>
```

The preceding code creates a transaction manager that handles transactions for the data source provided to it. The `txAdvice` uses this transaction manager and the attributes specify to create a transaction for all methods. Finally we need to apply this advice with an AOP pointcut.

```
<aop:config>
    <aop:pointcut id="daoMethods"
        expression="execution(* code.Ex3Dao.*(..))"/>
     <aop:advisor advice-ref="txAdvice" pointcut-ref="daoMethods"/>
</aop:config>
```

This basically says that all methods called on the `Ex3Dao` interface will be wrapped in a transaction. To make use of this, we only have to retrieve the `dao` from the application context and call a method on the `dao` instance.

```
Ex3Dao dao = (Ex3Dao) ctx.getBean("dao");
Integer id = dao.createCity(name,  countryCode, district, pop);
```

We can verify from this that there is no transaction management happening in our Java code and it is all configured with Spring. This is a very powerful notion and regarded as one of the most beneficial features of Spring.

14.3 Connection Pooling with Spring

In many situations, such as web applications, there will be a large number of small database transactions. When this is the case, it usually makes sense to create a pool of database connections available for web requests as needed. Although MySQL does not spawn an extra process when a connection is made, there is still a small amount of overhead to create and set up the connection. Pooling of connections also alleviates problems such as collecting large amounts of sockets in the `TIME_WAIT` state.

Setting up pooling of MySQL connections with Spring is as simple as changing the data source configuration in the application context. There are a number of configurations that we can use. The first example is based on the Jakarta Commons DBCP library. The example below replaces the source configuration that was based on `DriverManagerDataSource` with DBCP's BasicDataSource.

```
<bean id="dataSource" destroy-method="close"
  class="org.apache.commons.dbcp.BasicDataSource">
    <property name="driverClassName" value="${db.driver}"/>
    <property name="url" value="${db.jdbcurl}"/>
    <property name="username" value="${db.username}"/>
    <property name="password" value="${db.password}"/>
    <property name="initialSize" value="3"/>
</bean>
```

The configuration of the two solutions is very similar. The difference is that DBCP will pool connections to the database instead of creating a new connection every time one is requested. We have also set a parameter here called `initialSize`. This tells DBCP that we want three connections in the pool when it is created.

Chapter 15 Troubleshooting Connector/J Applications

This section explains the symptoms and resolutions for the most commonly encountered issues with applications using MySQL Connector/J.

Questions

- 15.1: When I try to connect to the database with MySQL Connector/J, I get the following exception:

  ```
  SQLException: Server configuration denies access to data source
  SQLState: 08001
  VendorError: 0
  ```

  What is going on? I can connect just fine with the MySQL command-line client.

- 15.2: My application throws an SQLException 'No Suitable Driver'. Why is this happening?

- 15.3: I'm trying to use MySQL Connector/J in an applet or application and I get an exception similar to:

  ```
  SQLException: Cannot connect to MySQL server on host:3306.
  Is there a MySQL server running on the machine/port you
  are trying to connect to?

  (java.security.AccessControlException)
  SQLState: 08S01
  VendorError: 0
  ```

- 15.4: I have a servlet/application that works fine for a day, and then stops working overnight

- 15.5: I cannot connect to the MySQL server using Connector/J, and I'm sure the connection parameters are correct.

- 15.6: Updating a table that contains a primary key that is either `FLOAT` or compound primary key that uses `FLOAT` fails to update the table and raises an exception.

- 15.7: I get an `ER_NET_PACKET_TOO_LARGE` exception, even though the binary blob size I want to insert using JDBC is safely below the `max_allowed_packet` size.

- 15.8: What should I do if I receive error messages similar to the following: “Communications link failure – Last packet sent to the server was X ms ago”?

- 15.9: Why does Connector/J not reconnect to MySQL and re-issue the statement after a communication failure instead of throwing an Exception, even though I use the `autoReconnect` connection string option?

- 15.10: How can I use 3-byte UTF8 with Connector/J?

- 15.11: How can I use 4-byte UTF8 (`utf8mb4`) with Connector/J?

- 15.12: Using `useServerPrepStmts=false` and certain character encodings can lead to corruption when inserting BLOBs. How can this be avoided?

Questions and Answers

15.1: When I try to connect to the database with MySQL Connector/J, I get the following exception:

```
SQLException: Server configuration denies access to data source
SQLState: 08001
VendorError: 0
```

What is going on? I can connect just fine with the MySQL command-line client.

Connector/J normally uses TCP/IP sockets to connect to MySQL (see Section 6.10, “Connecting Using Unix Domain Sockets” and Section 6.11, “Connecting Using Named Pipes” for exceptions). The security manager on the MySQL server uses its grant tables to determine whether a TCP/IP connection

is permitted. You must therefore add the necessary security credentials to the MySQL server for the connection by issuing a `GRANT` statement to your MySQL Server. See GRANT Statement, for more information.

> **Warning**
>
> Changing privileges and permissions improperly on MySQL can potentially cause your server installation to have non-optimal security properties.

> **Note**
>
> Testing your connectivity with the `mysql` command-line client will not work unless you add the `--host` flag, and use something other than `localhost` for the host. The `mysql` command-line client will try to use Unix domain sockets if you use the special host name `localhost`. If you are testing TCP/IP connectivity to `localhost`, use `127.0.0.1` as the host name instead.

15.2: My application throws an SQLException 'No Suitable Driver'. Why is this happening?

There are three possible causes for this error:

- The Connector/J driver is not in your `CLASSPATH`, see Chapter 4, *Connector/J Installation*.
- The format of your connection URL is incorrect, or you are referencing the wrong JDBC driver.
- When using DriverManager, the `jdbc.drivers` system property has not been populated with the location of the Connector/J driver.

15.3: I'm trying to use MySQL Connector/J in an applet or application and I get an exception similar to:

```
SQLException: Cannot connect to MySQL server on host:3306.
Is there a MySQL server running on the machine/port you
are trying to connect to?

(java.security.AccessControlException)
SQLState: 08S01
VendorError: 0
```

Either you're running an Applet, your MySQL server has been installed with the `skip_networking` system variable enabled, or your MySQL server has a firewall sitting in front of it.

Applets can only make network connections back to the machine that runs the web server that served the .class files for the applet. This means that MySQL must run on the same machine (or you must have some sort of port re-direction) for this to work. This also means that you will not be able to test applets from your local file system, but must always deploy them to a web server.

Connector/J normally uses TCP/IP sockets to connect to MySQL (see Section 6.10, “Connecting Using Unix Domain Sockets” and Section 6.11, “Connecting Using Named Pipes” for exceptions). TCP/IP communication with MySQL can be affected by the `skip_networking` system variable or the server firewall. If MySQL has been started with `skip_networking` enabled, you need to comment it out in the file `/etc/mysql/my.cnf` or `/etc/my.cnf` for TCP/IP connections to work. (Note that your server configuration file might also exist in the `data` directory of your MySQL server, or somewhere else, depending on how MySQL was compiled; binaries created by Oracle always look for `/etc/my.cnf` and *`datadir`*`/my.cnf`; see Using Option Files for details.) If your MySQL server has been firewalled, you will need to have the firewall configured to allow TCP/IP connections from the host where your Java code is running to the MySQL server on the port that MySQL is listening to (by default, 3306).

15.4: I have a servlet/application that works fine for a day, and then stops working overnight

MySQL closes connections after 8 hours of inactivity. You either need to use a connection pool that handles stale connections or use the `autoReconnect` parameter (see Section 6.3, “Configuration Properties”).

Also, catch `SQLExceptions` in your application and deal with them, rather than propagating them all the way until your application exits. This is just good programming practice. MySQL Connector/J will set the `SQLState` (see `java.sql.SQLException.getSQLState()` in your API docs) to `08S01` when it encounters network-connectivity issues during the processing of a query. Attempt to reconnect to MySQL at this point.

The following (simplistic) example shows what code that can handle these exceptions might look like:

Example 15.1 Connector/J: Example of transaction with retry logic

```
public void doBusinessOp() throws SQLException {
    Connection conn = null;
    Statement stmt = null;
    ResultSet rs = null;

    //
    // How many times do you want to retry the transaction
    // (or at least _getting_ a connection)?
    //
    int retryCount = 5;

    boolean transactionCompleted = false;

    do {
        try {
            conn = getConnection(); // assume getting this from a
                                    // javax.sql.DataSource, or the
                                    // java.sql.DriverManager

            conn.setAutoCommit(false);

            //
            // Okay, at this point, the 'retry-ability' of the
            // transaction really depends on your application logic,
            // whether or not you're using autocommit (in this case
            // not), and whether you're using transactional storage
            // engines
            //
            // For this example, we'll assume that it's _not_ safe
            // to retry the entire transaction, so we set retry
            // count to 0 at this point
            //
            // If you were using exclusively transaction-safe tables,
            // or your application could recover from a connection going
            // bad in the middle of an operation, then you would not
            // touch 'retryCount' here, and just let the loop repeat
            // until retryCount == 0.
            //
            retryCount = 0;

            stmt = conn.createStatement();

            String query = "SELECT foo FROM bar ORDER BY baz";

            rs = stmt.executeQuery(query);

            while (rs.next()) {
            }

            rs.close();
            rs = null;

            stmt.close();
            stmt = null;

            conn.commit();
            conn.close();
            conn = null;

            transactionCompleted = true;
        } catch (SQLException sqlEx) {
```

```
            //
            // The two SQL states that are 'retry-able' are 08S01
            // for a communications error, and 40001 for deadlock.
            //
            // Only retry if the error was due to a stale connection,
            // communications problem or deadlock
            //

            String sqlState = sqlEx.getSQLState();

            if ("08S01".equals(sqlState) || "40001".equals(sqlState)) {
                retryCount -= 1;
            } else {
                retryCount = 0;
            }
        } finally {
            if (rs != null) {
                try {
                    rs.close();
                } catch (SQLException sqlEx) {
                    // You'd probably want to log this...
                }
            }

            if (stmt != null) {
                try {
                    stmt.close();
                } catch (SQLException sqlEx) {
                    // You'd probably want to log this as well...
                }
            }

            if (conn != null) {
                try {
                    //
                    // If we got here, and conn is not null, the
                    // transaction should be rolled back, as not
                    // all work has been done

                    try {
                        conn.rollback();
                    } finally {
                        conn.close();
                    }
                } catch (SQLException sqlEx) {
                    //
                    // If we got an exception here, something
                    // pretty serious is going on, so we better
                    // pass it up the stack, rather than just
                    // logging it...

                    throw sqlEx;
                }
            }
        }
    } while (!transactionCompleted && (retryCount > 0));
}
```

> **Note**
>
> Use of the `autoReconnect` option is not recommended because there is no safe method of reconnecting to the MySQL server without risking some corruption of the connection state or database state information. Instead, use a connection pool, which will enable your application to connect to the MySQL server using an available connection from the pool. The `autoReconnect` facility is deprecated, and may be removed in a future release.

15.5: I cannot connect to the MySQL server using Connector/J, and I'm sure the connection parameters are correct.

Make sure that the `skip_networking` system variable has not been enabled on your server. Connector/J must be able to communicate with your server over TCP/IP; named sockets are not supported. Also ensure that you are not filtering connections through a firewall or other network security system. For more information, see Can't connect to [local] MySQL server.

15.6: Updating a table that contains a primary key that is either `FLOAT` or compound primary key that uses `FLOAT` fails to update the table and raises an exception.

Connector/J adds conditions to the `WHERE` clause during an `UPDATE` to check the old values of the primary key. If there is no match, then Connector/J considers this a failure condition and raises an exception.

The problem is that rounding differences between supplied values and the values stored in the database may mean that the values never match, and hence the update fails. The issue will affect all queries, not just those from Connector/J.

To prevent this issue, use a primary key that does not use `FLOAT`. If you have to use a floating point column in your primary key, use `DOUBLE` or `DECIMAL` types in place of `FLOAT`.

15.7: I get an `ER_NET_PACKET_TOO_LARGE` exception, even though the binary blob size I want to insert using JDBC is safely below the `max_allowed_packet` size.

This is because the `hexEscapeBlock()` method in `com.mysql.cj.AbstractPreparedQuery.streamToBytes()` may almost double the size of your data.

15.8: What should I do if I receive error messages similar to the following: “Communications link failure – Last packet sent to the server was X ms ago”?

Generally speaking, this error suggests that the network connection has been closed. There can be several root causes:

- Firewalls or routers may clamp down on idle connections (the MySQL client/server protocol does not ping).
- The MySQL Server may be closing idle connections that exceed the `wait_timeout` or `interactive_timeout` threshold.

Although network connections can be volatile, the following can be helpful in avoiding problems:

- Ensure connections are valid when used from the connection pool. Use a query that starts with `/* ping */` to execute a lightweight ping instead of full query. Note, the syntax of the ping needs to be exactly as specified here.
- Minimize the duration a connection object is left idle while other application logic is executed.
- Explicitly validate the connection before using it if the connection has been left idle for an extended period of time.
- Ensure that `wait_timeout` and `interactive_timeout` are set sufficiently high.
- Ensure that `tcpKeepalive` is enabled.
- Ensure that any configurable firewall or router timeout settings allow for the maximum expected connection idle time.

> **Note**
>
> Do not expect to be able to reuse a connection without problems if it has being lying idle for a period. If a connection is to be reused after being idle for any length of time, ensure that you explicitly test it before reusing it.

15.9: Why does Connector/J not reconnect to MySQL and re-issue the statement after a communication failure instead of throwing an Exception, even though I use the `autoReconnect` connection string option?

There are several reasons for this. The first is transactional integrity. The MySQL Reference Manual states that “there is no safe method of reconnecting to the MySQL server without risking some corruption of the connection state or database state information”. Consider the following series of statements for example:

```
conn.createStatement().execute(
  "UPDATE checking_account SET balance = balance - 1000.00 WHERE customer='Smith'");
conn.createStatement().execute(
  "UPDATE savings_account SET balance = balance + 1000.00 WHERE customer='Smith'");
conn.commit();
```

Consider the case where the connection to the server fails after the `UPDATE` to `checking_account`. If no exception is thrown, and the application never learns about the problem, it will continue executing. However, the server did not commit the first transaction in this case, so that will get rolled back. But execution continues with the next transaction, and increases the `savings_account` balance by 1000. The application did not receive an exception, so it continued regardless, eventually committing the second transaction, as the commit only applies to the changes made in the new connection. Rather than a transfer taking place, a deposit was made in this example.

Note that running with `autocommit` enabled does not solve this problem. When Connector/J encounters a communication problem, there is no means to determine whether the server processed the currently executing statement or not. The following theoretical states are equally possible:

- The server never received the statement, and therefore no related processing occurred on the server.
- The server received the statement, executed it in full, but the response was not received by the client.

If you are running with `autocommit` enabled, it is not possible to guarantee the state of data on the server when a communication exception is encountered. The statement may have reached the server, or it may not. All you know is that communication failed at some point, before the client received confirmation (or data) from the server. This does not only affect `autocommit` statements though. If the communication problem occurred during `Connection.commit()`, the question arises of whether the transaction was committed on the server before the communication failed, or whether the server received the commit request at all.

The second reason for the generation of exceptions is that transaction-scoped contextual data may be vulnerable, for example:

- Temporary tables.
- User-defined variables.
- Server-side prepared statements.

These items are lost when a connection fails, and if the connection silently reconnects without generating an exception, this could be detrimental to the correct execution of your application.

In summary, communication errors generate conditions that may well be unsafe for Connector/J to simply ignore by silently reconnecting. It is necessary for the application to be notified. It is then for the application developer to decide how to proceed in the event of connection errors and failures.

15.10: How can I use 3-byte UTF8 with Connector/J?

Because there is no Java-style character set name for `utfmb3` that you can use with the connection option `charaterEncoding`, the only way to use `utf8mb3` as your connection character set

is to use a `utf8mb3` collation (for example, `utf8_general_ci`) for the connection option `connectionCollation`, which forces a `utf8mb3` character set to be used. See Section 6.7, “Using Character Sets and Unicode” for details.

15.11: How can I use 4-byte UTF8 (`utf8mb4`) with Connector/J?

To use 4-byte UTF8 with Connector/J configure the MySQL server with `character_set_server=utf8mb4`. Connector/J will then use that setting, if `characterEncoding` and `connectionCollation` have not been set in the connection string. This is equivalent to autodetection of the character set. See Section 6.7, “Using Character Sets and Unicode” for details. You can use `characterEncoding=UTF-8` to use `utf8mb4`, even if `character_set_server` on the server has been set to something else.

15.12: Using `useServerPrepStmts=false` and certain character encodings can lead to corruption when inserting BLOBs. How can this be avoided?

When using certain character encodings, such as SJIS, CP932, and BIG5, it is possible that BLOB data contains characters that can be interpreted as control characters, for example, backslash, '\'. This can lead to corrupted data when inserting BLOBs into the database. There are two things that need to be done to avoid this:

1. Set the connection string option `useServerPrepStmts` to `true`.
2. Set `SQL_MODE` to `NO_BACKSLASH_ESCAPES`.

Chapter 16 Known Issues and Limitations

The following are some known issues and limitations for MySQL Connector/J:

- When Connector/J retrieves timestamps for a daylight saving time (DST) switch day using the `getTimeStamp()` method on the result set, some of the returned values might be wrong. In order to avoid such errors, we recommend setting a connection time zone that uses a monotonic clock by, for example, setting `connectionTimeZone=UTC`, and configuring other date-time connection properties according to your needs; see Section 6.6, “Handling of Date-Time Values” for details.
- The functionality of the property `elideSetAutoCommits` has been disabled due to Bug# 66884. Any value given for the property is ignored by Connector/J.
- MySQL Server uses a proleptic Gregorian calendar internally. However, Connector/J uses `java.sql.Date`, which is non-proleptic. Therefore, when setting and retrieving dates that were before the Julian-Gregorian cutover (October 15, 1582) using the `PreparedStatement` methods, always supply explicitly a proleptic Gregorian calendar to the `setDate()` and `getDate()` methods, in order to avoid possible errors with dates stored to and calculated by the server.
- To use Windows named pipes for connections, the MySQL Server that Connector/J wants to connect to must be started with the system variable `named_pipe_full_access_group`; see Section 6.11, “Connecting Using Named Pipes” for details.

Chapter 17 Connector/J Support

Table of Contents

17.1 Connector/J Community Support 159
17.2 How to Report Connector/J Bugs or Problems 159

17.1 Connector/J Community Support

You can join the `#connectors` channel in the MySQL Community Slack workspace, where you can get help directly from MySQL developers and other users.

17.2 How to Report Connector/J Bugs or Problems

The normal place to report bugs is http://bugs.mysql.com/, which is the address for our bugs database. This database is public, and can be browsed and searched by anyone. If you log in to the system, you will also be able to enter new reports.

If you find a sensitive security bug in MySQL Server, please let us know immediately by sending an email message to `<secalert_us@oracle.com>`. Exception: Support customers should report all problems, including security bugs, to Oracle Support at http://support.oracle.com/.

Writing a good bug report takes patience, but doing it right the first time saves time both for us and for yourself. A good bug report, containing a full test case for the bug, makes it very likely that we will fix sooner rather than later.

This section will help you write your report correctly so that you do not waste your time doing things that may not help us much or at all.

If you have a repeatable bug report, please report it to the bugs database at http://bugs.mysql.com/. Any bug that we are able to repeat has a high chance of being fixed sooner rather than later.

To report other problems, you can use one of the MySQL mailing lists.

Remember that it is possible for us to respond to a message containing too much information, but not to one containing too little. People often omit facts because they think they know the cause of a problem and assume that some details do not matter.

A good principle is this: If you are in doubt about stating something, state it. It is faster and less troublesome to write a couple more lines in your report than to wait longer for the answer if we must ask you to provide information that was missing from the initial report.

The most common errors made in bug reports are (a) not including the version number of Connector/J or MySQL used, and (b) not fully describing the platform on which Connector/J is installed (including the JVM version, and the platform type and version number that MySQL itself is installed on).

This is highly relevant information, and in 99 cases out of 100, the bug report is useless without it. Very often we get questions like, “Why doesn't this work for me?” Then we find that the feature requested was not implemented in that MySQL version, or that a bug described in a report has already been fixed in newer MySQL versions.

Sometimes the error is platform-dependent; in such cases, it is next to impossible for us to fix anything without knowing the operating system and the version number of the platform.

If at all possible, create a repeatable, standalone testcase that doesn't involve any third-party classes.

To streamline this process, we ship a base class for testcases with Connector/J, named '`com.mysql.cj.jdbc.util.BaseBugReport`'. To create a testcase for Connector/J using this

class, create your own class that inherits from `com.mysql.cj.jdbc.util.BaseBugReport` and override the methods `setUp()`, `tearDown()` and `runTest()`.

In the `setUp()` method, create code that creates your tables, and populates them with any data needed to demonstrate the bug.

In the `runTest()` method, create code that demonstrates the bug using the tables and data you created in the `setUp` method.

In the `tearDown()` method, drop any tables you created in the `setUp()` method.

In any of the above three methods, use one of the variants of the `getConnection()` method to create a JDBC connection to MySQL:

- `getConnection()` - Provides a connection to the JDBC URL specified in `getUrl()`. If a connection already exists, that connection is returned, otherwise a new connection is created.
- `getNewConnection()` - Use this if you need to get a new connection for your bug report (that is, there is more than one connection involved).
- `getConnection(String url)` - Returns a connection using the given URL.
- `getConnection(String url, Properties props)` - Returns a connection using the given URL and properties.

If you need to use a JDBC URL that is different from 'jdbc:mysql:///test', override the method `getUrl()` as well.

Use the `assertTrue(boolean expression)` and `assertTrue(String failureMessage, boolean expression)` methods to create conditions that must be met in your testcase demonstrating the behavior you are expecting (vs. the behavior you are observing, which is why you are most likely filing a bug report).

Finally, create a `main()` method that creates a new instance of your testcase, and calls the `run` method:

```
public static void main(String[] args) throws Exception {
      new MyBugReport().run();
 }
```

Once you have finished your testcase, and have verified that it demonstrates the bug you are reporting, upload it with your bug report to http://bugs.mysql.com/.

Index

, 93, 93

A

allowLoadLocalInfile connection property, 43
allowLoadLocalInfileInPath connection property, 43
allowMultiQueries connection property, 43
allowNanAndInf connection property, 45
allowPublicKeyRetrieval connection property, 40
allowReplicaDownConnections connection property, 54
allowSourceDownConnections connection property, 54
allowUrlInLocalInfile connection property, 44
alwaysSendSetIsolation connection property, 59
Authentication Methods
  Kerberos, 92
  PAM, 91
authenticationFidoCallbackHandler connection property, 34
authenticationPlugins connection property, 33
authenticationWebAuthnCallbackHandler connection property, 34
autoClosePStmtStreams connection property, 45
autoGenerateTestcaseScript connection property, 64
autoReconnect connection property, 53
autoReconnectForPools connection property, 53
autoSlowLog connection property, 63

B

blobsAreStrings connection property, 49
blobSendChunkSize connection property, 49

C

cacheCallableStmts connection property, 59
cacheDefaultTimeZone connection property, 44
cachePrepStmts connection property, 59
cacheResultSetMetadata connection property, 59
cacheServerConfiguration connection property, 60
callableStmtCacheSize connection property, 58
character sets
  with Connector/J, 80
characterEncoding connection property, 37
characterSetResults connection property, 37
client-side failover, 124
clientCertificateKeyStorePassword connection property, 42
clientCertificateKeyStoreType connection property, 41
clientCertificateKeyStoreUrl connection property, 41
clientInfoProvider connection property, 35
clobberStreamingResults connection property, 46
clobCharacterEncoding connection property, 49
compatibility information, 3
compensateOnDuplicateKeyUpdateCounts connection property, 45
connecting
  through JDBC and Connector/J, 22
  with Unix domain socket, 90
  with Windows named pipes, 90, 157
connection pooling, 117, 147
connection properties, 25
connection URL, 22
connectionAttributes connection property, 34
connectionCollation connection property, 37
connectionLifecycleInterceptors connection property, 34
connectionTimeZone connection property, 50, 75
Connector/J
  known issues, 157
  limitations, 157
  reporting problems, 159
  troubleshooting, 149
connectTimeout connection property, 38
continueBatchOnError connection property, 44
createDatabaseIfNotExist connection property, 35
customCharsetMapping connection property, 37

D

databaseTerm connection property, 35
defaultAuthenticationPlugin connection property, 33
defaultFetchSize connection property, 60
detectCustomCollations connection property, 35
disabledAuthenticationPlugins connection property, 33
disconnectOnExpiredPasswords connection property, 36
DNS SRV records, 22, 98
dnsSrv connection property, 38
dontCheckOnDuplicateKeyUpdateInSQL connection property, 60
dontTrackOpenResources connection property, 45
dumpQueriesOnException connection property, 64

E

elideSetAutoCommits connection property, 60
emptyStringsConvertToZero connection property, 47
emulateLocators connection property, 49
emulateUnsupportedPstmts connection property, 46
enableEscapeProcessing connection property, 60
enablePacketDebug connection property, 64
enableQueryTimeouts connection property, 60
error codes, 101
ER_ABORTING_CONNECTION, 101
ER_ACCESS_DENIED_ERROR, 101
ER_BAD_FIELD_ERROR, 101
ER_BAD_HOST_ERROR, 101
ER_BAD_TABLE_ERROR, 101
ER_BLOBS_AND_NO_TERMINATED, 101
ER_BLOB_CANT_HAVE_DEFAULT, 101
ER_BLOB_KEY_WITHOUT_LENGTH, 101
ER_BLOB_USED_AS_KEY, 101
ER_CANT_DO_THIS_DURING_AN_TRANSACTION, 101
ER_CANT_DROP_FIELD_OR_KEY, 101
ER_CANT_REMOVE_ALL_FIELDS, 101
ER_CANT_USE_OPTION_HERE, 101

ER_CHECK_NOT_IMPLEMENTED, 101
ER_CHECK_NO_SUCH_TABLE, 101
ER_COLLATION_CHARSET_MISMATCH, 101
ER_COLUMNACCESS_DENIED_ERROR, 101
ER_CONNECT_TO_SOURCE, 101
ER_CON_COUNT_ERROR, 101
ER_DBACCESS_DENIED_ERROR, 101
ER_DERIVED_MUST_HAVE_ALIAS, 101
ER_DUP_ENTRY, 101
ER_DUP_FIELDNAME, 101
ER_DUP_KEY, 101
ER_DUP_KEYNAME, 101
ER_DUP_UNIQUE, 101
ER_EMPTY_QUERY, 101
ER_FIELD_SPECIFIED_TWICE, 101
ER_FORCING_CLOSE, 101
ER_GRANT_WRONG_HOST_OR_USER, 101
ER_HANDSHAKE_ERROR, 101
ER_HOST_IS_BLOCKED, 101
ER_HOST_NOT_PRIVILEGED, 101
ER_ILLEGAL_GRANT_FOR_TABLE, 101
ER_ILLEGAL_REFERENCE, 101
ER_INVALID_DEFAULT, 101
ER_INVALID_USE_OF_NULL, 101
ER_IPSOCK_ERROR, 101
ER_KEY_COLUMN_DOES_NOT_EXITS, 101
ER_LOCK_DEADLOCK, 101
ER_LOCK_WAIT_TIMEOUT, 101
ER_MIX_OF_GROUP_FUNC_AND_FIELDS, 101
ER_MULTIPLE_PRI_KEY, 101
ER_NET_ERROR_ON_WRITE, 101
ER_NET_FCNTL_ERROR, 101
ER_NET_PACKETS_OUT_OF_ORDER, 101
ER_NET_PACKET_TOO_LARGE, 101
ER_NET_READ_ERROR, 101
ER_NET_READ_ERROR_FROM_PIPE, 101
ER_NET_READ_INTERRUPTED, 101
ER_NET_UNCOMPRESS_ERROR, 101
ER_NET_WRITE_INTERRUPTED, 101
ER_NEW_ABORTING_CONNECTION, 101
ER_NONEXISTING_GRANT, 101
ER_NONEXISTING_TABLE_GRANT, 101
ER_NONUNIQ_TABLE, 101
ER_NON_UNIQ_ERROR, 101
ER_NOT_ALLOWED_COMMAND, 101
ER_NOT_SUPPORTED_AUTH_MODE, 101
ER_NOT_SUPPORTED_YET, 101
ER_NO_DEFAULT, 101
ER_NO_PERMISSION_TO_CREATE_USER, 101
ER_NO_REFERENCED_ROW, 101
ER_NO_SUCH_INDEX, 101
ER_NO_SUCH_TABLE, 101
ER_NULL_COLUMN_IN_INDEX, 101
ER_OPERAND_COLUMNS, 101
ER_OUTOFMEMORY, 101
ER_OUT_OF_SORTMEMORY, 101
ER_PARSE_ERROR, 101
ER_PASSWORD_ANONYMOUS_USER, 101
ER_PASSWORD_NOT_ALLOWED, 101
ER_PASSWORD_NO_MATCH, 101
ER_PRIMARY_CANT_HAVE_NULL, 101
ER_READ_ONLY_TRANSACTION, 101
ER_REGEXP_ERROR, 101
ER_REQUIRES_PRIMARY_KEY, 101
ER_ROW_IS_REFERENCED, 101
ER_SELECT_REDUCED, 101
ER_SERVER_SHUTDOWN, 101
ER_SOURCE_NET_READ, 101
ER_SOURCE_NET_WRITE, 101
ER_SPATIAL_CANT_HAVE_NULL, 101
ER_SUBQUERY_NO_1_ROW, 101
ER_SYNTAX_ERROR, 101
ER_TABLEACCESS_DENIED_ERROR, 101
ER_TABLENAME_NOT_ALLOWED_HERE, 101
ER_TABLE_CANT_HANDLE_AUTO_INCREMENT, 101
ER_TABLE_CANT_HANDLE_BLOB, 101
ER_TABLE_EXISTS_ERROR, 101
ER_TABLE_MUST_HAVE_COLUMNS, 101
ER_TOO_BIG_FIELDLENGTH, 101
ER_TOO_BIG_ROWSIZE, 101
ER_TOO_BIG_SELECT, 101
ER_TOO_LONG_IDENT, 101
ER_TOO_LONG_KEY, 101
ER_TOO_LONG_STRING, 101
ER_TOO_MANY_KEYS, 101
ER_TOO_MANY_KEY_PARTS, 101
ER_TOO_MANY_ROWS, 101
ER_TOO_MANY_USER_CONNECTIONS, 101
ER_UNKNOWN_CHARACTER_SET, 101
ER_UNKNOWN_COM_ERROR, 101
ER_UNKNOWN_PROCEDURE, 101
ER_UNKNOWN_STORAGE_ENGINE, 101
ER_UNKNOWN_TABLE, 101
ER_UNSUPPORTED_EXTENSION, 101
ER_USER_LIMIT_REACHED, 101
ER_WARN_DATA_OUT_OF_RANGE, 101
ER_WARN_DATA_TRUNCATED, 101
ER_WARN_NULL_TO_NOTNULL, 101
ER_WARN_TOO_FEW_RECORDS, 101
ER_WARN_TOO_MANY_RECORDS, 101
ER_WRONG_AUTO_KEY, 101
ER_WRONG_COLUMN_NAME, 101
ER_WRONG_DB_NAME, 101
ER_WRONG_FIELD_SPEC, 101
ER_WRONG_FIELD_TERMINATORS, 101
ER_WRONG_FIELD_WITH_GROUP, 101
ER_WRONG_FK_DEF, 101
ER_WRONG_GROUP_FIELD, 101
ER_WRONG_KEY_COLUMN, 101
ER_WRONG_NAME_FOR_CATALOG, 101
ER_WRONG_NAME_FOR_INDEX, 101
ER_WRONG_NUMBER_OF_COLUMNS_IN_SELECT, 101
ER_WRONG_OUTER_JOIN, 101

ER_WRONG_PARAMCOUNT_TO_PROCEDURE, 101
ER_WRONG_SUM_SELECT, 101
ER_WRONG_TABLE_NAME, 101
ER_WRONG_TYPE_FOR_VAR, 101
ER_WRONG_VALUE_COUNT, 101
ER_WRONG_VALUE_COUNT_ON_ROW, 101
ER_WRONG_VALUE_FOR_VAR, 101
exceptionInterceptors connection property, 64
explainSlowQueries connection property, 63

F

failover
  Java clients, 121
failOverReadOnly connection property, 53
fallbackToSystemKeyStore connection property, 42
fallbackToSystemTrustStore connection property, 41
Fast Identity Online (FIDO) authentication, 93
fipsCompliantJsse connection property, 42
forceConnectionTimeZoneToSession connection property, 51, 75
fractional seconds connection property, 80
functionsNeverReturnBlobs connection property, 50

G

gatherPerfMetrics connection property, 63
generateSimpleParameterMetadata connection property, 46
getProceduresReturnsFunctions connection property, 48

H

ha.enableJMX connection property, 54
ha.loadBalanceStrategy connection property, 55
holdResultsOpenOverStatementClose connection property, 47

I

ignoreNonTxTables connection property, 65
includeInnodbStatusInDeadlockExceptions connection property, 65
includeThreadDumpInDeadlockExceptions connection property, 65
includeThreadNamesAsStatementComment connection property, 65
initialTimeout connection property, 54
Installing Connector/J
  With binary distribution, 7
  With Maven dependencies, 9
interactiveClient connection property, 36

J

J2EE
  connection pooling, 117
  load balancing, 124
JDBC
  and MySQL data types, 73
  background information for Connector/J, 107
  character sets, 80
  CLASSPATH, 8
  code examples, 19
  compatibility, 70
  configuration properties, 25
  driver for MySQL, 1
  SQLState codes, 101
  troubleshooting, 149, 157
  versions supported, 3
jdbcCompliantTruncation connection property, 47
JSON
  scheme validation, 134

K

Kerberos authentication
  with Connector/J, 92
KeyManagerFactoryProvider connection property, 43
keyStoreProvider connection property, 43
known issues
  Connector/J, 157

L

largeRowSizeThreshold connection property, 61
ldapServerHostname connection property, 33
limitations
  Connector/J, 157
load balancing
  with Connector/J, 124, 127
loadBalanceAutoCommitStatementRegex connection property, 56
loadBalanceAutoCommitStatementThreshold connection property, 56
loadBalanceBlocklistTimeout connection property, 56
loadBalanceConnectionGroup connection property, 56
loadBalanceExceptionChecker connection property, 56
loadBalanceHostRemovalGracePeriod connection property, 55
loadBalancePingTimeout connection property, 56
loadBalanceSQLExceptionSubclassFailover connection property, 56
loadBalanceSQLStateFailover connection property, 57
loadBalanceValidateConnectionOnSwapServer connection property, 57
localSocketAddress connection property, 38
locatorFetchBufferSize connection property, 50
logger connection property, 62
loggers, 139
logging, 139
logSlowQueries connection property, 62
logXaCommands connection property, 63

M

maintainTimeStats connection property, 59
maxAllowedPacket connection property, 39
maxByteArrayAsHex connection property, 62
maxQuerySizeToLog connection property, 62

maxReconnects connection property, 53
maxRows connection property, 47
metadataCacheSize connection property, 58
multi-host connections
  with Connector/J, 121
multifactor authentication (MFA), 93

N

named pipes, 90, 157
netTimeoutForStreamingResults connection property, 47
noAccessToProcedureBodies connection property, 48
noDatetimeStringSync connection property, 51
nullDatabaseMeansCurrent connection property, 49

O

ociConfigFile connection property, 34
ociConfigProfile connection property, 34
overrideSupportsIntegrityEnhancementFacility connection property, 65

P

packetDebugBufferSize connection property, 64
padCharsWithSpace connection property, 47
PAM authentication, 91
paranoid connection property, 40
password connection property, 32
password1 connection property, 32
password2 connection property, 33
password3 connection property, 33
passwordCharacterEncoding connection property, 36
passwords, 93
pedantic connection property, 66
pinGlobalTxToPhysicalConnection connection property, 57
populateInsertRowWithDefaultValues connection property, 47
prepStmtCacheSize connection property, 58
prepStmtCacheSqlLimit connection property, 58
preserveInstant connection property, 75
preserveInstants connection property, 51
processEscapeCodesForPrepStmts connection property, 46
profilerEventHandler connection property, 62
profileSQL connection property, 62
proleptic Gregorian calendar, 157
propertiesTransform connection property, 36

Q

queriesBeforeRetrySource connection property, 54
query attributes, 83
queryInfoCacheFactory connection property, 58
queryInterceptors connection property, 45
queryTimeoutKillsConnection connection property, 45

R

readFromSourceWhenNoReplicas connection property, 55
readOnlyPropagatesToServer connection property, 61
reconnectAtTxEnd connection property, 53
replication
  with Connector/J, 127
replicationConnectionGroup connection property, 57
reportMetricsIntervalMillis connection property, 63
requireSSL connection property, 44
resourceId connection property, 57
resultSetSizeThreshold connection property, 64
retriesAllDown connection property, 53
rewriteBatchedStatements connection property, 61
rollbackOnPooledClose connection property, 36

S

scrollTolerantForwardOnly connection property, 48
secondsBeforeRetrySource connection property, 54
selfDestructOnPingMaxOperations connection property, 55
selfDestructOnPingSecondsLifetime connection property, 55
sendFractionalSeconds connection property, 52
sendFractionalSecondsForTime connection property, 52, 80
serverAffinityOrder connection property, 57
serverConfigCacheFactory connection property, 59
serverRSAPublicKeyFile connection property, 40
session state tracker, 99
sessionVariables connection property, 36
SLF4J, 139
slowQueryThresholdMillis connection property, 63
slowQueryThresholdNanos connection property, 63
socketFactory connection property, 38
socketTimeout connection property, 38
socksProxyHost connection property, 38
socksProxyPort connection property, 38
socksProxyRemoteDns connection property, 39
Spring framework, 143
SQLState error codes, 101
SSL, 85
sslContextProvider connection property, 43
sslMode connection property, 40
strictUpdates connection property, 48

T

tcpKeepAlive connection property, 39
tcpNoDelay connection property, 39
tcpRcvBuf connection property, 39
tcpSndBuf connection property, 39
tcpTrafficClass connection property, 39
time zone conversion, 75
tinyInt1isBit connection property, 48
tlsCiphersuites connection property, 42
tlsVersions connection property, 42
Tomcat application server, 141

traceProtocol connection property, 63
trackSessionState connection property, 37
transformedBitIsBoolean connection property, 48
treatMysqlDatetimeAsTimestamp connection property, 52
treatUtilDateAsTimestamp connection property, 52
troubleshooting
  Connector/J, 149
  JDBC SQLState codes, 101
trustCertificateKeyStorePassword connection property, 41
trustCertificateKeyStoreType connection property, 41
trustCertificateKeyStoreUrl connection property, 41
trustManagerFactoryProvider connection property, 43

U

ultraDevHack connection property, 66
Unicode
  with Connector/J, 80
Unix doman socket, 90
useAffectedRows connection property, 36
useColumnNamesInFindColumn connection property, 66
useCompression connection property, 40
useConfigs connection property, 35
useCursorFetch connection property, 59
useHostsInPrivileges connection property, 49
useInformationSchema connection property, 49
useLocalSessionState connection property, 58
useLocalTransactionState connection property, 58
useNanosForElapsedTime connection property, 62
useOldAliasMetadataBehavior connection property, 66
useOnlyServerErrorMessages connection property, 65
user connection property, 32
useReadAheadInput connection property, 61
useServerPrepStmts connection property, 46
useSSL connection property, 44
useStreamLengthsInPrepStmts connection property, 46
useUnbufferedInput connection property, 40
useUsageAdvisor connection property, 64

V

validation
  for JSON schemas, 134
verifyServerCertificate connection property, 44

W

Web Authentication (WebAuthn) authentication, 93

X

X DevAPI
  client-side failover, 124
xdevapi.auth connection property, 66
xdevapi.compression connection property, 66
xdevapi.compression-algorithms connection property, 67
xdevapi.compression-extensions connection property, 67
xdevapi.connect-timeout connection property, 68
xdevapi.connection-attributes connection property, 68
xdevapi.dns-srv connection property, 68
xdevapi.fallback-to-system-keystore connection property, 68
xdevapi.fallback-to-system-truststore connection property, 69
xdevapi.ssl-keystore connection property, 69
xdevapi.ssl-keystore-password connection property, 69
xdevapi.ssl-keystore-type connection property, 69
xdevapi.ssl-mode connection property, 69
xdevapi.ssl-truststore connection property, 69
xdevapi.ssl-truststore-password connection property, 69
xdevapi.ssl-truststore-type connection property, 69
xdevapi.tls-ciphersuites connection property, 70
xdevapi.tls-versions connection property, 70

Y

yearIsDateType connection property, 52

Z

zeroDateTimeBehavior connection property, 52